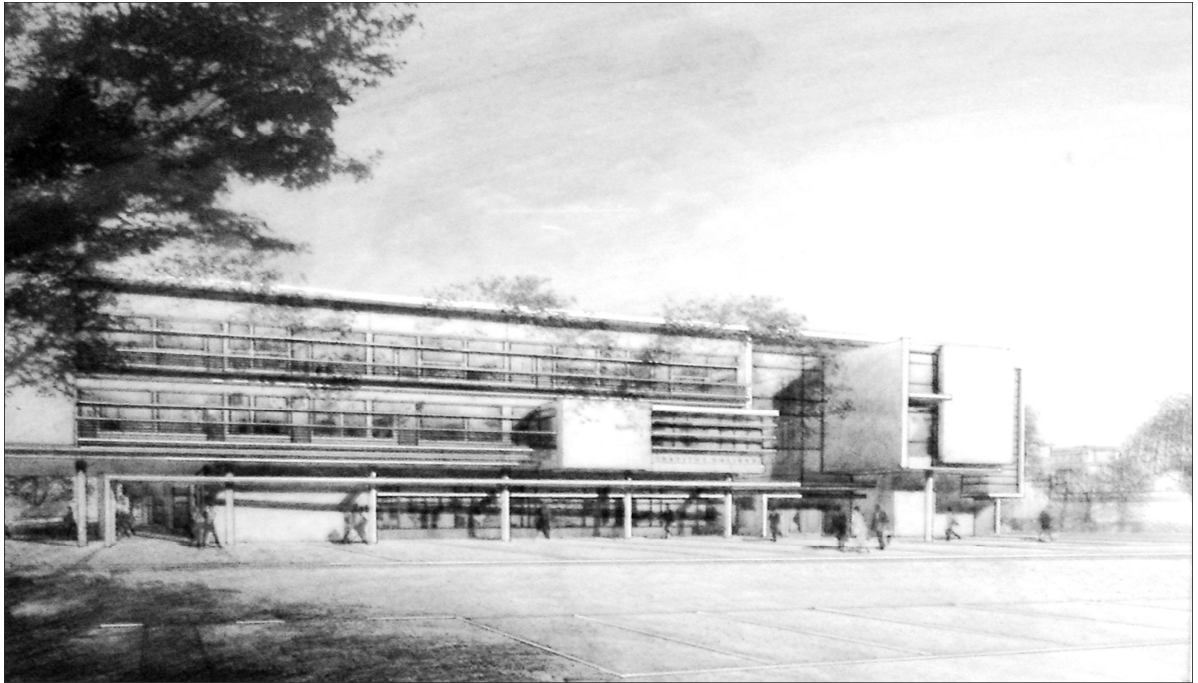




ASPECTS MATERIELS & LOGICIELS DU MICROCONTRÔLEUR PIC 16F877

RESUME DU COURS PENDANT TP PIC

THIEBAUT Bernard

METHODOLOGIE DE TRAVAIL EN TP

Trois grandes méthodes pour développer des applications.

① PAR SIMULATION

Qui ne coûte rien, pourvu qu'on dispose d'un PC
(méthode idéale étudiants qui par essence ne sont pas très riches !!!)

MPLAB (à télécharger **gratuitement** sur le site de **MICROCHIP**) est un environnement assez puissant pour créer et faire évoluer vos applications.

- avec des outils de **gestion de projets**,
- avec des outils **d'édition des fichiers sources**,
- avec des outils de **compilation/linkage** :
 - en **assembleur** :
suite **MPASM/MPLINK/MPLIB**
fournie en standard avec **MPLAB**
 - en **langage C** :
versions gratuites et compatibles avec **MPLAB**
(mais à durée limitée) existent aussi sur le net,
- avec un **simulateur** pour travailler alors dans un environnement **virtuel** reproduisant le fonctionnement du PIC de votre choix et permettant :
 - le mode d'exécution **en pas à pas** pour la mise au point,
 - la visualisation et l'évolution des **SFR** et des **GPR** sous forme symbolique grâce à des fenêtres « **watch** » : suivre les variables pendant l'exécution du programme et corriger les erreurs
(**pas les erreurs de syntaxe** détectées à la compilation, mais bien **les erreurs de logique** de votre programme !!!)
 - la simulation **d'évènements extérieurs**...

Méthode idéale pour vous muscler (pour pas cher) en PIC !!!

METHODOLOGIE DE TRAVAIL EN TP (suite)

② PAR “DEBUGAGE” SUR L’APPLICATION

Idéale pour **la mise au point** des programmes, mais outil **ICD3** nécessaire.
(méthode assez proche de ce que l’on fait dans l’industrie)

Elle nécessite en, plus de **MPLAB**, d’avoir :

- un boîtier type **ICDx** (comme **In Circuit Débugueur**),
- une carte d’évaluation avec le **PIC 16F877** : carte **PICDEM 2 PLUS**.

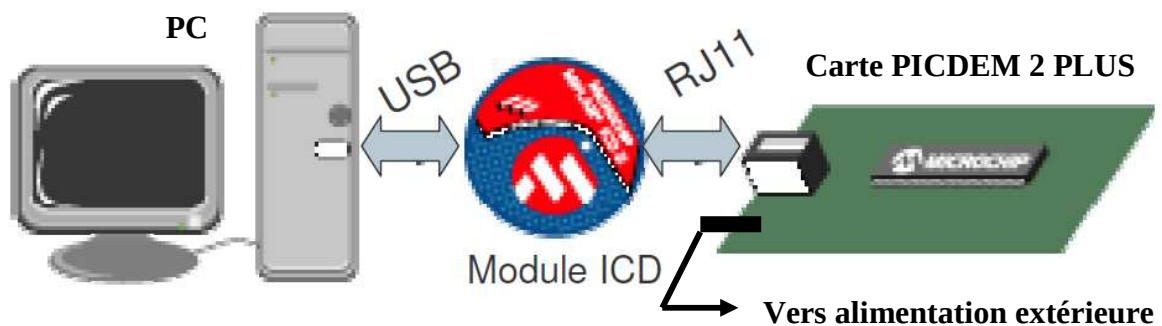


Figure VII.1. (page 214)

Le **PC** (station de travail ou **HOST**), avec tous les outils de développement : **MPLAB** (mais pas en mode simulation) : mode « **débugage** » par **ICDx**,

Le boîtier **ICDx** : puissant outil pour s’interfacer, par liaison **USB**, entre le **PC** et la carte **PICDEM 2 PLUS**. Le programme est « débogué » sur la carte applicative mais sous contrôle total du boîtier **ICDx** (mode pas à pas, fenêtres watch... pour la mise au point).

La carte **PICDEM 2 PLUS** (appelée **carte cible** ou **target board**) : reliée par liaison **RJ11** au boîtier **ICDx** et avec sa **propre alimentation** extérieure.

On est passé **du monde virtuel** (simulation) **au monde réel**.

Méthode idéale pour un débogage in situ !!!

METHODOLOGIE DE TRAVAIL EN TP (suite)

③ PAR PROGRAMMATION

Idéale pour le **stade final** d'un projet
et pour rendre votre application totalement **autonome**.

*(méthode utilisée dans la **dernière** phase de votre application)*

L'EEPROM de code du PIC est programmée avec le code machine de votre programme qui en principe a atteint une version **fiable** et **définitive** suite à la méthode précédente.

De même, **l'EEPROM de données** sera programmée avec les données (semi)permanentes de votre programme.

A partir de cet instant, on peut retirer la liaison **RJ11** entre la carte **PICDEM 2 PLUS** et le boîtier **ICDx** :

à chaque mise sous tension (ou à chaque reset)
de la carte PICDEM 2 PLUS,
c'est votre programme applicatif qui fonctionne.

Le « **cordon ombilical** »
avec les outils de développement
a été **coupé !!!**

Méthode idéale pour rendre une application autonome !!!

ELEMENTS POUR LE TUTORIAL 1

« CONCEPTION » D'UN MODULE

Ecrire en **assembleur** le module de calcul de la **valeur absolue** ayant pour **équivalent** le prototype de la fonction C suivant :

unsigned int abs (signed int i)

Cette fonction sera faite en assembleur sous la **forme d'un module**.

Pour **simplifier**, on raisonnera avec des entiers de taille **un octet**.

unsigned char abs (signed char i)

Eléments d'explications :

Il est fondamental de bien comprendre la notion de **module** qui se trouve « **au cœur** » de la programmation **modulaire**.

Méthode de **conception** (un peu toujours la même) en **4 étapes** :

① **Découpage d'un problème complexe**
en bouts de taille plus petite
(élaboration du cahier des charges de chaque module)

② **Préparation de « l'encapsulation »**
(comment le module est vu de l'extérieur)

③ **Ecriture du corps du module**
(comment le module agit à l'intérieur)

④ **Test et validation du module**
(comment finir « proprement » le module)

**① Découpage d'un problème complexe
en bouts de taille plus petite
(élaboration du cahier des charges de chaque module)**

Face à un problème **complexe**, il faut savoir repérer des **modules de base**, chacun correspondant à une tâche facilement identifiable.
(c'est la **phase d'analyse**)

En les **regroupant** ensuite, on réalise cette application plus complexe (comme dans un jeu de « Légo »).

C'est une partie délicate, car un **mauvais** découpage peut conduire à un programme final **plus difficile** à réaliser.

En TP, vous n'aurez pas (ou peu) à réfléchir sur cette étape car je vous indiquerai le (ou les) module(s) à écrire :

ce sera la **contrainte** (on dit le cahier **des charges**) à **respecter**.

Il est **important** et **impératif** que vous respectiez ce cahier des charges.

*Cette méthode de travail est un bon exemple de « **type de travail en équipe** » : chaque élément de l'équipe fait ce qu'il a à faire, sous la direction d'un **responsable de projet** qui a fait préalablement le découpage du travail. Pour que la **somme** de tous les développements de toutes les personnes de l'équipe soit **exploitable**, on conçoit bien que :*

chaque personne ne peut pas faire ce qu'il veut « dans son coin » !

Le cahier des charges est là justement pour « canaliser » le travail de chacun.

Résumé du cours PIC : chapitre V.8. (informations supplémentaires pendant manipulations TP 0)
--

Comment est fourni le cahier des charges du module à concevoir ?

**la meilleure image en est donnée
par le prototype d'une fonction en C :**

valeur_retournée nom_fonction (liste_et_type_paramètres);

On ne voit pas comment est codée « l'intérieur » de la fonction.

Mais on a tous les éléments pour savoir comment elle se comportera
« vue de l'extérieur ».

Si la programmation nécessite évidemment :

- de la **rigueur**,
- mais aussi de **l'imagination**,

il ne faut pas croire que vos capacités imaginatives soient bridées
parce qu'on vous impose un cahier des charges strict à respecter...

Votre espace de liberté, nécessaire à toute programmation, se trouvera « à l'intérieur du module » !!!
--

② Préparation de « l'encapsulation » (comment le module est vu de l'extérieur)

- pour « renseigner » le module en indiquant :
sa **documentation** : ce que fait le module et comment il le fait,

- savoir comment on l'utilise :
le (ou les) paramètre(s) éventuel(s) en entrée,

- savoir ce qu'il renvoie :
la valeur (éventuelle) retournée en sortie.

L'idée est de fournir suffisamment d'éléments à un utilisateur potentiel de ce module (qui ne l'aura pas forcément écrit) pour qu'il puisse s'en servir **proprement**. Pour cet utilisateur (qui ne sera pas forcément vous) le module apparaît comme :

une « **boîte noire** » avec des **entrées** et une **sortie**.

```
*****  
;  
; module abs : calcule la valeur absolue  
; prototype : unsigned char abs (signed char)  
; entrée : W contient le signed char dont on veut calculer  
; la valeur absolue)  
; sortie : W renvoie (retourne) la valeur absolue  
*****  
  
abs                                ; point d'entrée = nom du module  
  
.....                            ; corps du module  
.....                            ; corps du module  
  
RETURN                            ; point de sortie = retour au main  
*****
```


③ **Ecriture du corps du module** **(comment le module agit à l'intérieur)**

Remarque préliminaire importante

Bien comprendre une chose (et beaucoup d'étudiants ont du mal) :
quand on arrive au point d'entrée du module **abs**,
(première instruction du module)
W contient la valeur du char
dont on veut calculer la valeur absolue.

et il faut faire avec ça

Revenons maintenant au corps du module **abs**.

L'idée est de tester (par masquage) le **MSB** de **W** (bit de signe) :

- si **MSB = 0** : le paramètre est **positif**, on le renvoie tel quel :
$$(\text{abs}(i) = i \text{ si } i \geq 0)$$
- si **MSB = 1** : le paramètre est **strictement négatif**, on doit calculer son complément à deux avant de le renvoyer :
$$(\text{abs}(i) = -i \text{ si } i < 0)$$

Par contre, nous utiliserons une **case mémoire « tampon »** pour stocker le registre **W** avant de faire le masque. Pourquoi ?

Cette case tampon est une **variable locale** du module **abs**.

Il faut donc réserver de la place pour cette locale dans un **GPR** par :

CBLOCK 0x20 ; zone des variable à partir de 0x20

tampon : **1** ; taille de **1** octet pour **tampon**

ENDC ; fin de la zone des variables

Résumé du cours PIC : chapitre V.8. (informations supplémentaires pendant manipulations TP 0)
--

Le corps de la fonction commentée est fourni ci-dessous :

abs	MOVWF	tampon	; point d'entrée = nom du module ; sauve paramètre dans la variable ; locale tampon
	ANDLW	0x80	; masque sur MSB (bit de signe)
	BTFSC	STATUS,Z	; Z = 0 ? (MSB = 1 ?)
	GOTO	positif	; non, alors paramètre positif
negatif	COMF	tampon	; complément à un (inversion bits)
	INCF	tampon	; complément à deux (+1)
positif	MOVE	tampon,W	; retour avec valeur absolue dans W
	RETURN		; point de sortie = retour au main

④ **Test et validation du module** **(ou comment finir « proprement » le module)**

Cette étape (souvent négligée à tort par les étudiants)
est pourtant **très importante**.

Au niveau de la programmation pure, c'est certainement l'étape la plus facile, car elle consiste à écrire un main qui a la forme :

MOVLW		; passage du paramètre : mettre ici une valeur de test
CALL	abs	; appel du module abs
NOP		; au retour du module, W contient la valeur absolue ; du paramètre passé en entrée

qui est constitué de 3 parties :

- préparation du (ou des) paramètre(s) à passer au module,
 - appel du module,
 - et un « NOP » final pour indiquer le retour.
- (ce « NOP » disparaîtra bien sûr quand vous utiliserez ce module...)

Ce main n'a pas d'autre intérêt que de « tester » le module.
(lui fournir l'environnement minimal pour qu'il puisse fonctionner)

Mais cette dernière étape de test est fondamentale !!!

Car, suivant la « philosophie » de la **programmation modulaire**,
chaque module est une **brique**.

Et comment construire un mur solide
avec des briques qui n'auraient pas été testées à **150%** ?

Si vous mettez à disposition des modules non fiables,
si vous les placez en librairies,
c'est une véritable « **bombe à retardement** »
que vous abandonnez derrière vous !!!

Résumé du cours PIC : chapitre V.8. (informations supplémentaires pendant manipulations TP 0)

Un utilisateur doit avoir une **sécurité absolue** sur la fiabilité les modules (fonctions) qu'il utilise.

**Si son programme principal ne marche pas,
il cherchera les erreurs dans ce qu'il a lui-même écrit
et n'ira pas les chercher à l'intérieur
des fonctions de la bibliothèque.**

Imaginez que vous écriviez un programme en utilisant une fonction toute faite, fournie par les bibliothèques du C.

Si votre programme ne marche pas, il faut que ça vienne de ce que vous avez vous-même écrit et pas de cette fonction (à la limite cela peut venir d'une mauvaise utilisation de votre part de cette fonction, mais pas du code de celle-ci).

Sinon, comme on dit familièrement,
cela risque de vite devenir **indémer...**

Quelques conseils généraux pour la phase de test

- faire les tests avec les valeurs limites :
les « **effets de bord** » qu'on aura l'occasion de voir souvent en TP,
- être **exhaustif** : penser à utiliser toutes les valeurs possibles pour le (ou les) paramètre(s) d'un module et même celles auxquelles on n'a pas pensé !!!!,
- faire preuve de **rigueur** et d'**imagination** ...

Remarque générale

Au final et vous l'avez peut-être déjà compris, j'accorderai dans ces TP autant d'importance à la méthodologie mise en œuvre qu'aux résultats obtenus.

ELEMENTS POUR LE TUTORIAL 2

PRINCIPE DE L'ADRESSAGE INDIRECT

L'adressage **indirect** permet de mettre en œuvre un **pointeur**.

Pour accéder à une variable, on utilise un pointeur qui contient l'adresse de cette variable (c'est la valeur du pointeur).

Ce qui est différent de l'adressage direct où on accède à une variable par son nom : son adresse est dans ce cas connue par le compilateur au moment de la compilation.

Exemple en langage C

```
char * ptr ;    // déclaration d'un pointeur ptr de type char *
char i ;        // déclaration d'une variable i de type char
:
:
i = 5 ;         // exemple d'adressage direct

ptr = 0x020;    // initialisation du pointeur ptr (adressage direct)

i = *ptr;       // exemple d'adressage indirect : on initialise i avec
                // le contenu de la case pointée par ptr
                // c'est-à-dire avec le contenu la case d'adresse 0x020
```

**Comme souvent, quand on programme en C,
on ne sait pas comment ça fonctionne
au niveau du hardware (dans le PIC) !!!**

Résumé du cours PIC : chapitre VI.2.1. (informations supplémentaires pendant manipulations TP 0)

Deux registres SFR interviennent dans ce mode d'adressage :

① **FSR** (File Select Register) à l'offset **0x04** de chaque bank :

FILE	02h	FILE	82h	FILE	102h	FILE	182h
STATUS	03h	STATUS	83h	STATUS	103h	STATUS	183h
FSR	04h	FSR	84h	FSR	104h	FSR	184h
PORTA	05h	TRISA	85h		105h		185h

Il contient les **8 bits de poids faible** du **pointeur** (adresse opérande)

Or avec **8 bits**, on ne peut adresser que $2^8 = 256$ cases mémoires.

D'où le bit **IRP** (9^{ème} bit du pointeur) :

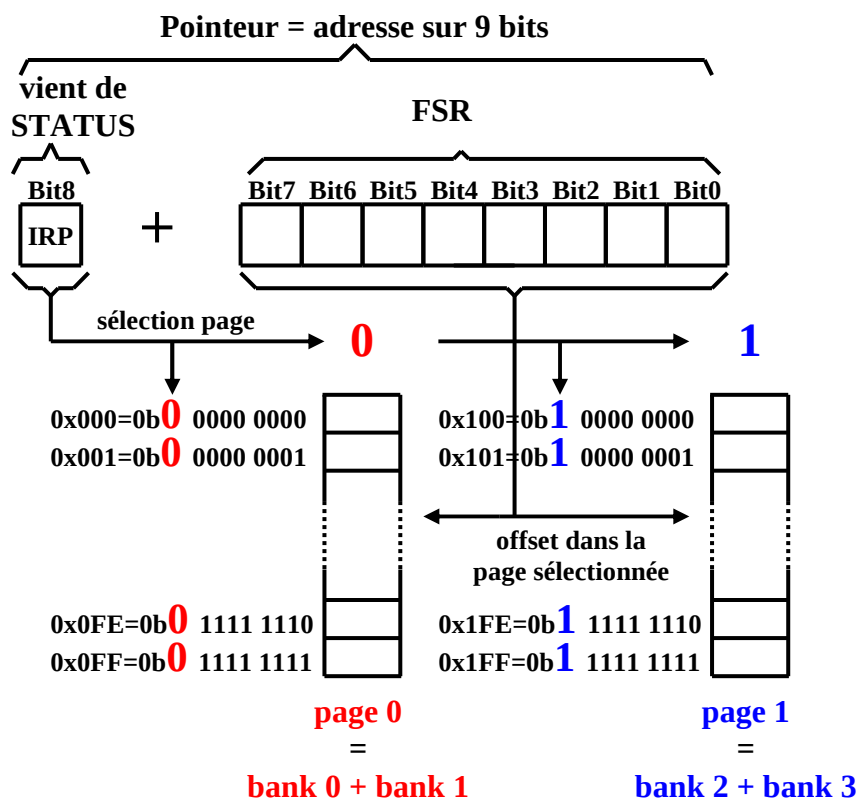


Figure III.7. (page 81)

② **Indirect.addr** (ou **INDF**, **INDirect File**) offset **0x00** de chaque bank :
il sert pour indiquer l'utilisation de **l'adressage indirect**.

Indirect addr. ^(*)	00h	Indirect addr. ^(*)	80h	Indirect addr. ^(*)	100h	Indirect addr. ^(*)	180h
TMR0	01h	OPTION_REG	81h	TMR0	101h	OPTION_REG	181h

Adressage indirect : fonctionnement

Comment se traduit cet exemple en bas niveau ?

En quatre temps :

① **Initialiser** le registre **FSR** avec l'adresse de la variable (c'est-à-dire son offset dans la page sélectionnée),

Ici, initialisation du registre **FSR** avec **0x20**

```
MOVLW    0x20    ; charge l'offset de l'adresse dans W
MOVWF    FSR     ; et le met dans FSR
           ; sans gérer RP0/RP1 : pourquoi ?
```

② **Positionner** le bit **IRP** de **STATUS** en fonction de la page dans laquelle se trouve la variable concernée (pointée),

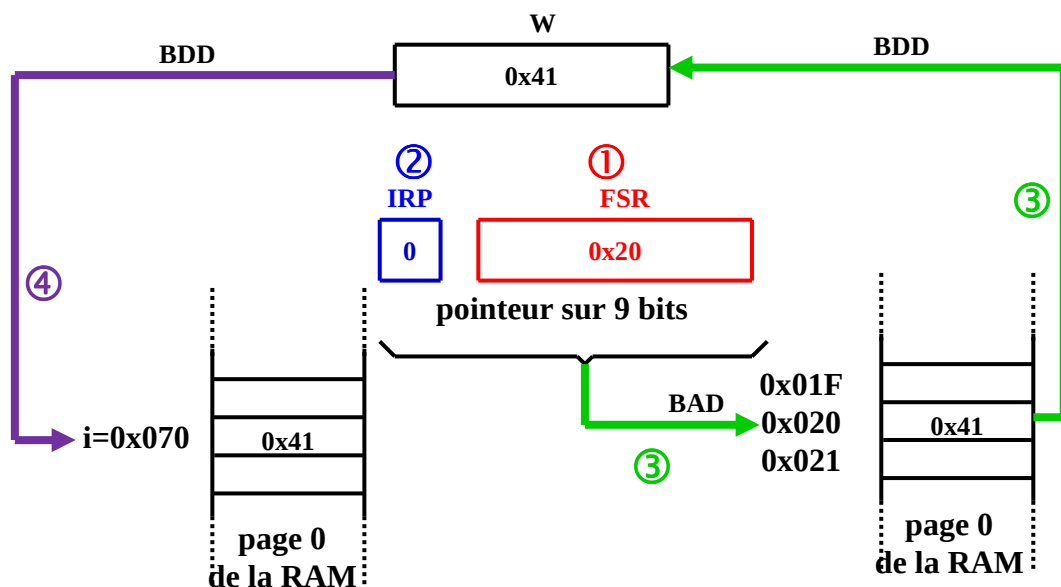
Ici, mise à **0** bit **IRP** : **BCF STATUS,IRP**

(sans gérer **RP0/RP1** : pourquoi ?)

③ **Faire un accès** (en lecture ou en écriture) sur le registre **INDF**.

Ici, pour charger le registre **W** avec le contenu de la case **0x020**

```
MOVF     INDF,W   ; lecture (indirecte) case 0x020 dans W
           ; sans gérer RP0/RP1 : pourquoi ?
```



Résumé du cours PIC : chapitre VI.2.1.

④ Et enfin, **mettre à jour** la variable **i** (par adressage direct).

Ici, le contenu de **W** est placé dans la case **0x070** (adresse de i)

MOVWF **i** ; copie de **W** dans la case de **i**
 ; sans gérer **RP0/RP1** : zone commune

Enfin, chaque fois que l'assembleur rencontrera INDF dans une instruction, il sait qu'il s'agit de l'adressage indirect de la case mémoire dont l'adresse (sur 9 bits) se trouve dans le registre FSR pour ses 8 bits de poids faible complétée par le bit IRP du registre STATUS pour son 9^{ème} bit.

Et le tutorial 2 dans tout cela ?

Il s'agit de tester un module qui calcule la longueur d'une chaîne de caractères en mémoire et qui reçoit en paramètre un pointeur vers le premier caractère de cette chaîne.

Ce qui en C donnerait :

```
int string_length (char *s)
{
    int longueur = 0;
    while(*s++) longueur++;
    return longueur;
}
```

On vous demandera d'ailleurs dans le tutorial 2 de faire le lien avec le langage C !!!

Remarques

Erreurs fréquentes avec l'adressage indirect :

- oubli d'initialiser le pointeur via **FSR** :
dans ce cas là, le registre **FSR** contient n'importe quoi et on n'accèdera pas à la case désirée,

- confusion entre la valeur du pointeur (une adresse) et la valeur de la variable pointée par ce pointeur (le contenu de la case mémoire dont l'adresse est la valeur du pointeur).

Si ptr est une variable de type pointeur, cela revient, en langage C, à confondre :

ptr et ***ptr**

- confusion entre l'adresse de la case mémoire où est stocké ce pointeur et l'adresse fournie par ce pointeur (valeur du pointeur)

Si ptr est une variable de type pointeur, cela revient, en langage C, à confondre :

&ptr et **ptr**

BIT DE PARITE P (sur caractères 7 bits)

Le **bit de parité P** est le **MSB** (bit 7) que l'on rajoute à un caractère (codé sur 7 bits et donc avec son MSB = 0) pour obtenir un nouveau caractère (sur 8 bits).

En pratique, il existe 2 types de parité :

① la parité **paire** (la plus utilisée) :

Dans ce cas, **le nombre total de bits à 1** du nouveau caractère donc y compris le bit **P**) doit être **pair**.

Pas clair, alors un exemple ?

Si le caractère au départ vaut **0x53**, soit en binaire :

0	1	0	1	0	0	1	1
MSB							

Il y a **4 bits** à 1, donc il faut mettre **P** à **0**.

Ce qui donne le nouveau caractère :

0	1	0	1	0	0	1	1
P							

Soit : **0x53** (idem)

De telle sorte que :

le nombre total de bits à 1 du nouveau caractère soit pair.

BIT DE PARITE P (suite)

Par contre, si le caractère au départ vaut **0x43**, soit en binaire :

0	1	0	0	0	0	1	1
MSB							

Il y a **3 bits** à 1, donc il faut mettre **P** à **1**.

Ce qui donne le nouveau caractère :

1	1	0	0	0	0	1	1
P							

Soit : **0xC3**

De telle sorte que :

le nombre total de bits à 1 du nouveau caractère soit pair.

② la parité **impaire** (moins fréquente) :

Dans ce cas, **le nombre total de bits à 1** du nouveau caractère donc y compris le bit **P**) doit être **impair**.

Donc cette notion de parité n'a rien à voir avec la parité au sens arithmétique du terme.

BIT DE PARITE P (suite)

A quoi ça sert ?

Souvent utilisé en **télécommunication** et dans les liaisons série particulièrement entre un émetteur et un récepteur de caractères qui utilisent tous les deux (grâce à une entente préalable) la même parité (souvent paire).

Tous les caractères seront donc émis en parité paire et le récepteur vérifie (en comptant le nombre de bits à 1 de chaque caractère) si le caractère reçu est également en parité paire.

On peut ainsi **détecter** un nombre impair d'erreurs de transmission (comme par exemple un parasite qui aurait changé l'état d'un seul bit dans un caractère) et rejeter le ou les caractère(s) en erreur (ou tout le message suivant protocole).

Peut-on, avec cette technique **corriger** une erreur sur un seul bit ?

Si non, réfléchissez...

**et proposez sur votre compte-rendu
une méthode le permettant.**

Temporisations logicielles : introduction

On a souvent besoin de placer des **temporisations** (délais, attentes) à certains instants **stratégiques** de l'exécution d'un programme.

↪ **temporisations matérielles :**

Avec un des **3 timers internes** du PIC pour mesurer le temps de façon très précise (voir TP N°3)

↪ **temporisations logicielles :**

Faites avec de **simples boucles** la plupart du temps à l'intérieur d'un module de temporisation appelé depuis le main.

Utilisées si l'application principale est non critique au niveau temps réel, l'idée étant de faire « perdre » (consommer) du temps à la CPU. Du coup, l'application ne peut pas faire autre chose pendant ce temps là. On trouvera :

- des temporisations logicielles de **durée fixe**,
Limitées car toujours la même durée, sauf si modification du code.

- des temporisations logicielles de **durée paramétrable**.

Plus souples à utiliser et donc plus intéressantes car elles s'adapteront automatiquement à la durée que l'on souhaite réaliser (grâce à un paramètre passé au module en assembleur ou à la fonction en C)

Nous allons présenter d'abord un exemple de durée fixe.

Puis, nous réaliserons en TP des temporisations de durée paramétrable que nous réutiliserons par la suite...

Résumé du cours PIC : chapitre VI.1.
(informations supplémentaires pendant manipulations TP 1)

Temporisations logicielles de durée fixe

Cas d'un module avec une simple boucle

```

;*****
;
; module tempo_test : temporisation void tempo_test (void)
; réalisé par simple boucle logicielle
;*****
tempo_test                                ; point d'entrée du module
        MOVLW 10                        ; valeur prise pour l'exemple
        MOVWF tampon                    ; variable locale de tempo_test

encore    DECFSZ tampon                  ; tampon --
          GOTO   encore                  ; temporisation non finie
          RETURN                        ; point de sortie du module

```

Figure VI.1. (page 191)

Pour calculer la durée totale d'exécution de ce module, il faut le « **dérouler mentalement** » comme s'il s'exécutait réellement sur le PIC, et **compter** le nombre de cycles consommés :

Instruction	Nombre cycles	Total
MOVLW 10	1	Exécutée une seule fois : 1
MOVWF tampon	1	Exécutée une seule fois : 1
DECFSZ tampon	1 si reste dans boucle 2 si saut hors boucle	Répétées 9 fois soit : $9 \times (2+1) = \mathbf{27}$ Quand saut hors boucle : Exécutée une seule fois : 2
GOTO encore	2 reste dans la boucle	
RETURN	2	Exécutée une seule fois : 2
Total		33

Figure VI.2. (page 191)

Temporisations logicielles de durée fixe

Cas d'une simple boucle généralisation

Si on appelle **x** la valeur littérale chargée dans **W** à la première instruction du module (ici $x=10$), alors le nombre de cycles N'_1 pris par le module **tempo_test** est de :

$$N'_1 = 3.(x-1)+6 = 3x+3 \text{ cycles.}$$

En pratique, comme le module **tempo_test** est appelé par une instruction **CALL** tempo_test depuis le programme principal (soit 2 cycles supplémentaires), il faut rajouter ces 2 cycles à N'_1 ce qui donne au final :

$$N_1 = 3x + 5 \text{ cycles}$$

pour la durée totale du module **tempo_test**
(durée totale « aller+retour »)

Or la vitesse de traitement est de 1 Mips pour un quartz à **4 MHz**.
(comme sur la carte **PICDEM 2 PLUS** utilisée en TP)

Cela signifie qu'un cycle dure dans ce cas **1 μ s**.

Et donc la durée T_1 du module **tempo_test** sera de :

$T_1 = (3x + 5) \mu s$

Remarque

On peut rajouter des instructions **NOP** (No OPeration) dans la boucle pour modifier cette formule.

C'est d'ailleurs un des intérêts de cette instruction un peu bizarre, qui ne fait rien, sinon perdre du temps CPU !!!

Temporisations logicielles de durée fixe

Cas d'une simple boucle (suite)

  **Piège : attention aux « effets de bord » !!**

On pourrait penser, au premier abord, que la valeur **maximale** $T_{1\max}$ de T_1 est obtenue pour la valeur **maximale** $x_{\max}$ de x :
W étant sur un octet, on a $x_{\max} = 0xFF = 255$.

Mais comme la variable locale **tampon** est décrémentée avant d'être testée à 0 (par l'instruction **DECFSZ tampon**), si elle vaut **0x00** avant cette instruction, après la décrémentation, elle vaudra **0xFF** ($\neq 0x00$), et donc on restera dans la boucle.

Ainsi, **bizarrement**, la valeur maximale $T_{1\max}$ de **tempo_test** est réalisée pour $x = 256$ (dans la formule précédente) mais obtenue par un **MOVLW 0x00** pour la première instruction du module :

$$T_{1\max} = 3.256 + 5 = 773 \mu s.$$

Il faut bien avoir bien compris cette « singularité », liée à un « effet de bord » car vous aurez à la gérer en TP, par exemple dans le cas des temporisations paramétrables...

Pour des temporisations plus importantes, plusieurs techniques sont possibles et parmi celles-ci :

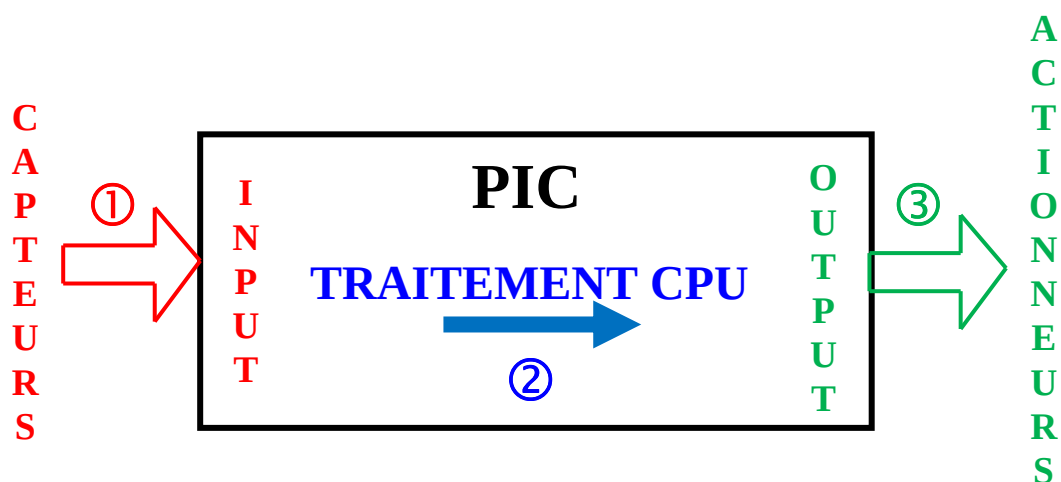
- on peut « **imbriquer** » des boucles les unes dans les autres : mais le calcul de la durée de temporisation devient vite rébarbatif...
- on peut utiliser le module **tempo_test** avec le principe de la programmation modulaire : c'est ce qu'on vous demande de faire en TP.

Rentrée dans le « cœur du sujet »

C'est la partie la plus **intéressante** et la plus **fondamentale**.

Interfacer directement le **PIC** avec le monde **extérieur** :

Car un microcontrôleur n'est pas fait
que pour faire des calculs en **interne**
(comme on l'a fait jusqu'ici)
mais bien pour agir sur son environnement **externe** !!!



- ① pour **recupérer** des informations venant de l'extérieur :
via des **capteurs** reliés à ses **périphériques d'entrée**,
- ② ensuite **les traiter** en **interne** suivant un algorithme dépendant de l'application à réaliser :
via sa **CPU**,
- ③ puis **avoir une action** sur cet environnement :
via des **actionneurs** reliés à ses **périphériques de sortie**.

Les Entrées/Sorties logiques

Interfaces qui sont certainement **le plus souvent utilisées**.

Peut-être aussi **les plus simples à mettre en œuvre**.

(car pour chaque PORT, seuls 2 registres sont concernés)

Généralités et principes

Le PIC 16F877 possède un maximum de 33 lignes d'Entrées/Sorties logiques (ou E/S Tout (5 V) Ou Rien (0 V) ou TOR en abrégé) réparties sur 5 ports qu'on appelle :

PORTA, PORTB, PORTC, PORTD et PORTE

Certaines lignes de ces ports TOR sont partagées avec d'autres **fonctions** par souci de diminution du nombre de broches du circuit intégré : auquel cas, si la **fonction correspondante est active**, la ligne de port (en tant qu'Entrée/Sortie logique) ne l'est plus.

Exemple:

Si on utilise le **Convertisseur Analogique/Numérique** via une des lignes du **PORTA**, cette ligne ne pourra plus, à cet instant là, fonctionner en entrée/sortie logique, mais fonctionnera seulement en entrée analogique.

Résumé du cours PIC : chapitre VIII.1.
(informations supplémentaires pendant manipulations TP 1)

Généralités et principes (suite)

A chaque port est associé un **registre de PORT** de taille 8 bits portant le même nom que ce port.

Voici un extrait de la cartographie complète des SFR, isolant les registres correspondant à ces 5 ports.

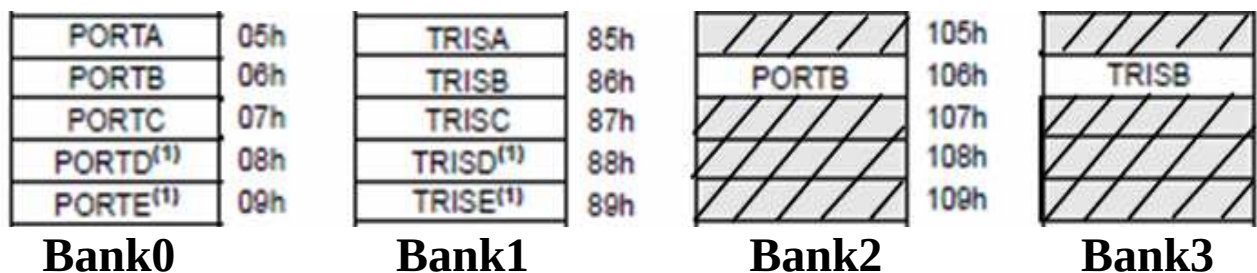


Figure VIII.1. (page 218)

Exemple : pour le PORTB, le registre de port s'appellera **PORTB**.

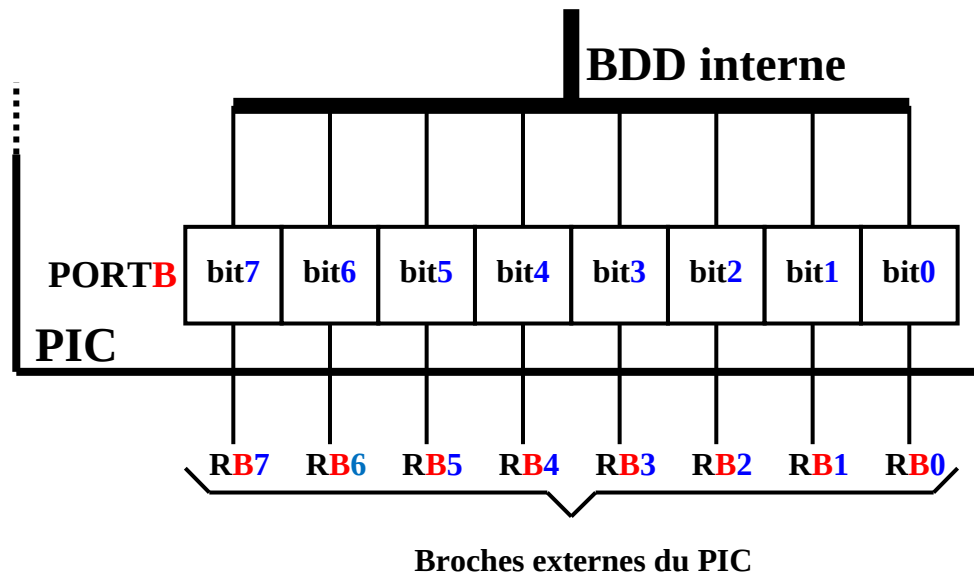


Figure VIII.2. (page 218)

Ce registre de port se comporte un peu **comme une case mémoire** mais reliée vers l'extérieur **bit à bit à huit broches** du PIC.

(ici **R_Bi**, pour i=0 (LSB) à 7 (MSB) pour le **PORT_B**)

Généralités et principes (suite)

Chacune de ces lignes peut être individuellement programmée pour agir, à certains moments (mais jamais en même temps) :

soit comme une **entrée (input)**, soit comme une **sortie (output)**.

On parle alors de bidirectionnalité par ligne.

C'est bien un paramétrage ligne à ligne (et non globalement sur l'ensemble des 8 lignes du port) car :

à chaque port **PORTX** est associé
un **registre de direction** de port appelé **TRISX** (X=A,B,...,E).

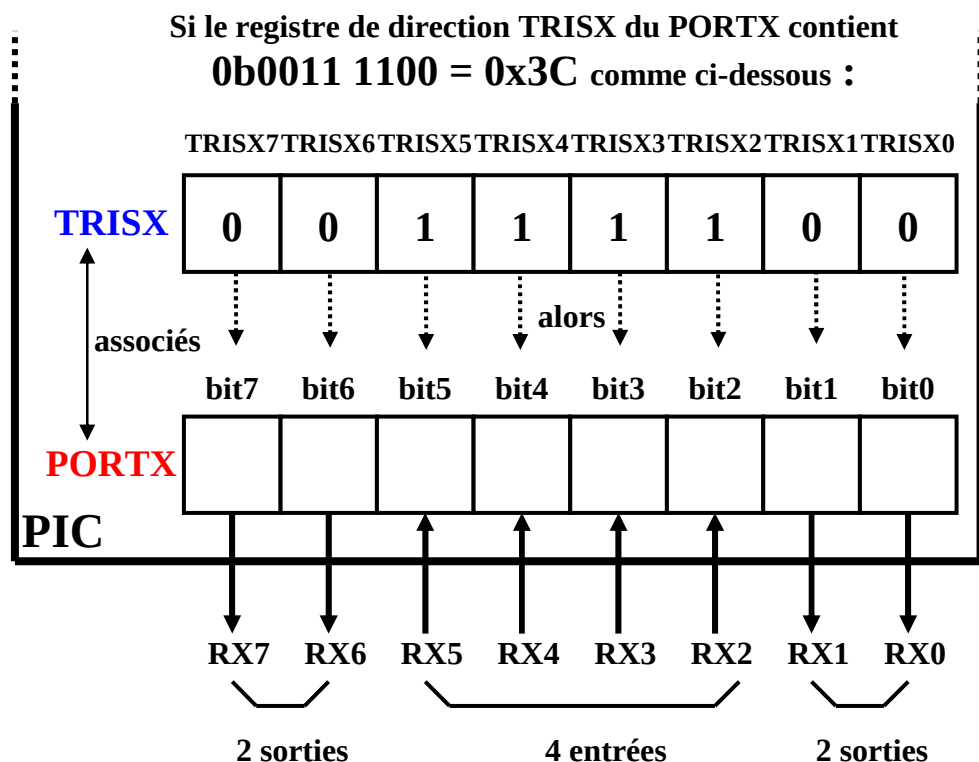


Figure VIII.3. (page 220)

Moyen mnémotechnique

TRISXi = 1 ⇒ ligne RXi en Intput TRISXi = 0 ⇒ ligne RXi en Output

Généralités et principes (suite)

C'est le programmeur qui écrira la valeur adéquate **TRISX** pour piloter correctement chaque ligne du **PORTX** en fonction des affectations qu'il a choisies ou de celles imposées par le hardware, dans une phase appelée **initialisation** du (ou des) port(s).

💣💣💣 **erreur fréquente :**

on oublie d'initialiser les ports avant de les utiliser !!!

*La bidirectionnalité offre plus de souplesse que si les ports étaient **monodirectionnels** (tout le temps figés en entrée ou tout le temps figés en sortie, comme sur certains types de microcontrôleurs).*

*Le fait que **chaque ligne** d'un port soit programmable **individuellement** est aussi mieux que si c'était l'ensemble des 8 lignes du port qui était soit en entrée, soit en sortie.*

« Mais comment une même broche peut-elle fonctionner tantôt en entrée et tantôt en sortie ? »

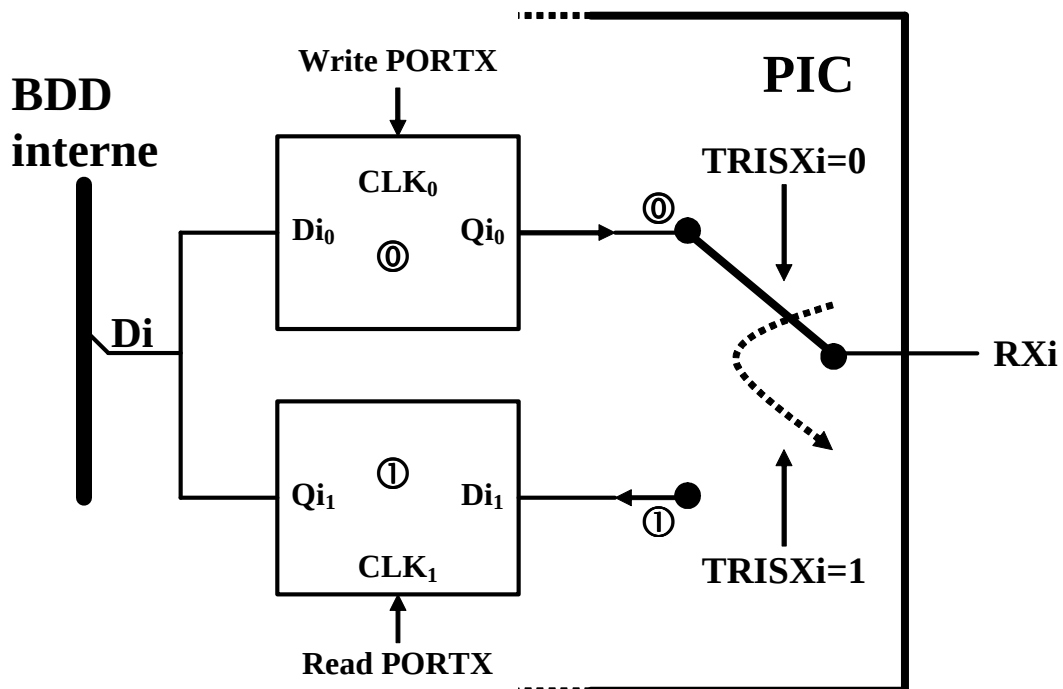


Figure VIII.4. (page 221)

Résumé du cours PIC : chapitre VIII.1.
(informations supplémentaires pendant manipulations TP 1)

Fonctionnement en sortie

L'écriture d'un octet dans le registre du port **PORTX** se retrouve **bit à bit** sur les broches **RXi** correspondantes du PIC :

chaque bit à **1** donne **5V** sur la broche correspondante
et
chaque bit à **0** donne **0V** sur la broche correspondante

Ne pas oublier de mettre avant
les broches correspondantes **en sortie** via **TRISX !!!**

Exemple :

Si les **8 lignes** du **PORTB** sont en **sortie** et chacune reliée à une résistance **R** et une **LEDi** ($i=0$ à 7).

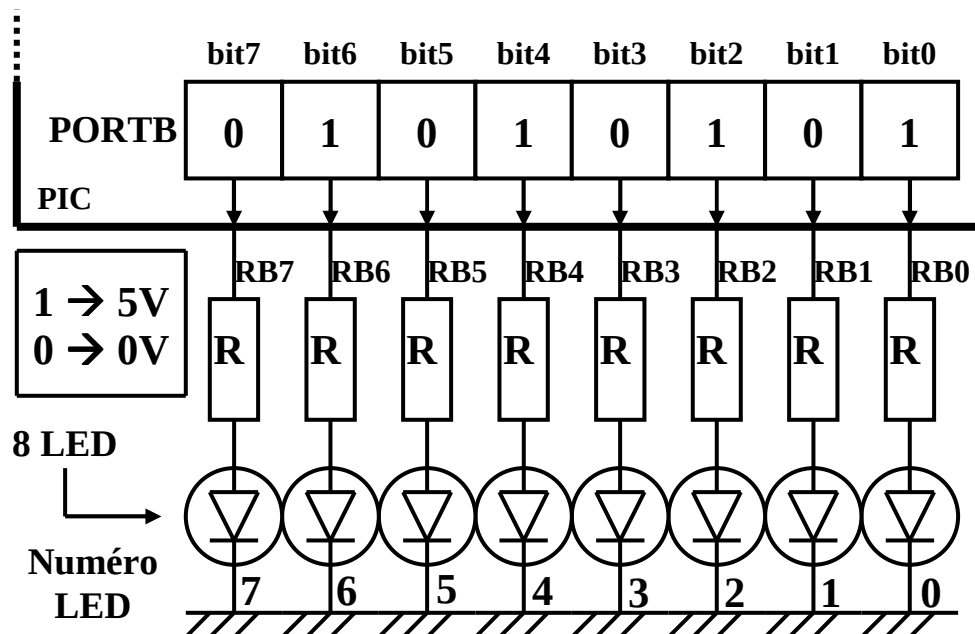


Figure VIII.5. (page 222)

Quelles sont les LEDs allumées dans cette configuration ?

Fonctionnement en sortie (suite)

Mise en œuvre en assembleur

; pour mettre les 8 bits du **PORTB** en sortie :

BANKSEL	TRISB	; adressabilité du SFR TRISB
MOVLW	0x00	; W = 0x00
MOVWF	TRISB	; les 8 bits de TRISB à 0
		; donc PORTB entier en sortie

; pour allumer par exemple uniquement la led numéro 2 :

BANKSEL	PORTB	; adressabilité du SFR PORTB
BSF	PORTB,2	; allume uniquement la led N°2

; pour éteindre par exemple uniquement la led numéro 2 :

BCF	PORTB,2	; éteint uniquement la led N°2
------------	----------------	--------------------------------

; pour allumer toutes les leds en même temps

MOVLW	0xFF	; W = 0xFF
MOVWF	PORTB	; les 8 bits de PORTB à 1
		; donc toutes les leds allumées

Fonctionnement en sortie (suite)

*Autre méthode pour piloter une LED en sortie
dite en « logique négative »*

*Méthode donnée en exemple avec la ligne **RB1** configurée en sortie:*

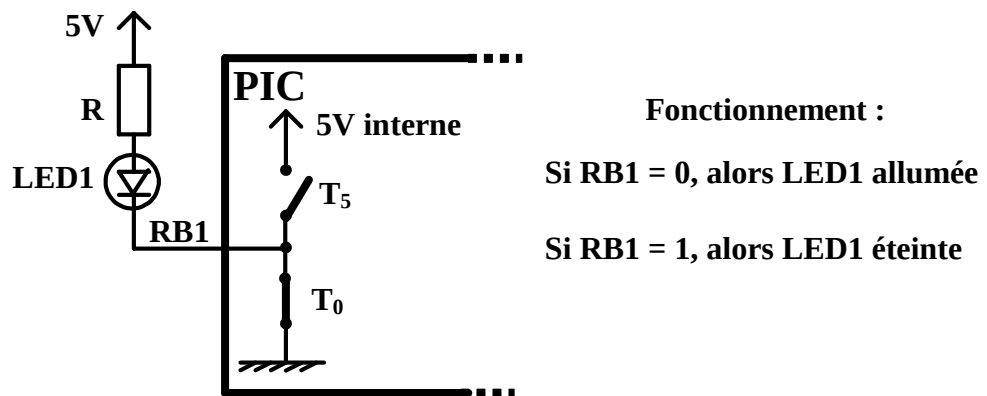


Figure VIII.6. (page 223)

*Lorsque **TRISB1=0** (broche **RB1** en sortie) :*

*Les deux interrupteurs T_0 et T_5 sont en faits deux transistors commandés en opposition de phase et pouvant être soit dans l'état **passant**, soit dans l'état **bloqué**.*

*Ils forment ce qu'on appelle un étage « **totem pôle** ».*

*Mais la logique de commande pour piloter la **LED** est ici l'inverse.*

Conclusion importante !!!

**Toujours regarder le hardware (le schéma)
avant de réaliser le software
(le programme pilotant ce hardware) !!!**

Fonctionnement en entrée

La **lecture** de l'octet contenu dans le registre du port **PORTX** reflète **bit à bit** la valeur des tensions présentes sur les broches **RXi** correspondantes du PIC :

5V sur la broche donne le bit correspondant à **1**
et
0V sur la broche donne le bit correspondant à **0**

Ne pas oublier de mettre avant
les broches correspondantes **en entrée** via **TRISX !!!**

Exemple : si les **8 lignes** du **PORTC** sont en **entrée** et chacune reliée à une résistance **R** (référéncée au 5V) et un **interrupteur Ki**.

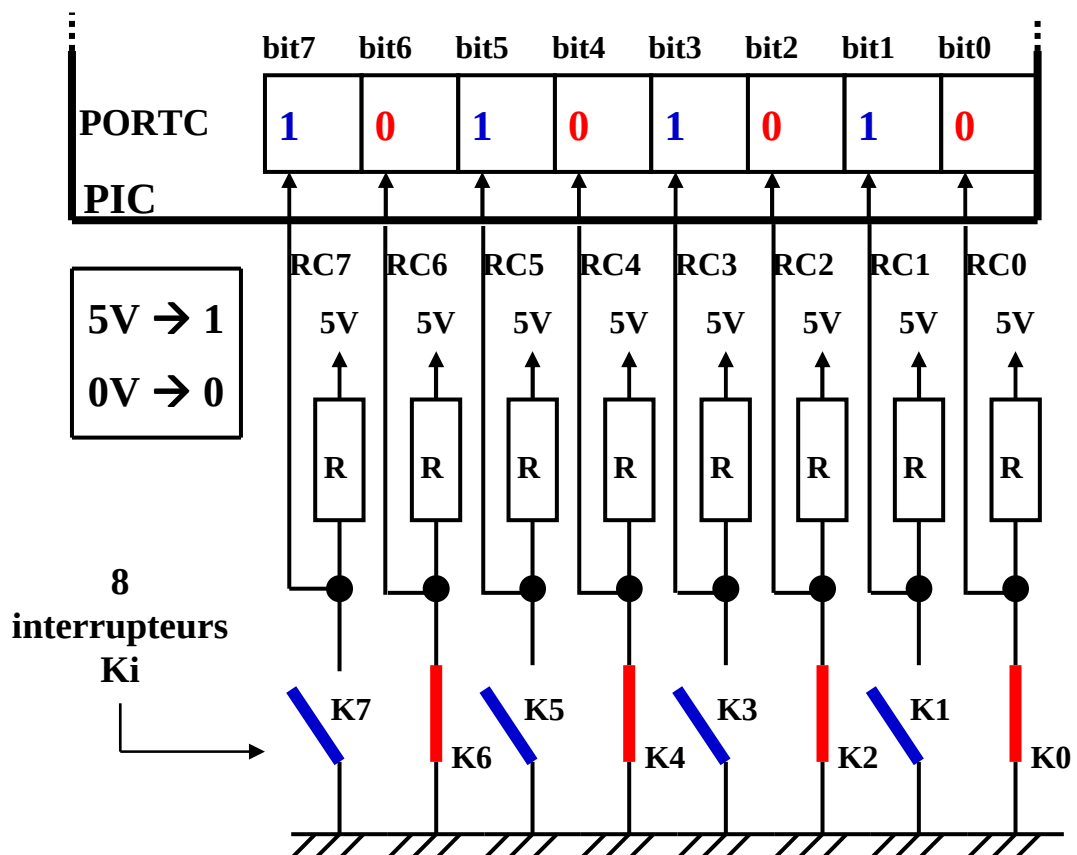


Figure VIII.9. (page 225)

Fonctionnement en entrée (suite)

Autre méthode pour câbler un interrupteur en entrée

La résistance **R** est référencée par rapport à la masse :
résistance de « **pull-down** » (**tirage vers le bas**, ici vers 0 V).

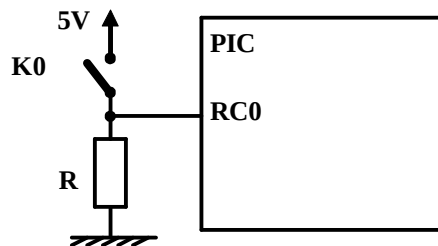


Figure VIII.10. (page 226)

La logique est alors inversée par rapport au cas précédent où **R** était une résistance de « **pull-up** » (référéncée au 5 V).

Cas des Boutons Poussoirs (BP)

Les interrupteurs présentés jusqu'ici étaient des interrupteurs à 2 états **permanents** : **ON** ou **OFF**.

Par contre, les **Boutons Poussoirs** ne gardent un de ces deux états (généralement **ON**) que quand on appuie dessus (état appelé **fugitif**).

La problématique est alors de détecter cet appui par logiciel.

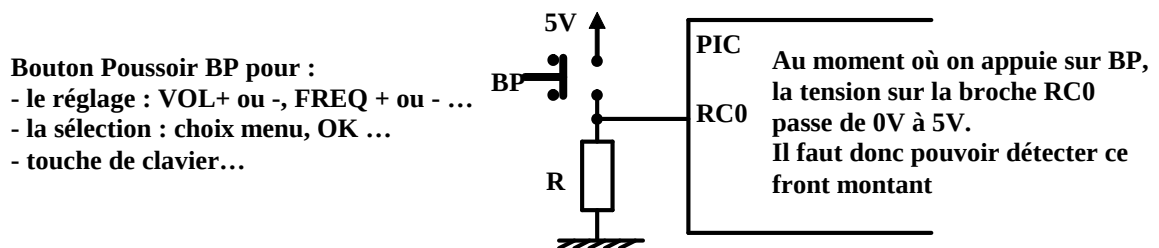


Figure VIII.11. (page 226)

Temporisations logicielles en C

Il y a beaucoup de possibilités en C pour les écrire :

boucles **for**, **while**, **do ... while...**

Mais c'est plus délicat en C de calculer « à la main » la durée réalisée car on ne sait pas, à priori, comment ces structures seront implantées en langage machine par le compilateur.

On étalonne donc les temporisations **expérimentalement** en utilisant la fonction « **Stopwatch** » du simulateur.

- exemples pour temporisations fixes :

Voici trois exemples d'une fonction de temporisation **tempo_1ms** d'une durée fixe la plus proche possible de **1 ms**.

Sur ces exemples, la constante **t1ms** a été définie par une directive **#define** **t1ms**

boucle for	boucle while	boucle do ... while
<pre>// tempo_1_ms : // par boucle for // dure 995µs si t1ms=76 // dure 1008µs si t1ms=77 void tempo_1ms(void) { unsigned int i; for(i=0;i<t1ms;i++); }</pre>	<pre>//tempo_1_ms : // par boucle while // dure 991µs si t1ms=75 // dure 1006µs si t1ms=76 void tempo_1ms(void) { unsigned int i=0; while(i<t1ms) i++; }</pre>	<pre>// tempo_1_ms : // par boucle do while // dure 995µs si t1ms=76 // dure 1008µs si t1ms=77 // équivalent à la boucle for void tempo_1ms(void) { unsigned int i=0; do i++; while(i<t1ms); }</pre>

Figure VI.5. (page 197)

Temporisations logicielles en C (suite)

Remarque

En pratique, choisir une durée un peu plus faible car on peut toujours l'augmenter, sans en changer le code fonctionnel par rajout de « nop » (une diminution est plus problématique ...).

*A ce sujet, voici comment rajouter une (ou plusieurs) instructions assembleur **au sein** d'un programme en C (on parle d'assembleur «**in line**»).*

- pour **une seule** instruction :

asm("instruction"); // exemple : **asm**("MOVLW 0x55");

- pour **un bloc d'instructions** : **couple de directives** **#asm / #endasm**

	#asm	
	instruction 1	// exemple : MOVLW 0x35
	instruction 2	// exemple : ANDLW 0x0F
	instruction 3	// exemple : CLRW
	#endasm	

Mais attention, quand on rajoute des instructions assembleur de ne pas perturber le fonctionnement normal du programme en C.

*Comme dans les exemples ci-dessus où ces instructions modifient **W**, alors que **W** pouvait à cet instant contenir une valeur importante pour le déroulement du programme en C.*

*Par contre, en rajoutant des « **NOP** », il n'y a aucun souci...*

Temporisations logicielles en C (suite)

*Prenons le cas de la tempo avec la boucle **for** (ou **do ... while**) et **t1ms = 76** qui dure **995 μ s**.*

*Combien de **NOP** faut-il rajouter et à quel endroit ?*

*Il faut bien sûr **5 NOP** (l'instruction **NOP** dure **un** cycle soit **1 μ s**).*

*Si on les place dans la boucle **do ... while**, elles seront exécutées **76** fois, ce qui est trop.*

*Donc, placer ces «**5 NOP**» juste après la déclaration de la locale **i**.*

- exemples de temporisations paramétrables :

On peut aussi adapter les fonctions précédentes pour faire des temporisations de durées variables de prototype :

// temporisation paramétrable de **ms** milliseconde(s)

void **tempo_p_ms** (unsigned int **ms**);

// temporisation paramétrable de **s** seconde(s)

void **tempo_p_s** (unsigned int **s**);

Principaux inconvénients des temporisations logicielles (fixes ou paramétrables) :

- manque de précision,**
- durée dépendant du quartz : si on veut les utiliser telles quelles sur un PIC avec un quartz de fréquence différente, leurs durées changeront ...**

**D'où l'utilisation des timers internes du PIC :
on parle alors de temporisations matérielles.**

Positionnement de bits individuels en C

En C, plusieurs possibilités pour positionner un bit dans un registre **SFR** ou **GPR** de nom **FOO**.

(**FOO** est souvent utilisé en info pour nommer quelque chose !!!)

- technique par masques :

moins évoluées, proche du hardware et de l'assembleur

Bit concerné	Masque (en binaire)	Masque (en hexa)	Masque inversé (en binaire)
0	0b 0000 000 1	0x0 1	0b 1111 111 0
1	0b 0000 00 1 0	0x0 2	0b 1111 11 0 1
2	0b 0000 0 1 00	0x0 4	0b 1111 1 0 11
3	0b 0000 1 000	0x0 8	0b 1111 0 111
4	0b 000 1 0000	0x 1 0	0b 111 0 1111
5	0b 00 1 0 0000	0x 2 0	0b 11 0 1 1111
6	0b 0 1 00 0000	0x 4 0	0b 1 0 11 1111
7	0b 1 000 0000	0x 8 0	0b 0 111 1111

Figure VI.12. (page 208)

Deux exemples pour illustrer la technique des masques :

- mise à **1** du bit **3** du registre **FOO** : avec un **ou logique**
`FOO = FOO | 0x08 ;` // **ou logique** bit à bit | avec le masque
(ou `FOO |= 0x08 ;`) // idem
- mise à **0** du bit **3** du registre **FOO** : avec un **et logique**
`FOO = FOO & (~0x08) ;` // **et logique** bit à bit & avec le
// masque **inversé**
(ou `FOO &= (~0x08) ;`) // idem

Positionnement de bits individuels en C (suite)

Si on rajoute la ligne suivante dans son source en C :

```
#define bit(x) (1 << (x))    // décalage de 1 = 0b 0000 0001
                             // de x bits sur la gauche : opérateur <<
```

plus besoin de calculer soi-même les masques en hexadécimal.

Ainsi les deux exemples précédents s'écriront :

- mise à 1 du bit 3 du registre FOO : avec un **ou logique**
 FOO = FOO | bit(3); // **ou logique** bit à bit | avec bit(3)
 (ou FOO |= bit(3);) // idem
- mise à 0 du bit 3 du registre FOO : avec un **et logique**
 FOO = FOO & (~bit(3)); // **et logique** bit à bit & avec
 // bit(3) **inversé**
 (ou FOO &= (~bit(3));) // idem

- technique par des macros :

assez peu utilisée mais citée ici pour mémoire

Il faut alors définir les deux macros suivantes dans son source :

```
// mise à 1 du bit bitno de var
#define bitset(var, bitno) ((var) |= 1UL << (bitno))

// mise à 0 du bit bitno de var
#define bitclr(var, bitno) ((var) &= (~(1UL << (bitno))))
```

Ainsi les deux exemples précédents s'écriront :

- mise à 1 du bit 3 du registre FOO :
 bitset(FOO,3);
- mise à 0 du bit 3 du registre FOO :
 bitclr(FOO,3);

Positionnement de bits individuels en C (suite)

- technique par les noms symboliques des bits :
plus évoluée car c'est le compilateur C qui fait « le job »

C'est cette technique que vous utiliserez systematiquement en TP car elle est plus lisible et évite surtout d'avoir à connaître la **position du bit** concerné et le registre **SFR** correspondant dans lequel se trouve ce bit.

Il suffit de connaître son nom !!!

C'est le C qui fait automatiquement la correspondance, car les bits individuels de la plupart des registres **SFR** ont été déclarés (via les fichiers header **pic.h** et **pic168xa.h**) et sont reconnus par le couple compilateur/linker.

Il faut donc garder les mêmes noms de bits que ceux fournis et utilisés par MICROCHIP dans le datasheet du PIC (et ne pas inventer ses propres noms !!!)

Par exemple, pour mettre à **5V** la ligne **2** du **PORTB**, on écrira tout simplement :

RB2 = 1 ;

(au lieu de **PORTB = PORTB | 0x04**)

En ayant bien sûr auparavant initialisé cette ligne **2** en **sortie** par :

TRISB2 = 0 ;

(au lieu de **TRISB = TRISB & (~0x04)**)

Test de bits individuels en C

Dans une application à base de microcontrôleur interfacée avec l'extérieur, il est fondamental de savoir lorsqu'un évènement extérieur (et nécessitant un traitement) s'est produit, justement pour lancer ce traitement.

Les exemples sont nombreux (liste non exhaustive...) :

- une ligne d'un PORT (programmée en entrée) est passée du niveau bas au niveau haut ou du niveau haut au niveau bas,
- un convertisseur analogique/numérique a terminé sa conversion...
- un intervalle de temps s'est écoulé,
- un caractère est arrivé sur une liaison série asynchrone,
- un caractère a été entièrement transmis sur une liaison série asynchrone,

Chacun de ces évènements se traduit au niveau du PIC par le passage de **0 à 1** (le plus souvent) ou de **1 à 0** d'un bit particulier situé dans un **SFR** du PIC. Et donc, chacun de ces évènements est associé à ce bit qu'on peut appeler aussi par extension un **flag**.

(☛ : ne pas confondre avec les flags Z, C et DC de la CPU du PIC).

Lorsqu'on ne travaille pas sous interruption, la technique à mettre en œuvre pour détecter l'apparition d'un évènement s'appelle la **scrutation** (ou **polling** en anglais).

Elle consiste à **surveiller l'état du flag** associé à cet évènement **en attendant** par exemple que ce flag passe de 0 à 1 (ou de 1 à 0).

Test de bits individuels en C (suite)

Là aussi, plusieurs procédés pour effectuer ce pooling en C.

- technique par des masques : proche du hard

Quelques exemples pour illustrer la technique de pooling avec des masques :

// attente que le **bit 7** du registre **SFR FOO** passe de 0 à **1**

```
while (!(FOO & 0x80)) ;    // tant que bit 7 de FOO = 0, on attend
                          // à partir de là, le bit 7 de FOO = 1
```

// autre écriture du même exemple si on a écrit dans son source :

```
#define bit(x) (1 << (x))
```

```
while (!(FOO & bit(7))) ;  // tant que bit 7 de FOO = 0, on attend
                          // à partir de là, le bit 7 de FOO = 1
```

// attente que le **bit 0** du registre **SFR FOO** passe de 1 à **0**

```
while (FOO & 0x01) ;      // tant que bit 0 de FOO = 1, on attend
...                        // à partir de là, le bit 0 de FOO = 0
```

// autre écriture du même exemple :

```
#define bit(x) (1 << (x))
```

```
while (FOO & bit(0)) ;    // tant que bit 0 de FOO = 1, on attend
...                       // à partir de là, le bit 0 de FOO = 0
```

Test de bits individuels en C (suite)

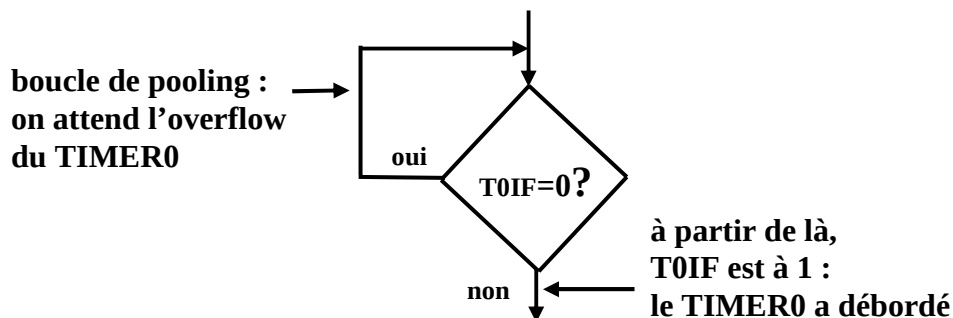
- technique par les noms symboliques des bits :

Deux exemples pour cette technique avec le nom du bit testé.

Exemple 1 :

Il existe, pour le TIMER0 du PIC, un bit particulier qui indique, lorsqu'il passe à 1 que le TIMER0 est passé de **0xFF** à **0x00** : on dit que le TIMER0 a subi un **overflow** (un dépassement de capacité). Ce bit s'appelle d'ailleurs le bit **T0IF** (Timer**0** Interrupt **F**lag). C'est le bit 2 du registre SFR **INTCON**.

Pour détecter par pooling, le passage de **0** à **1** de ce bit **T0IF** :



// attente que le flag **T0IF** passe de **0** à **1**

```
while(!T0IF) ;    // tant que T0IF = 0, on attend
...               // à partir de là le flag T0IF = 1
(au lieu de while (!(INTCON & 0x04)) !!!)
```

Exemple 2 :

// attente que la ligne **RA4** (supposée en entrée) passe de **1** à **0**

```
while(RA4);       // tant que RA4 = 1, on attend
...               // à partir de là, l'entrée RA4 = 0
```

Les interruptions matérielles

C'est une des parties **les plus délicates à comprendre** mais aussi **à mettre en œuvre** dans le cadre d'une application.

Car travailler sous **interruption matérielle** (on dit sous **IT**) est lié à la **notion de temps réel** et nécessite de faire beaucoup de choses au niveau de la programmation.

Si on en oublie une, cela ne fonctionnera pas et le débogage sera toujours assez délicat (car le boîtier **ICD3** a ses limites pour la gestion du temps réel et des évènements extérieurs).

Pourtant, les IT constituent un moyen **puissant, structurant** et **efficace** pour gérer et traiter des **évènements intérieurs et/ou extérieurs**.

Il faut donc bien en comprendre
toute la « mécanique sous-jacente ».

Définitions et exemples

Tout d'abord, qu'entend-on par **évènement** ?

C'est un évènement lié au hardware indiquant que quelque chose d'important a eu lieu dans l'environnement intérieur et/ou extérieur du PIC

On parle de **la source ou de la cause** de l'IT.

Le PIC 16F877 peut gérer **14** types d'évènements différents.

Les interruptions matérielles (suite)

Quelques exemples (liste non exhaustive) :

- ① **liés aux ports TOR** : transition (front montant ou descendant) sur une ligne d'entrée d'un PORT : appui sur un BP ou un clavier...
- ② **liés au convertisseur analogique/numérique** : une conversion vient de se terminer... (voir plus loin dans ce TP)
- ③ **liés aux timers internes** : un compteur de temps est passé de 0xFF à 0x00 (débordement d'un timer)... (cf. TP 3)
- ④ **liés à l'USART** : un caractère est reçu sur une liaison série...

Ce qu'il faut bien comprendre, c'est que cet évènement intérieur et/ou extérieur peut arriver n'importe quand, et donc de façon totalement asynchrone par rapport au programme s'exécutant sur le PIC à cet instant.

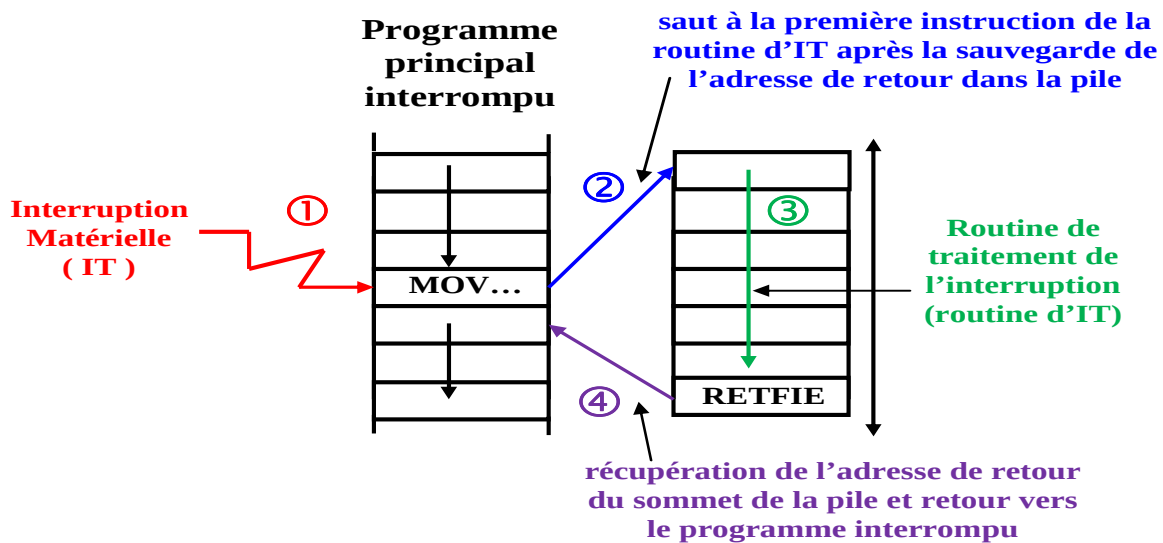
Mais, dans tous les cas de figure, il faut lancer un traitement qui devra être exécuté pour traiter cette interruption.

- ① **faire l'action** correspondant à l'appui sur le BP ou le clavier...
- ② **lire la valeur numérique** fournie par le CAN pour pouvoir la traiter et éviter que la prochaine conversion ne vienne l'écraser...
- ③ **incrémenter une variable compteur** (de temps) pour gérer correctement le temps qui s'écoule...
- ④ **recupérer le caractère** qui vient d'arriver sur la liaison série pour éviter que le caractère suivant ne vienne l'écraser avant...

Les interruptions matérielles (suite)

Mécanisme général d'une interruption matérielle

Ce traitement qui doit être effectué à chaque IT est appelé la **routine de traitement de l'interruption** (ou la routine d'IT).



Principe du traitement d'une interruption matérielle

Figure VIII.29. (page 240)

Le programme en cours sur le PIC :

- est donc « **interrompu** » par l'IT (①),
- **donne la main** à la routine d'IT (un peu comme pour un module ou une fonction : ② $\equiv$ **ticket « aller »**),
- **qui s'exécute alors** (③).
- puis, **à la fin de la routine d'IT** (④), le programme interrompu reprendra son exécution en continuant à l'instruction **qui suit celle** qui s'exécutait au moment de l'IT ($\equiv$ « **ticket retour** »).

Les interruptions matérielles (suite)

Mécanisme général d'une interruption matérielle

Mais attention, ce n'est pas une instruction du programme qui appelle l'interruption, comme pour une fonction en C ou un module en assembleur, mais la routine d'IT sera appelée « **automatiquement** » quand l'IT se produira.

Lorsque que j'entends des étudiants dire : « quand **j'**appelle la routine d'IT ... », c'est qu'ils n'en n'ont pas bien compris le mécanisme.

Une routine d'interruption **ne s'appelle pas au moyen d'une instruction CALL existant explicitement dans le programme** (c'est-à-dire comme pour une fonction ou un module), mais l'appel est fait quand l'IT a lieu (et c'est en ce sens que les interruptions sont **asynchrones** par rapport au programme ...).

En fait, si on y regarde bien, la seule ressemblance entre une routine d'IT et un module (ou une fonction) est que, comme eux, une routine d'IT a :

- un **nom** : qu'on supposera dans cette présentation générale être **traitement_it**,
- un **début** : première instruction de la routine d'IT,
- et une **fin** (instruction **RETFIE** en assembleur) : avec un retour ensuite au programme interrompu.

Et c'est tout. Tout le reste n'est que différence.

Mais allons un peu plus loin...

Les interruptions matérielles (suite)

Spécificités d'une routine d'interruption

Voici les neuf différences notables entre une routine d'IT et un(e) « classique » module (fonction) :

① les interruptions peuvent avoir lieu **n'importe quand**. L'instruction qui s'exécute au moment de l'IT **se termine** avant de donner automatiquement la main à la **routine d'IT**.

Il n'y aura pas dans votre programme une instruction comme :

CALL traitement_it ; si programmation en assembleur
ou traitement_it() // si programmation en C

② une routine d'IT **ne peut pas avoir de paramètres en entrée** (impossible de les lui passer) **ni ne peut renvoyer une valeur de retour au programme interrompu** (qui n'a aucun moyen de pouvoir la traiter). Par contre, une routine d'IT peut parfaitement modifier une variable globale (qui pourra être testée ensuite).

③ à certains moments « stratégiques », le programme peut décider **de masquer toutes les interruptions**. Pour cela, il existe un bit, le **bit 7** du registre **SFR INTCON** (comme **INTerrupt CONtrol** register) appelé bit **GIE** (comme **Global Interrupt Enable**) :

Si bit **GIE = 0**, **toutes les interruptions sont masquées**.

Attention : ce bit est un masque **global**, valable pour masquer **toutes** les interruptions.

(c'est le cas au démarrage ou après un reset du PIC).

Conséquence : au moment où on souhaite travailler sous IT, il faudra mettre ce bit **GIE à 1** (**démasquage** de toutes les IT).

Les interruptions matérielles (suite)

Le bit **GIE** est **automatiquement mis à 0** au moment du saut vers la routine d'interruption **traitement_it**, qui est donc **non interruptible**.

Ce qui interdit, par défaut, **l'imbrication** des interruptions. Par contre, rien n'interdit d'appeler une fonction (un module) dans la routine d'IT.

Au moment du retour vers le programme interrompu, **GIE** sera **automatiquement remis à 1**.

(par RETFIE = (GIE=1) + RETURN)

④ de façon plus « fine » que par le bit **GIE**, chaque interruption peut aussi être **individuellement** masquée : on parle alors de masquage **local**.

Si on décide, dans cette présentation générale, d'affecter un numéro (par exemple un entier n) à chaque source d'IT (pour les différencier entre elles), cela signifie qu'il existe un bit particulier, (dans un des registres **SFR** du PIC), bit qu'on appellera ici **PER n _IE** (comme **PER**iphéric N° n Interrupt **E**nable) permettant d'autoriser ou de masquer cette interruption N° n :

- lorsque ce bit **PER n _IE = 1** :
le fonctionnement sous IT du périphérique N° n est **possible**,

- lorsque ce bit **PER n _IE = 0** :
l'interruption du périphérique N° n est **masquée**.

Les interruptions matérielles (suite)

Exemples pour les 4 cas d'interruption vues précédemment :

- bit **INTE** : pour valider l'IT sur front sur la ligne d'entrée **RB0**.
- bit **ADIE** : pour valider l'IT sur conversion terminée du **CAN**.
- bit **TOIE** : pour valider l'IT sur débordement du **TIMER0**.
- bit **RCIE** : pour valider l'IT sur réception d'un caractère (**USART**)

⑤ la **survenue** de l'évènement **N°n** (associé au périphérique **N°n**) positionne un flag particulier (dans un des registres SFR du PIC). Nous appellerons ici ce flag **PERn_IF** (comme **PER**iphéric **N°n** Interrupt Flag)

Exemples pour les 4 cas d'interruption vues précédemment :

- bit **INTF** : pour signaler le front sur la ligne d'entrée **RB0**.
- bit **ADIF** : pour indiquer la fin de la conversion du **CAN**.
- bit **TOIF** : pour signaler le débordement du **TIMER0**.
- bit **RCIF** : pour signaler la réception d'un caractère par **l'USART**.

Les interruptions matérielles (suite)

Ainsi, lorsque ce flag **PERn_IF** passe à 1 alors que le flag associé **PERn_IE** est aussi à 1, sans oublier le flag **GIE** qui doit être aussi à 1, une interruption est générée, comme indiqué ci-dessous :

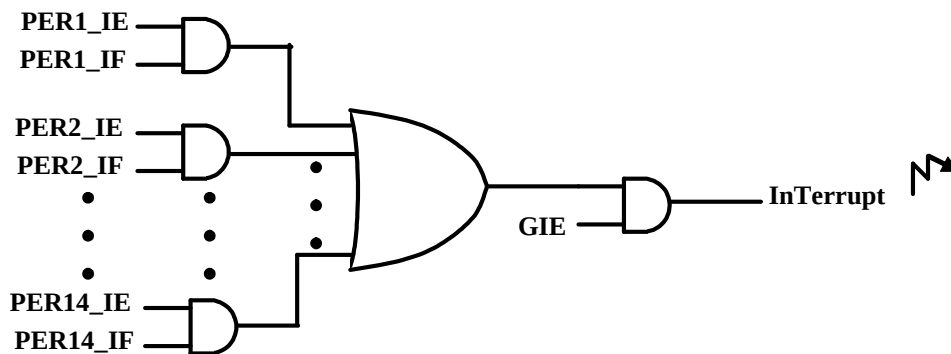


Figure VIII.30. (page 243)

En fait le schéma réel, légèrement différent, est fourni sur [la figure VIII.31. du polycopié de cours complet.](#)

On y voit bien que pour qu'une IT soit générée (sortie **InTerrupt** passant à 1), et donc que la routine **traitement_it** soit exécutée, il faut que :

- **GIE = 1** : **toutes** les interruptions (**globalement**) **démasquées**,

et

- **au moins** un couple :

$\left\{ \begin{array}{l} \textbf{PERn_IE = 1} : \text{IT N}^\circ\textbf{n (localement) démasquée,} \\ \textbf{PERn_IF = 1} : \text{l'évènement N}^\circ\textbf{n s'est produit.} \end{array} \right.$

Les interruptions matérielles (suite)

⑥ on remarque en examinant le schéma précédent, que quelque soit l'évènement extérieur, il n'y a **qu'une seule** routine d'IT exécutée.

Remarque : *sur certains micro/processeurs/contrôleurs (famille 68xxx de chez Motorola, famille 86 et Pentium de chez Intel...), chaque cause d'IT peut provoquer sa propre routine de traitement d'interruption : on dit alors que les IT matérielles sont **vectorisées**.*

Pour un PIC, quelque soit l'évènement extérieur, ce sera la routine **traitement_it commune** à toutes les IT qui sera exécutée.

Mais où se trouve en mémoire cette routine de traitement d'IT ?

Ceci est imposé par le constructeur du PIC :

à l'adresse absolue **0x0004** de l'EEPROM de programme.

Contrairement aux modules/fonctions qui peuvent être placées n'importe où en mémoire de programme, emplacements fixés soit par la directive **ORG** (en assembleur) soit par le couple compilateur/linker (en C).

Ainsi, à chaque IT, le PC sera chargé automatiquement par l'adresse **0x0004**. (ce qui permettra au PIC d'exécuter la première instruction de la routine d'IT, puis la suivante ... jusqu'au **RETFIE** final).

En C, on indiquera le type **interrupt** devant la déclaration de la routine d'IT, pour que le compilateur/linker place cette routine à l'adresse **0x0004** et la termine comme on l'a vu par l'instruction **RETFIE** (au lieu de l'instruction RETURN qui termine les modules ou fonctions).

Les interruptions matérielles (suite)

Dernier point à aborder : que faut-il mettre dans cette routine d'IT ?

La réponse à cette question nous permettra de mettre en évidence les trois dernières spécificités de la routine d'IT.

⑦ puisque'il n'y a qu'une seule routine d'IT commune à toutes les causes d'interruption, il faudra dans cette routine **trouver la cause** de l'interruption pour effectuer le traitement adéquat.

Pour cela, il faut construire une espèce de « **daisy chain software** » qui va venir **balayer** toutes les causes possibles d'interruption et qui va détecter celle (ou celles) qui a (ont) déclenché l'interruption.

```
    if (INTF && INTE)      // interruption due à un front actif sur RB0?
    {
// mettre ici le traitement à faire si front sur RB0
    }

    if (ADIF && ADIE)      // interruption due au CAN ?
    {
// mettre ici le traitement à faire à chaque conversion
    }

    if (TOIF && TOIE)      // interruption débordement du timer0?
    {
// mettre ici le traitement à faire pour gérer le temps qui s'écoule
    }

    if (RCIF && RCIE)      // interruption arrivée d'un caractère ?
    {
// mettre ici le traitement à faire à chaque réception de caractère
    }
```

Figure VIII.32. (page 245)

Les interruptions matérielles (suite)

⑧ voilà un point qui est souvent oublié dans la routine d'IT :
l'acquittement de l'interruption.

Qu'est ce que c'est ?

Imaginez un évènement extérieur N°n ayant eu lieu et donc ayant positionné le flag **PERn_IF** à 1.

Si les bits **GIE** et **PERn_IE** sont à 1, la routine d'IT est exécutée.

Si aucun acquittement n'est fait dans cette routine d'IT, à son retour vers le programme interrompu, le flag **PERn_IF** sera toujours à 1, déclenchant tout de suite une autre IT ... et ainsi de suite.

Le PIC sera en état de **permanente** et **répétitive** interruption :
il est « planté » !!!

Acquitter une interruption consiste donc, dans la routine d'IT, à **effacer la cause de cette interruption**.

En pratique, cela revient à **remettre à 0** le bit **PERn_IF**.

Il y a en fait deux possibilités :

- certains flags **PERi_IF** (les « **software** » flags) doivent être **explicitement** effacés par une instruction de la routine d'IT :

PERi_IF = 0 ;

- d'autres flags **PERj_IF** (les « **hardware** » flags) sont effacés **implicitement** par l'action liée au traitement de l'IT.

Par exemple, dans le cas d'une IT due à la réception d'un caractère sur la liaison série (**RCIF=1**), **la lecture** de ce caractère dans la routine d'IT remet **automatiquement ce flag RCIF à 0**.

Les interruptions matérielles (suite)

⑨ le dernier point est peut-être le plus délicat à comprendre et aussi à mettre en œuvre, **surtout quand on programme en assembleur.**

Il est lié à la notion de **contexte**. Qu'est ce que c'est ?

Lorsqu'un programme est interrompu, supposons qu'au moment de l'IT, **W** contienne la valeur 0x03 utilisée pour un calcul intermédiaire. Dans la routine d'IT, **W** pourra être modifié (et le sera certainement). Supposons qu'au retour de la routine d'IT, **W** contienne la valeur 0x05. Le calcul dans le programme interrompu sera continué mais avec une valeur dans **W** qui aura changé et donc il sera faux. Voilà un exemple de ce qu'on appelle le **contexte**.

Le contexte est donc l'ensemble des informations qui sont nécessaires pour le fonctionnement normal d'un programme et que la routine d'IT ne doit pas modifier.

Conclusion : il faut :

- les sauver au début de la routine d'IT (**sauvegarde du contexte**),
et

- les restituer tels quels avant le retour vers le programme interrompu (**restitution du contexte**).

Pour plus de détails en assembleur voir le [polycopié de cours complet aux pages 246 à 248](#).

En langage C, ces phases de sauvegarde et de restitution du contexte sont automatiquement (et de façon transparente pour le programmeur) rajoutées par le compilateur respectivement au début et à la fin de la routine d'IT (grâce au type **interrupt** déclaré devant la déclaration de la routine d'IT).

Résumé du cours PIC : chapitre VIII.2.
(informations supplémentaires pendant manipulations TP 2)

SYNTHESE : CANEVAS EN C POUR LA ROUTINE D'IT

Canevas type en C de la routine **traitement_it** (pour IT N°1 et N°2).

```
// routine d'interruption traitement_it de type interrupt (cf. ⑥)
// sans argument (void) et sans valeur retournée (void) (cf. ②)
// la sauvegarde et la restitution du contexte sont faits automatiquement et de
// façon transparente par le compilateur C (cf. ⑨)

void interrupt traitement_it (void)  // noter le type interrupt !!!
{
    if (PER1_IF && PER1_IE)  // interruption N°1 ? (cf. ⑤ et ⑦)
    {
        // oui, alors mettre ici le traitement de l'évènement extérieur N°1
        PER1_IF=0;           // acquittement de l'interruption N°1 (cf. ⑧)
    }
    if (PER2_IF && PER2_IE)  // interruption N°2 ? (cf. ⑤ et ⑦)
    {
        // oui, alors mettre ici le traitement de l'évènement extérieur N°2
        PER2_IF =0;          // acquittement de l'interruption N°2 (cf. ⑧)
    }
}

void main(void)
{
    // préparation du fonctionnement sous interruption : gestion des flags et des masques
    PER1_IF = 0;  // efface une éventuelle survenue évènement N°1
    PER2_IF = 0;  // efface une éventuelle survenue évènement N°2
    PER1_IE = 1;  // démasquage local de l'interruption N°1 (cf. ④)
    PER2_IE = 1;  // démasquage local de l'interruption N°2 (cf. ④)
    GIE=1;        // démasquage global de toutes les interruptions (cf. ③)
    while(1)      // boucle « applicative » sans fin
    {
        ...
        // ici dans cet exemple le main ne fait rien : mettez ici votre application
        // notez qu'il n'y aura jamais d'appel à traitement_it dans ce main (cf. ①)
        ...
    }
}
```

Figure VIII.35. (page 250)

Les EEPROM internes

Il y a deux blocs EEPROM dans le PIC16F877 :

- la **mémoire EEPROM de données de taille 256 octets**. pour les données (semi) permanentes (ne pas confondre avec la RAM qui contient les SFR et les GPR et les données volatiles).
- la **mémoire EEPROM de programme (déjà vue) de taille 8K cases de 14 bits chacune**. (nommée **FLASH EEPROM**) pour le code machine du programme, mais pouvant aussi servir, si l'EEPROM de données précédente est pleine, pour stocker des données (semi) permanentes supplémentaires.

On a déjà vu une première méthode pour écrire des valeurs dans ces mémoires au moment de **la phase de programmation** du PIC :

- pour les données en **EEPROM de données** avec les directives **ORG 0x2100** et **DE**
- code en **EEPROM de programme** par **programmation**.

Il existe une autre méthode, que l'on va présenter maintenant, pour y accéder **en dynamique** (pendant l'exécution du programme) en **lecture** ou en **écriture**.

Les EEPROM internes (suite)

Plusieurs registres SFR sont concernés et sont à utiliser pour assurer ces accès.

- registres contenant la **valeur à écrire** en EEPROM ou la **valeur lue** dans l'EEPROM :

EEDATA et **EEDATH** (EEprom **DATA** et **DATA** High)

- registres contenant **l'adresse** de la case EEPROM concernée par l'accès en lecture ou en écriture.
Plus précisément l'**offset** de la case par rapport aux adresses de base des EEPROM qui sont :

- **0x2100** pour **l'EEPROM de données**,
- **0x0000** pour **l'EEPROM de programme**.

EEADR et **EEADRH** (EEprom **ADResse** et **ADResse** High)

- registres permettant le **paramétrage** et le **contrôle** de l'accès réalisé :

EECON1 et **EECON2** (EEprom **CON**trôle 1 et 2)

Les EEPROM internes (suite)

Compte tenu :

- des capacités mémoire des EEPROM :
8 bits adresses pour les **256** ($=2^8$) cases **EEPROM données**,
13 bits adresses pour les **8192** ($=2^{13}$) cases **EEPROM programme**.
- des tailles de chaque case :
8 bits pour **l'EEPROM de données**,
14 bits pour **l'EEPROM de programme**.

le schéma suivant montre comment sont utilisés les registres :

EEDATA, EEDATH, EEADR et EEADRH

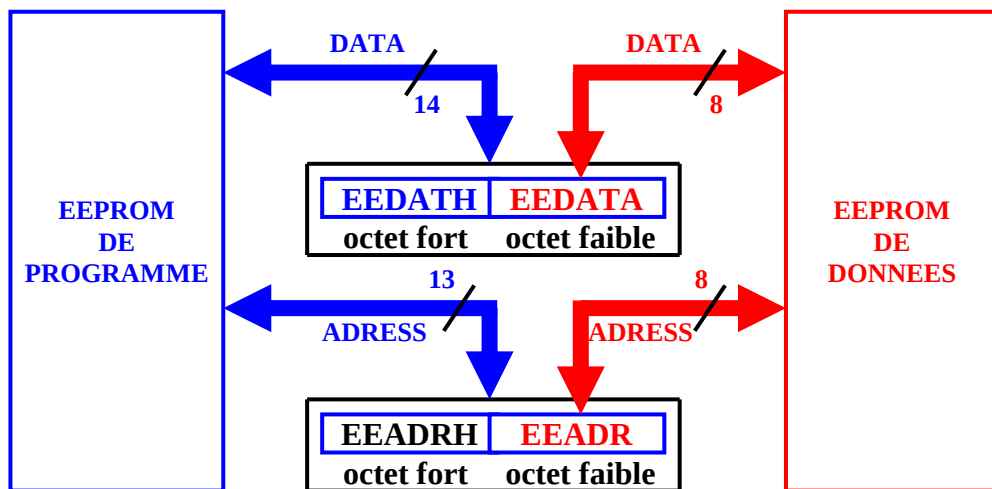


Figure VIII.37. (page 254)

EECON1 (situé à l'adresse 0x18C)
(page 42 du datasheet)

	R/W-x	U-0	U-0	U-0	R/W-x	R/W-0	R/S-0	R/S-0
EECON1	EEPGD	—	—	—	WRERR	WREN	WR	RD
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0

Figure VIII.38. (page 254)

Résumé du cours PIC : chapitre VIII.3.
(informations supplémentaires pendant manipulations TP 2)

Les EEPROM internes (suite)

Bit	Valeur	Signification et commentaires
EEPGD		Permet de sélectionner l'EEPROM concernée
	0	Accès (lecture ou écriture) à la mémoire EEPROM de données
	1	Accès (lecture ou écriture) à l' EEPROM de programme
RD		Accès EEPROM en lecture
	0	Pas de lecture
	1	Initialise un cycle de lecture EEPROM (données ou programme) Repasse automatiquement à 0 à la fin de la lecture : - à l'instruction suivante pour EEPROM données, - après les 2 instructions suivantes pour EEPROM programme.
WREN		Permet d'ouvrir une « fenêtre » d'écriture EEPROM
	0	Pas de « fenêtre » d'écriture
	1	Ouvre « fenêtre » d'écriture : doit être mis à 1 avant début cycle d'écriture en EEPROM et remis à 0 à la fin cycle d'écriture
WR		Accès EEPROM en écriture
	0	Pas d'écriture
	1	Initialise un cycle d'écriture EEPROM (données ou programme). Un tel cycle est plus long qu'une écriture en RAM. Il faut toujours s'assurer que ce cycle soit fini avant de refaire une autre écriture en EEPROM. Ce bit repassera automatiquement à 0 pour indiquer la fin de l'écriture EEPROM proprement dite. Le flag EEIF du SFR PIR2 passera alors de son côté à 1 (voir plus loin comment gérer ces bits WR ou EEIF)
WRERR		Bit d'état indiquant un problème pendant le cycle d'écriture (il y a eu par exemple un reset du PIC)
	0	Le cycle d'écriture s'est bien déroulé
	1	Le cycle d'écriture EEPROM a été interrompu et du coup la donnée n'a pas pu être écrite en EEPROM !!!

Figure VIII.39. (page 255)

Les EEPROM internes (suite)

Le registre **EECON2** est utilisé pour permettre une procédure un peu spéciale permettant d'éviter des écritures « intempestives » en EEPROM, un peu comme une espèce de « **clef de protection** ».

Cette procédure consiste à enchaîner (et donc **sans interruption**) une séquence de 5 instructions (imposées par **MICROCHIP**) avant l'écriture proprement dite.

On détaillera cette protection dans les exemples de programmes vus plus loin.

Mais l'idée est que « statistiquement » cette procédure a très très très peu de chance de se produire sans qu'on le souhaite réellement (par un dysfonctionnement lié à un parasite par exemple).

Ce qui évite d'écrire n'importe quoi en EEPROM de données ou de programme.

L'EEPROM de données

Nous donnerons des exemples de modules en assembleur permettant d'accéder en lecture ou en écriture à l'EEPROM de données.

En TP, nous aurons à les écrire en langage C.

Accès en lecture

Pour lire un octet dans l'EEPROM de données, la procédure à respecter est la suivante :

① charger le registre SFR **EEADR** avec l'offset de la case EEPROM données concernée par la lecture.

Par exemple :

0 pour la case d'adresse absolue 0x210**0**,
1 pour la case d'adresse absolue 0x210**1**,
2 pour la case d'adresse absolue 0x210**2**...

② sélectionner la mémoire EEPROM de données : **EEPGD = 0**,

③ lancer un cycle de lecture : **RD = 1**,

Ce bit repassera automatiquement à 0 tout de suite après le transfert de la donnée entre l'EEPROM de données et le registre **EEDATA**.

④ récupérer le contenu du registre SFR **EEDATA** (qui contient alors le contenu de la case EEPROM concernée suite à la phase ③ précédente).

L'EEPROM de données (suite)

Les **4 phases précédentes** ont été écrites sur le module ci-dessous :

```

;*****
;
; module read_data_eeprom de prototype :
; char read_data_eeprom (char offset)
; lecture d'un octet à l'adresse offset de l' EEPROM de données
; en entrée :W contient l'offset dans l'EEPROM de données
; (paramètre passé par W)
; en sortie : le contenu de cette adresse est renvoyé dans W
;*****
read_data_eeprom

    banksel    EEADR                ; phase ①
    movwf     EEADR                ; initialisation de EEADR via W

    banksel    EECON1              ; phase ②
    bcf        EECON1,EEPGD        ; sélection EEPROM de données

    bsf        EECON1,RD           ; phase ③ : cycle de lecture

    banksel    EEADR                ; phase ④
    movf       EEDATA,W            ; récupération dans W

    return

;*****

```

Figure VIII.40. (page 257)

L'EEPROM de données (suite)

Accès en écriture

Pour écrire un octet dans l'EEPROM de données, la procédure à respecter est la suivante :

① charger le registre SFR **EEADR** avec l'offset de la case EEPROM données concernée par l'écriture.

Par exemple :

- 0** pour la case d'adresse absolue 0x210**0**,
- 1** pour la case d'adresse absolue 0x210**1**,
- 2** pour la case d'adresse absolue 0x210**2**...

② charger le registre SFR **EEDATA** avec la valeur à écrire dans la case EEPROM concernée,

③ sélectionner la mémoire EEPROM de données : **EEPGD = 0**,

④ ouverture d'une « fenêtre » d'écriture : **WREN = 1**,

⑤ masquer toutes les interruptions pour préparer la « clef de protection » : **GIE = 0**,

⑥ écrire les 5 instructions (qui ne doivent pas être interrompues) fournies par **MICROCHIP** pour réaliser la clef de protection contre les écritures « intempestives »,

L'EEPROM de données (suite)

⑦ on peut alors ré autoriser les interruptions : **GIE = 1**,

⑧ fermeture de la « fenêtre » d'écriture : **WREN = 0**,

Mais à partir de là, la valeur **n'est pas encore écrite dans l'EEPROM** de données.

Elle ne le sera réellement que lorsque le bit **WR** passera à 0 (et en même temps le bit **EEIF** passera de son côté à 1).

Pour éviter d'effectuer une autre écriture en EEPROM avant que la valeur courante ne soit écrite, il faut donc attendre (par pooling) que le bit **WR** passe à 0 (ou que le bit **EEIF** passe à 1). Car sinon, il y aura un conflit d'écriture en EEPROM.

C'est l'objectif de la phase ⑨ suivante.

⑨ attendre par une boucle de scrutation que le bit **WR** passe à 0. Compter environ 4 à 5 ms (temps d'écriture typique dans une case EEPROM), ce qui est énorme par rapport au temps de cycle instruction qui est de 1 µs pour un quartz à 4 MHz.

Remarque:

Pour phase ⑨, on aurait pu aussi attendre que le bit **EEIF** passe à 1. Mais il aurait alors fallu ensuite ne pas oublier de le remettre à 0 !!!

L'EEPROM de données (suite)

Les **9 phases précédentes** ont été écrites sur le module ci-dessous :

```
*****  
;  
; module write_data_eeprom de prototype :  
; void write_data_eeprom (char offset, char valeur)  
; écriture octet valeur à l'adresse offset de l' EEPROM de données  
; en entrée : W contient l'offset dans l' EEPROM  
(paramètre passé par W)  
; write_data_eeprom_par_1 : valeur à écrire  
(paramètre 1)  
; en sortie : aucune valeur retournée (void)  
*****  
write_data_eeprom  
    banksel    EEADR                ; phase ①  
    movwf     EEADR                ; initialisation de EEADR via W  
  
    movf      write_data_eeprom_par_1,W ; phase ②  
    movwf     EEDATA                ; initialisation EEDATA  
                                    ; via W  
  
    banksel    EECON1              ; phase ③  
    bcf       EECON1,EEPGD         ; sélection EEPROM de données  
  
    bsf       EECON1,WREN          ; phase ④  
                                    ; ouverture « fenêtre » d'écriture
```

Figure VIII.41. (page 259)

L'EEPROM de données (suite)

```

bcf          INTCON,GIE ; phase ⑤ : interdiction des IT
                                ; pendant les 5 instructions
                                ; instructions suivantes
                                ; non interruptibles

movlw        0x55          ; phase ⑥ : « clef de protection »
movwf        EECON2        ; ces 5
movlw        0xAA          ; instructions
movwf        EECON2        ; ne doivent pas être interrompues
bsf          EECON1,WR      ; lancement de l'écriture

bsf          INTCON,GIE     ; phase ⑦ : on ré autorise les IT

bcf          EECON1,WREN    ; phase ⑧
                                ; fermeture « fenêtre » d'écriture

write_data_eeeprom_attend    ; phase ⑨
    btfsc    EECON1,WR      ; écriture EEPROM terminée ?
    goto     write_data_eeeprom_attend ; non, alors on attend

    return                  ; oui, alors fin du module...
;*****

```

Figure VIII.41. suite (page 259)

L'EEPROM de données (suite et fin)

*Pour améliorer la fiabilité de ce module **write_data_eeprom**, on peut y rajouter les instructions qui permettent de vérifier que la valeur qu'on souhaite écrire en EEPROM de données a bien été écrite en EEPROM de données.*

Comment cela?

Tout simplement en enchaînant une lecture de la case mémoire où on vient d'écrire : on peut ainsi comparer que la valeur lue correspond bien à celle qu'on souhaitait écrire.

Ce qui revient à faire du « write and verify ».

Dans ce cas là, il faut concevoir une fonction dont le prototype serait :

*char **write_and_verify_data_eeprom** (char offset, char valeur)*

*et donc qui renvoie en retour **un code erreur** comme par exemple :*

- soit **0x01**, si **tout s'est bien passé** :

*valeur lue **identique** à la valeur qu'on voulait écrire,*

- soit **0x00**, s'il y a eu **un problème** :

*valeur lue **différente** de la valeur qu'on voulait écrire.*

Le Convertisseur Analogique Numérique

Généralités et principes

Pour passer du domaine de **l'analogique** (où les grandeurs physiques comme $V_a(t)$ sont **continues** et donc valables $\forall t$) au domaine du **numérique** (où les variables sont **discrètes** et deviennent des **nombres** $V_a(t_1)$, $V_a(t_2)$, $V_a(t_3)$, ... $V_a(t_n)$, représentant la valeur du signal analogique à certains instants particuliers t_1 , t_2 , t_3 , ... t_n , appelés **instants d'échantillonnage**).

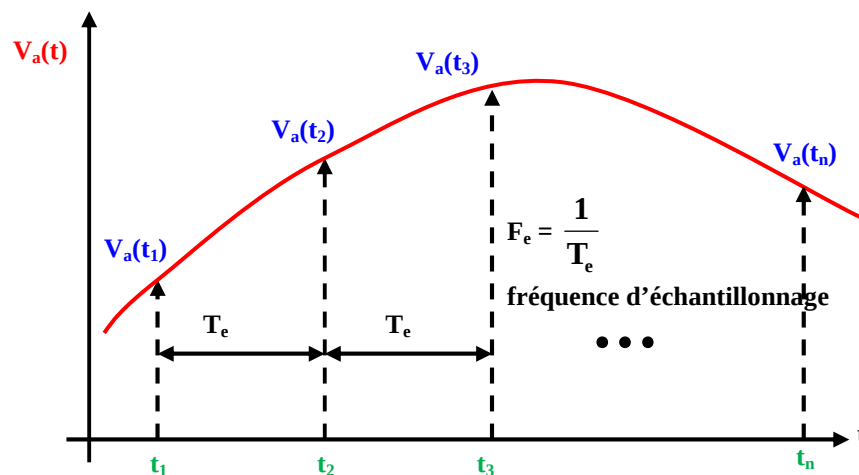


Figure VIII.46. (page 268)

Les applications sont nombreuses (liste non exhaustive...) :

- **phénomènes lents** : mesure d'une température, d'une pression, du degré d'humidité de l'air, de la vitesse du vent ou d'un véhicule, d'une concentration chimique, du module d'une force...
- **phénomènes un peu plus rapides** : numérisation du son, de la voix, de la musique...
- **phénomènes encore plus rapides** : numérisation d'une image, vidéo numérique, imagerie médicale...

Le Convertisseur Analogique Numérique (suite)

Correspondance Analogique/Numérique

La tension **analogique** V_{CAN} à l'entrée du CAN varie de **0** à V_{REF} .

L'intervalle $[0; V_{REF}]$ détermine la **plage de conversion**
(Full Scale Input Voltage)

La valeur **numérique** N est ici codée sur **10 bits** :

de $0b00\ 0000\ 0000 = 0x000 = (0)_{10}$ à $0b11\ 1111\ 1111 = 0x3FF = (1023)_{10}$

V_{REF} est divisée en $2^{10} = (1024)_{10}$ intervalles égaux de taille $q = \frac{V_{REF}}{2^{10}}$

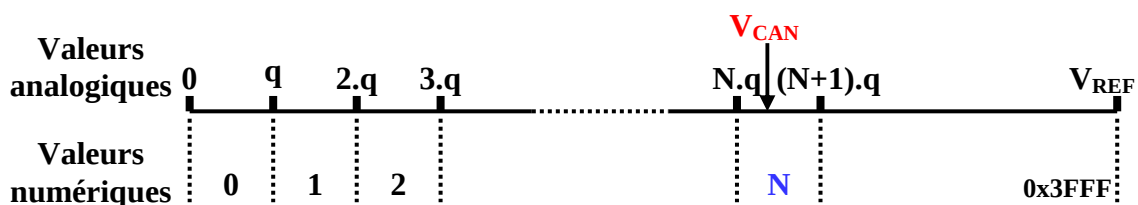


Figure VIII.52. (page 273)

q est appelée le **pas de quantification** (lié à la précision).

Exemple : pour $V_{REF} = 5V$, on obtiendra : $q \approx 5\text{ mV}$.

On a donc : $N = E\left(\frac{V_{CAN}}{q}\right)$ avec $E(\) =$ partie entière de $(\)$.

Réciproquement, comme V_{CAN} est dans l'intervalle $[N.q ; (N+1).q]$:

$$V_{CAN} \approx \left(N + \frac{1}{2}\right) \cdot q \left(\pm \frac{q}{2}\right)$$

On parle d'une erreur égale à $\frac{1}{2} \text{ LSB}$. Ici ce sera $\frac{q}{2} = \frac{V_{REF}}{2^{11}} = \frac{V_{REF}}{2048}$.

Le Convertisseur Analogique Numérique (suite)

Linéarité et précision

Les relations précédentes se résument sur la **courbe de transfert** entre valeur **analogique** et valeur **numérique** :

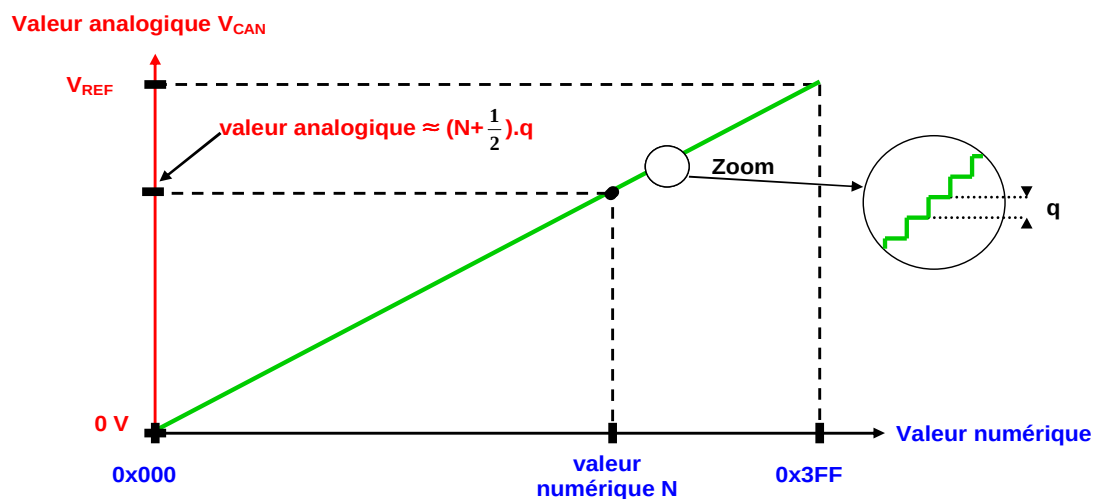


Figure VIII.51. (page 273)

elle est **linéaire**, bien qu'en fait elle soit constituée d'un très grand nombre ($2^{10} = 1024$) de marches d'escaliers (cf. zoom ci-dessus).

Remarque générale sur les CAN

Chaque fois qu'on rajoute un bit pour le codage en numérique de N , on a un convertisseur deux fois plus précis.

Avec 10 bits comme ici, la précision obtenue est largement suffisante dans la plupart des applications...

Le Convertisseur Analogique Numérique (suite)

Pour cette présentation résumée, on ne peut rentrer dans le détail des différents types de convertisseurs et du processus complet d'échantillonnage (voir **pages 268 à 276 du polycopié de cours complet**).

Ce qu'il faut néanmoins savoir, c'est qu'il y a 2 étapes importantes imposant une certaine chronologie et des durées qui s'ajoutent :

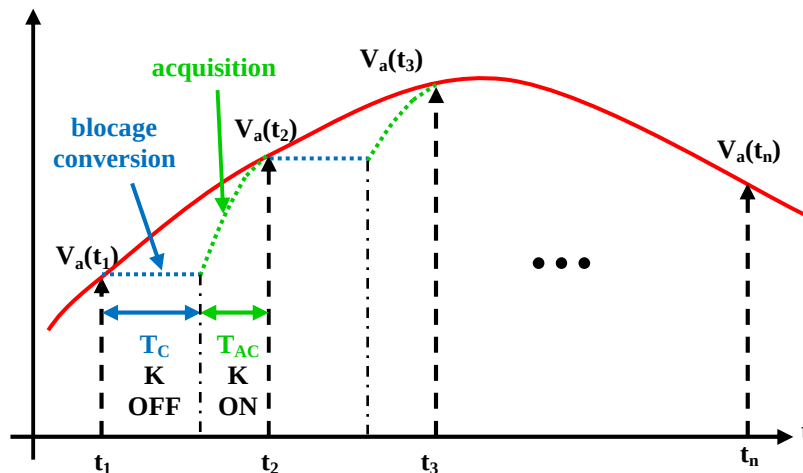
① **la conversion analogique/numérique** (non instantanée) mais qui dure un temps (**temps de conversion T_C**). **La lecture de la valeur numérique doit se faire uniquement quand la conversion est finie (donc après la durée T_C car sinon on obtient un résultat erroné).**

Cette phase est automatiquement précédée du **blocage** du signal analogique pour qu'il soit constant à l'entrée du CAN pendant la phase de conversion (par un bloqueur : interrupteur K sur **OFF** + condensateur). Puis, elle est aussi automatiquement suivie par un **déblocage** de ce même signal analogique. D'où la phase suivante.

② **l'acquisition** : à cause du déblocage (K sur **ON** + circuit RC), l'entrée analogique du CAN prend un certain temps pour retrouver la valeur du signal analogique (**temps d'acquisition T_{AC}**)

Le Convertisseur Analogique Numérique (suite)

On distingue ces 2 phases sur le schéma ci-dessous :



Quelques ordres de grandeur :

Le **temps de conversion T_C** égale au moins à $12.T_{AD}$ où T_{AD} est la durée de conversion pour obtenir chaque bit de N.

(car le convertisseur est à approximations successives + ouverture K + fermeture K du bloqueur)

Pour $T_{AD} = 2 \mu s$ (voir plus loin), on obtient : **$T_C = 24 \mu s$** .

De « savants calculs » pour **T_{AC}** montrent que : **$T_{AC} = 14 \mu s$** .

La **période d'échantillonnage T_e** sera donc **$> 38 \mu s$** .

Par exemple $T_{emin} = 40 \mu s$, soit une fréquence d'échantillonnage $F_{emax} = \frac{1}{T_{emin}} = 25 \text{ kHz}$. Et donc, d'après le

théorème de Shanon, ($2.F_{max} \leq F_e$), on ne pourra traiter numériquement des signaux avec ce PIC que si leur fréquence max F_{max} est plus petite que :

12 à 12,5 kHz.

Le Convertisseur Analogique Numérique (suite)

Synoptique général d'implantation dans le PIC

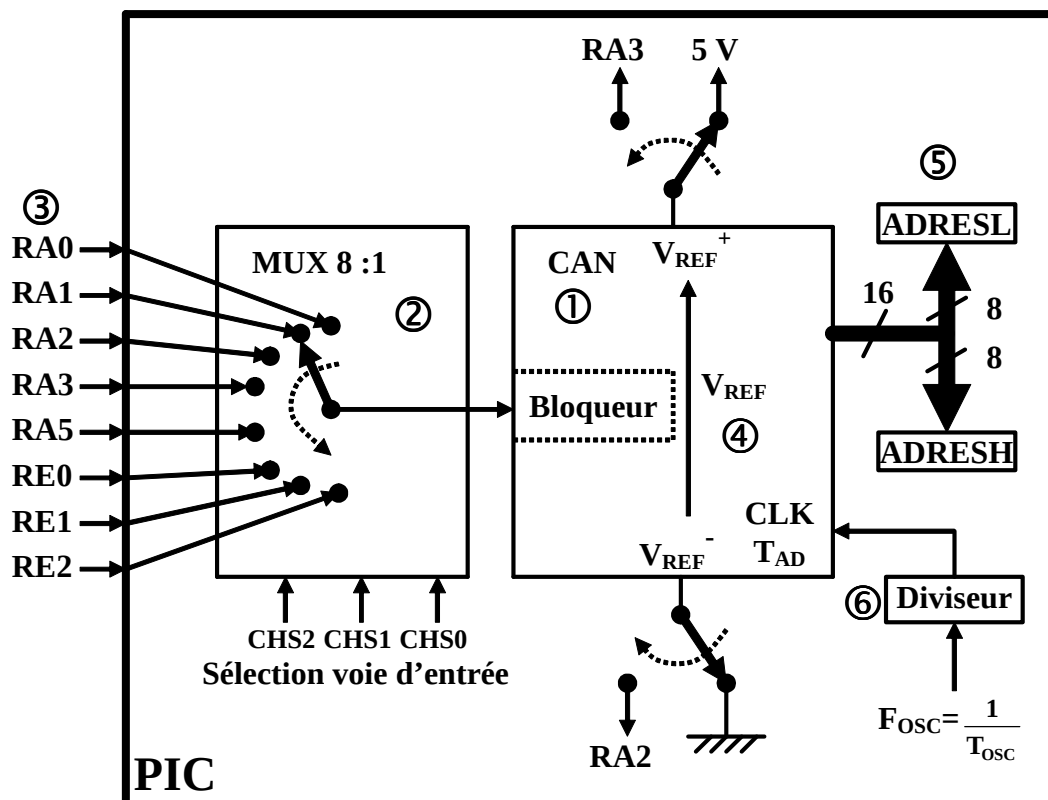


Figure VIII.55. (page 277)

Résumé du cours PIC : chapitre VIII.4.
(informations supplémentaires pendant manipulations TP 2)

Le Convertisseur Analogique Numérique (suite)

Registres associés pour utiliser le CAN

Deux registres SFR **ADCON0** et **ADCON1** pour piloter le CAN

ADCON0 (situé à l'adresse 0x01F)
(page 111 du datasheet)

	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	U-0	R/W-0
ADCON0	ADCS1	ADCS0	CHS2	CHS1	CHS0	GO/ DONE	–	ADON
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0

Bit	Valeur	Signification et commentaires
CHS2 :CHS0		Pour sélectionner le canal de conversion
	000	Canal 0 : broche d'entrée RA0/AN0 reliée au CAN
	001	Canal 1 : broche d'entrée RA1/AN1 reliée au CAN
	010	Canal 2 : broche d'entrée RA2/AN2 reliée au CAN
	011	Canal 3 : broche d'entrée RA3/AN3 reliée au CAN
	100	Canal 4 : broche d'entrée RA5/AN4 reliée au CAN
	101	Canal 5 : broche d'entrée RE0/AN5 reliée au CAN
	110	Canal 6 : broche d'entrée RE1/AN6 reliée au CAN
	111	Canal 7 : broche d'entrée RE2/AN7 reliée au CAN
ADCS1 :ADCS0		Choix du diviseur pour fixer T_{AD} (voir plus loin)
	00	Diviseur = 2 donc T _{AD} = 2.T_{OSC}
	01	Diviseur = 8 donc T _{AD} = 8.T_{OSC}
	10	Diviseur = 32 donc T _{AD} = 32.T_{OSC}
	11	T _{AD} fourni par oscillateur RC dans CAN (mode SLEEP)
ADON		Permet de mettre en route le CAN
	0	Le CAN est éteint
	1	Met en route le CAN : on peut alors l'utiliser
GO/ DONE		Bit de commande et d'état
	1	Si on le met à 1, lance une conversion (le faire après avoir mis bit ADON à 1) et il reste à 1 pendant la conversion.
	0	Mis à 0 automatiquement à la fin de la conversion

Résumé du cours PIC : chapitre VIII.4.
(informations supplémentaires pendant manipulations TP 2)

Le Convertisseur Analogique Numérique (suite)

Registres associés pour utiliser le CAN (suite)

ADCON1 (situé à l'adresse 0x09F)
(page 112 du datasheet)

	R/W-0	U-0	U-0	U-0	R/W-0	R/W-0	R/W-0	R/W-0
ADCON1	ADFM	–	–	–	PCFG3	PCFG2	PCFG1	PCFG0
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0

Bit	Valeur	Signification et commentaires
PCFG3 :PCFG0	Voir ci-dessous	Permet d'affecter les lignes concernées des PORTA et PORTE soit comme E/S TOR (D), soit comme entrées analogiques (A) et de choisir la plage de conversion V_{REF}^{+} V_{REF}^{-} (comme indiqué sur tableau ci-dessous où VSS (0V) et VDD(5V))
ADFM		Format du résultat numérique de la conversion (voir plus loin)
	0	Justification à gauche
	1	Justification à droite

PCFG3: PCFG0	AN7 ⁽¹⁾ RE2	AN6 ⁽¹⁾ RE1	AN5 ⁽¹⁾ RE0	AN4 RA5	AN3 RA3	AN2 RA2	AN1 RA1	AN0 RA0	VREF+	VREF-	CHAN/ Refs ⁽²⁾
0000	A	A	A	A	A	A	A	A	VDD	VSS	8/0
0001	A	A	A	A	VREF+	A	A	A	RA3	VSS	7/1
0010	D	D	D	A	A	A	A	A	VDD	VSS	5/0
0011	D	D	D	A	VREF+	A	A	A	RA3	VSS	4/1
0100	D	D	D	D	A	D	A	A	VDD	VSS	3/0
0101	D	D	D	D	VREF+	D	A	A	RA3	VSS	2/1
011x	D	D	D	D	D	D	D	D	VDD	VSS	0/0
1000	A	A	A	A	VREF+	VREF-	A	A	RA3	RA2	6/2
1001	D	D	A	A	A	A	A	A	VDD	VSS	6/0
1010	D	D	A	A	VREF+	A	A	A	RA3	VSS	5/1
1011	D	D	A	A	VREF+	VREF-	A	A	RA3	RA2	4/2
1100	D	D	D	A	VREF+	VREF-	A	A	RA3	RA2	3/2
1101	D	D	D	D	VREF+	VREF-	A	A	RA3	RA2	2/2
1110	D	D	D	D	D	D	D	A	VDD	VSS	1/0
1111	D	D	D	D	VREF+	VREF-	D	A	RA3	RA2	1/2

A = Analog input D = Digital I/O

Le Convertisseur Analogique Numérique (suite)

Remarque sur bit **ADFM** de **ADCON1**

Le résultat de la conversion analogique/numérique est placé dans les deux registres SFR, de taille 8 bits chacun :

ADRESH (comme **High**) et **ADRESL** (comme **Low**),
ce qui est suffisant pour les 10 bits du résultat.

Il existe deux formats pour stocker les 10 bits **D9** à **D0** du résultat.

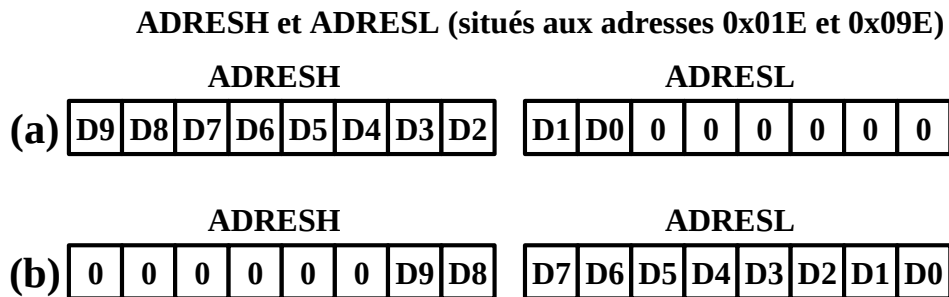


Figure VIII.61. (page 280)

- justification à **gauche (a)** : 10 bits du résultat « rangés » dans **ADRESH/ADRESL** en se « calant » à partir de la gauche de **ADRESH**. Les 6 bits poids faibles de **ADRESL** mis à **0** par CAN.

Intérêt : en ne lisant que **ADRESH**, on ne récupère que les 8 bits de poids forts du résultat ($\equiv$ précision d'un CAN sur 8 bits, suffisante dans certaines applications). Le résultat de la conversion sera alors déclaré comme : **unsigned char** (taille 1 octet).

- la justification à **droite (b)** : 10 bits du résultat « rangés » dans **ADRESH/ADRESL**, en se « calant » à partir de la droite de **ADRESL**. Les 6 bits poids forts de **ADRESH** mis à **0** par le CAN.

Intérêt : on obtient la précision maximum sur 10 bits du CAN. Le résultat de la conversion sera alors déclaré comme :
unsigned short int (taille 2 octets).

Le Convertisseur Analogique Numérique (suite)

Remarque sur bits ADCS1 et ADCS0 de ADCON0

Pour que le CAN fonctionne correctement, il faut que $T_{AD} \geq 1,6 \mu s$.
En pratique, il faudra choisir la plus petite valeur de T_{AD} supérieure à $1,6 \mu s$, pour avoir le temps de conversion le plus petit possible.

Diviseur \ $F_{osc}(T_{osc})$	4 MHz (0,25 μs)	20 MHz (0,05 μs)
2	0,5 μs	0,1 μs
8	2,0 μs	0,4 μs
32	8,0 μs	1,6 μs

Les valeurs barrées ne seront donc pas possibles, car $\leq$ à $1,6 \mu s$.
Avec un quartz 4 MHz, et un diviseur par 8, on obtient : $T_{AD} = 2 \mu s$.

Comment alors gérer une conversion ?

① par **polling** (scrutation) sur le bit **GO/DONE** de **ADCON0**

- lancer une conversion en forçant ce bit à **1** (**GO**), puis attendre par polling qu'il passe à **0** quand la conversion est terminée (**DONE**).
- lire le résultat dans les registres **ADRESH** (et **ADRESL**).

② par **interruption**, car quand la conversion est finie, le flag **ADIF** du registre SFR **PIR1** passe à **1** et une IT sera générée si le bit **ADIE** de **PIE1** a été mis à **1** (démasquage local de cette IT).

Le Convertisseur Analogique Numérique (suite)

Séquencement d'une conversion

Au final, voilà les différents points à prendre en compte :

- phase d'initialisation :

- gérer les 4 bits **PCFG3, PCFG2, PCFG1, PCFG0** de **ADCON1** : pour configurer les broches du PIC et choisir la plage de conversion.
- gérer le bit **ADFM** de **ADCON1** : pour le format du résultat.
- gérer les 2 bits **ADCS1** et **ADSC0** de **ADCON0** : pour fixer une valeur de $T_{AD} \geq 1,6 \mu s$ (en fonction du quartz).
- gérer les 3 bits **CHS2, CHS1** et **CHS0** de **ADCON0** : pour sélectionner le canal concerné par la conversion,
- gérer le bit **ADON** de **ADCON0** : pour mettre en route le CAN.

- phase de conversion (en technique de polling) :

- attendre le temps d'acquisition avant toute nouvelle conversion
- lancer une conversion via le bit **GO/DONE** de **ADCON0**
- attendre la fin de conversion par scrutation de ce bit **GO/DONE**
- lire le résultat conversion dans les registres **ADRESH/ADRESL** (ou **ADRESH** seulement).

Il est **fortement** conseillé, pour la structuration du programme, d'écrire chacune de ces 2 phases sous la forme d'un module (si on utilise l'assembleur) ou d'une fonction (si on programme en C).

Les Timers internes

Généralités sur la fonction timer

Temporisations logicielles : beaucoup d'inconvénients...
manque de précision, dépendant du quartz, temps CPU perdu...

D'où l'implantation dans le PIC d'un bloc **timer** qui fournit une **référence temporelle** indispensable dans beaucoup d'applications :

- gérer des temporisations et/ou des délais très **précisément**,

Mais aussi :

- connaître un instant de façon **exacte** pour «**dater**» un évènement extérieur,

- **mesurer des intervalles de temps** comme avec un chronomètre,

- générer un **front de tension en sortie** à un instant précis pour **piloter** un périphérique extérieur,

- générer des **signaux de caractéristiques temporelles précises** :
signaux carrés, signaux de rapport cyclique donné ou tout autre type de signal, plus ou moins complexe...

- fabriquer des « **chiens de garde** » ou « **watch-dog** ».

Les Timers internes (suite)

Le **cœur du timer** est un registre de type **compteur** (appelé ici **TIMER**) de taille 8 ou 16 bits qui **s'incrémente en permanence** à chaque impulsion de son horloge d'entrée, appelée **tick** d'horloge.

La période T_{tick} de ces ticks d'horloge à l'entrée du registre **TIMER** est appelée la **granularité** du timer.

Attention, à ne pas confondre T_{tick} et T_{osc} (période du quartz).

Cette horloge d'entrée de **TIMER** (le tick) est souvent fabriquée à partir du quartz, grâce à une chaîne en cascade de 2 **prédiviseurs** :

- un **pré diviseur de valeur fixe f** (fixée par **construction et non modifiable**),
- un **pré diviseur de valeur programmable p** (valeur choisie par **programmation**),

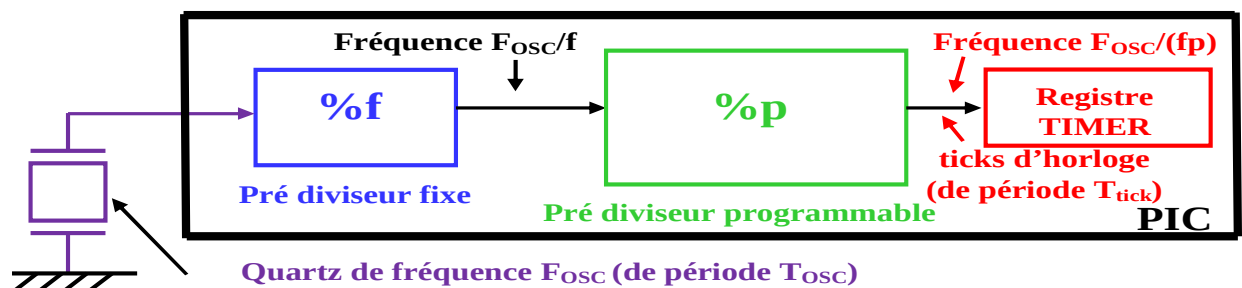


Figure VIII.66. (page 289)

Pour les PIC, $f = 4$, et du coup : $T_{\text{tick}} = f \cdot p \cdot T_{\text{osc}} = 4 \cdot p \cdot T_{\text{osc}}$

Comme $T_{\text{tick}} > T_{\text{osc}}$, on pourra fabriquer des délais de valeurs **assez grandes** même avec des registres **TIMER** de taille 8 ou 16 bits.

On peut aussi utiliser la sortie d'un capteur d'objets qu'on injecte alors directement à l'entrée du registre **TIMER** à la place des « ticks d'horloge » : le timer se transforme alors **en compteur d'objets**.

Les Timers internes (suite)

Pour les PIC, il existe deux types de registre **TIMER** :

① les **TIMER** de type ① : à valeur initiale **VI = 0** (toujours) et à valeur finale **VF** **programmable**

ils comptent donc de **VI = 0** jusqu'à **VF**, et positionnent un flag qu'on appelle ici **TIMER_IF** à **1** dès que la valeur VF est atteinte (ou plus exactement quand le compteur passe de **VF** à **VF+1**)

On parle alors de **débordement** (ou **overflow**) du timer.

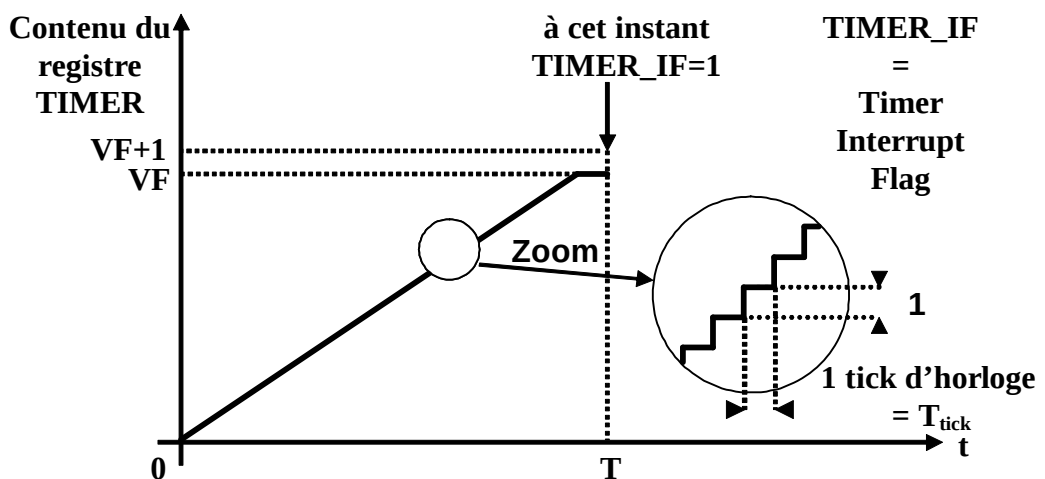


Figure VIII.67. (page 290)

La durée **T** mesurée par ce type de timer est donc donnée par :

$$(1) \quad \boxed{T = T_{\text{Tick}} \cdot (VF + 1) = f.p. \cdot T_{\text{osc}} \cdot (VF + 1)}$$

avec $T_{\text{osc}} = \frac{1}{F_{\text{osc}}} = \text{période du quartz du PIC.}$

Les Timers internes (suite)

Exemple

Avec $f = 4$, $p = 16$, $VF = 0xFF = 255$ (choisis par programmation ou plus exactement par initialisation du timer) et avec un quartz de $F_{OSC} = 4 \text{ MHz}$ soit $T_{OSC} = 250 \text{ ns}$:

$$T = T_{\max} = 4.16.256.250 \text{ ns} = 4,096 \text{ ms}$$

C'est la plus grande valeur de temporisation qu'on puisse avoir **pour cette valeur de p**.

Réciproquement, l'équation **(1)** nous permettra de trouver les valeurs de **p** et de **VF** en fonction de la valeur de **T** que l'on souhaite obtenir.

Comme nous avons deux inconnues **p** et **VF**, plusieurs solutions seront possibles, et parmi celles-ci :

- certaines ne seront **pas implantables** (toutes les valeurs de **p** ne sont pas disponibles),
- et d'autres seront plus **précises** (en général celles qui correspondent à la **plus petite** valeur implantable de **p** car dans ce cas là, le tick d'horloge sera le plus petit). On dit que l'on améliore ainsi la **granularité temporelle**, et donc la précision du timer...

Les Timers internes (suite)

② les **TIMER** de type ②: à valeur initiale **VI programmable** et à valeur finale (toujours) : **VF = 0xFF** (si taille 8 bits) ou **VF = 0xFFFF** (si taille 16 bits).

ils comptent donc de **VI** jusqu'à **VF = 0xFF** (ou **0xFFFF**), et positionnent un flag qu'on appelle ici **TIMER_IF** à 1 dès que la valeur 0x00 (ou 0x0000) est atteinte (c'est-à-dire 1 tick d'horloge après avoir atteint la valeur 0xFF (ou 0xFFFF)).

On parle alors de **débordement** (ou **overflow**) du timer.

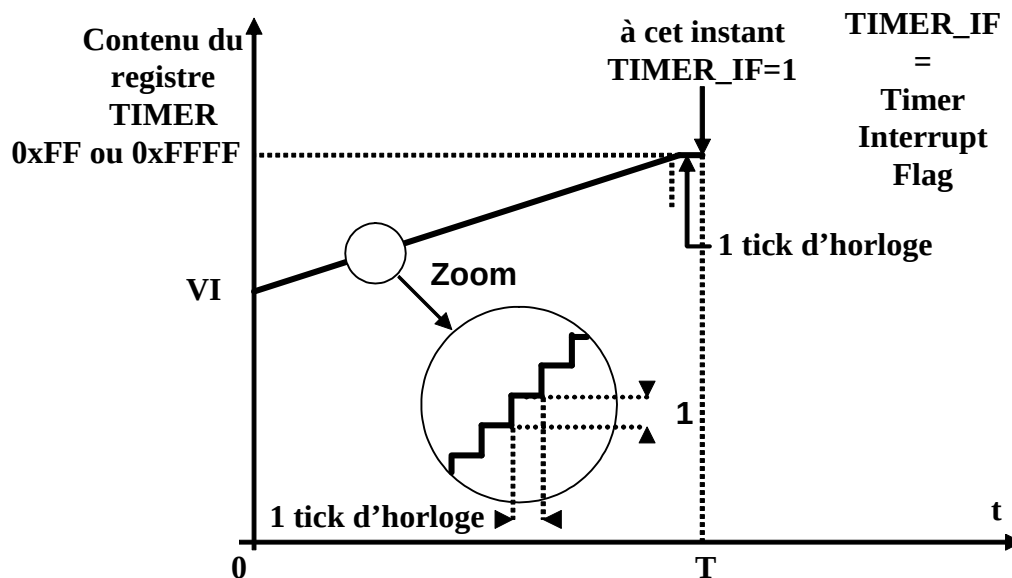


Figure VIII.68. (page 291)

La durée **T** mesurée par ce type de timer est donc donnée par :

$$(2) \quad \boxed{T = T_{\text{Tick}} \cdot (2^n - VI) = f.p. \cdot T_{\text{Osc}} \cdot (2^n - VI)}$$

avec : $T_{\text{Osc}} = \frac{1}{F_{\text{osc}}} = \text{période du quartz du PIC.}$

n = nombre de bits du registre **TIMER** (8 ou 16)

Les Timers internes (suite)

Exemple

Avec **f = 4**, **p = 8**, **VI = 0x0000** (choisis par programmation ou plus exactement par initialisation du timer de taille prise ici à 16 bits) et avec un quartz de **F_{OSC} = 4 MHz** soit **T_{OSC} = 250 ns** :

$$T = T_{\max} = 4.8.65536.250 \text{ ns} = 524,288 \text{ ms}$$

C'est la plus grande valeur de temporisation qu'on puisse avoir **pour cette valeur de p**.

Réciproquement, l'équation **(2)** nous permettra de trouver les valeurs de **p** et de **VI** en fonction de la valeur de **T** que l'on souhaite obtenir.

Comme nous avons deux inconnues **p** et **VI** (**n** étant fixé par la taille du registre **TIMER**) plusieurs solutions seront possibles, et parmi celles-ci :

- certaines ne seront **pas implantables** (toutes les valeurs de **p** ne sont pas disponibles),
- et d'autres seront **plus précises** (en général celles qui correspondent à la **plus petite** valeur implantable de **p** car dans ce cas là, le tick d'horloge sera le plus petit). On dit que l'on améliore ainsi la granularité temporelle, et donc la précision du timer.

Les Timers internes (suite)

Comment gérer ce flag **TIMER_IF** ?

On connaît la réponse : soit par **pooling**, soit par **interruption**.

① technique par pooling :

Après avoir fait toutes les initialisations nécessaires (choix du **pré diviseur programmable p**, écriture de **VF** ou de **VI** , sans oublier de mettre le flag **TIMER_IF** à **0**) on lance le timer et on attend que ce flag passe à **1**, ce qui est le signe que la durée T programmée s'est bien écoulée.

Pour cette attente, il faudra écrire en C une ligne du type :

```
while ( TIMER_IF == 0 ) ; // tant que TIMER_IF = 0, on attend  
(ou while( !TIMER_IF ) ; // idem que la ligne précédente)
```

Immédiatement suivie de :

```
TIMER_IF = 0; // pour remettre ce flag à 0.
```

Finalement, ce n'est pas un gain important par rapport à la solution des temporisations logicielles : au lieu de « perdre » du temps en exécutant des boucles de temporisations « calibrées et calculées », ici on perd du temps « à attendre » que le flag **TIMER_IF** passe à **1**. Dans les deux cas, on ne fait rien en parallèle.

Le seul avantage d'utiliser la fonction timer avec la technique du pooling est de pouvoir plus facilement, plus rapidement et de façon plus précise réaliser les temporisations.

Les Timers internes (suite)

② technique par interruption :

Il existe aussi un flag qu'on appellera dans cette présentation générale **TIMER_IE** (comme **TIMER Interrupt Enable**) qui permet de démasquer l'interruption du timer.

Quand la durée T souhaitée s'est écoulée, le flag **TIMER_IF** passe à **1** et une IT sera générée si ce flag **TIMER_IE** a été mis à **1**.

Il faut alors penser à :

- **dans la fonction d'initialisation** du timer :
 - initialiser correctement le **pré diviseur programmable p** et éventuellement écrire une valeur initiale **VI** (ou une valeur finale **VF**) dans le registre **TIMER**,
 - penser à **mettre en route** le timer (si c'est nécessaire) et à mettre le flag **TIMER_IF** à 0,
 - gérer les différents **masques** d'interruptions : local (**TIMER_IE**), semi global (**PEIE**) et global (**GIE**),
- **et dans la routine d'IT** :
 - **acquitter** l'IT via le bit **TIMER_IF** (en le remettant à **0**),
 - **éventuellement** écrire une valeur initiale **VI** (ou une valeur finale **VF**) dans le registre **TIMER** du timer,
 - effectuer le **traitement associé** au fait que la durée T programmée s'est écoulée.

Les Timers internes (suite)

Dans le **PIC 16F877**, il existe 3 timers **indépendants** appelés :

TIMER0, TIMER1 et TIMER2.

Chacun possède :

- son propre pré diviseur **fixe**, de valeur **f = 4**,
- son propre pré diviseur **programmable** de valeur **p**,
- son propre registre **TIMER**,
- son propre flag **TIMER_IF**,
- son propre flag **TIMER_IE**.

Les Timers internes (suite)

Le TIMER0 : principales caractéristiques

Elles sont résumées ci-dessous :

- **8** valeurs possibles du pré-diviseur **p** programmable :
2, 4, 8, 16, 32, 64, 128 et 256,
- registre TIMER appelé **TMR0** sur **8 bits** et accessible en lecture ou en écriture,
- fonctionnement de **VI** (valeur initiale de **8 bits** à charger dans le registre **TMR0**) à **0xFF** : timer de type ②,
- le débordement est signalé par le flag **T0IF** (Timer **0** Interrupt Flag, situé dans le registre SFR **INTCON**) qui passe à **1** quand le registre **TMR0** passe de **0xFF** à **0x00**,
- un bit **T0IE** (comme Timer **0** Interrupt Enable également situé dans le registre SFR **INTCON**) permettra, localement, **d'autoriser** (si **T0IE=1**) ou **d'interdire** (si **T0IE=0**) la génération d'une interruption lorsque le flag **T0IF** passe à **1**,
- le **TIMER0** fonctionne en continu (pas de bit pour le mettre en route).

Les Timers internes (suite)

Le TIMER0 : synoptique

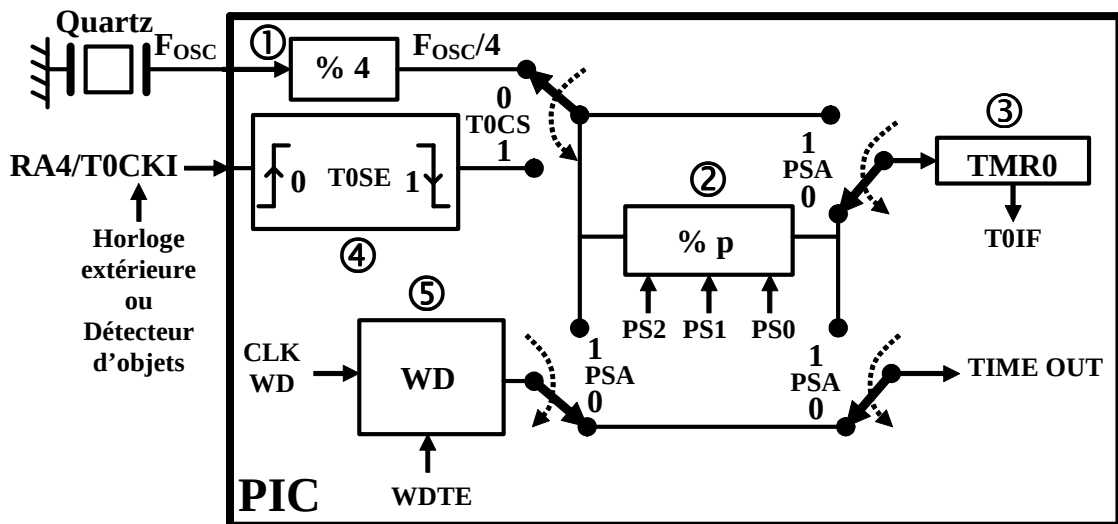


Figure VIII.70. (page 294)

- bloc ① : le pré diviseur **fixe** par $f = 4$.
- bloc ② : le pré diviseur **programmable** p , choisi par les 3 bits **PS2**, **PS1** et **PS0**.
- bloc ③ : le registre **TMR0**, avec son bit d'overflow **T0IF** qui passera à 1 quand le contenu de **TMR0** passe de **0xFF** à **0x00**.
- bloc ④ : mode comptage des impulsions d'une horloge extérieure ou des évènements extérieurs (détecteur d'objets par exemple) signalés sur la broche **RA4/T0CKI** (**à programmer en entrée !!!**)

Quatre multiplexeurs (3 pilotés par le bit **PSA**, et 1 piloté par le bit **T0CS**) pour choisir les différents modes de fonctionnement.

- bloc ⑤ : pour réaliser la fonction « **watch-dog** » générant un signal de **TIME OUT** (prévu pour provoquer un **RESET** du PIC en cas de problème).

Résumé du cours PIC : chapitre VIII.5.
(informations supplémentaires pendant manipulations TP 3)

Le TIMER0 : registre de contrôle associé

Les bits de configuration sont dans le registre **OPTION_REG** :

OPTION_REG REGISTER (situé aux adresses 0x081 et 0x181)
(pages 19 et 48 du datasheet)

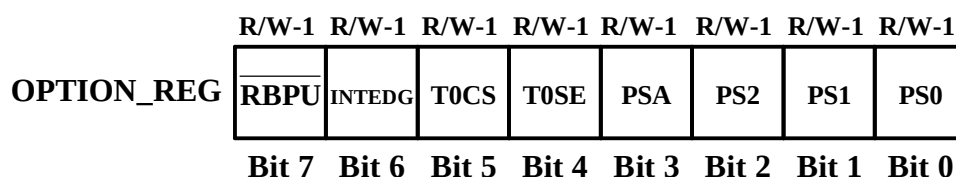


Figure VIII.71. (page 296)

Bit	Valeur	Signification et commentaires	
PSA		Permet d'affecter le pré diviseur programmable soit au TIMER0 , soit au WATCH DOG	
	0	Pré diviseur affecté au TIMER0	
	1	Pré diviseur affecté au WATCH DOG	
PS2 :PS0		Pour choisir la valeur p du pré diviseur programmable	
		Si PSA=0 (pour TIMER0)	Si PSA=1 (pour WD)
	000	2	1
	001	4	2
	010	8	4
	011	16	8
	100	32	16
	101	64	32
	110	128	64
	111	256	128
T0CS		Choix du mode de fonctionnement	
	0	Comptage du temps (via le quartz du PIC)	
	1	Horloge extérieure ou compteur d'évènements sur RA4	
T0SE		Choix du front actif comptage sur RA4	
	0	Front montant	
	1	Front descendant	

Figure VIII.72. (page 296)

Le **TIMER0** : exemple de mise en œuvre

Réaliser un signal carré de fréquence $f_0 = 1 \text{ kHz}$ sur la broche **RD1**.

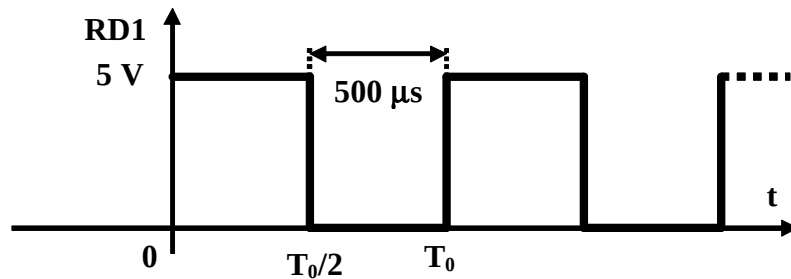


Figure VIII.73. (page 297)

$$T_0 = \frac{1}{f_0} = 1 \text{ ms.}$$

Il faut changer l'état de la sortie **RD1** toutes les $0,5 \text{ ms} = 500 \mu\text{s}$.

On va donc générer, grâce au **TIMER0**, une tempo de $500 \mu\text{s}$.

La formule (2) précédente donne :

$$500 \mu\text{s} = f.p.(2^n - VI).T_{\text{OSC}} \quad \text{avec } n = 8 \text{ et } T_{\text{OSC}} = 250 \text{ ns (4 MHz),}$$

$$500\,000 = 4.p.(256 - VI).250 \text{ et donc : } p.(256 - VI) = 500.$$

On prend la plus petite valeur implantable pour p (pour avoir la plus grande précision).

Le tableau de la figure VIII.72. précédente donne $p_{\min} = 2$.

$$256 - VI = 250 \quad \text{ce qui nous fournit : } \mathbf{VI = 6}$$

(valeur initiale à charger dans le registre **TMR0** du **TIMER0**)

Le TIMER0 : par pooling

```
void main(void)
{
    // initialisation du PORTD
    TRISD1 = 0 ;                // RD1 en sortie

    // initialisation du registre OPTION_REG
    T0CS = 0 ;                  // quartz du PIC
    T0SE = 0 ;                  // indifférent la valeur 1 irait aussi
    PSA = 0 ;                   // pré diviseur affecté au TIMER0
    PS2 = 0 ; PS1 = 0 ; PS0 = 0 ; // p = 2

    // initialisation du registre TMR0
    TMR0=6 ;                    // VI=6 (voir calcul précédent)

    // initialisation du flag T0IF
    T0IF=0 ;                    // mise à 0 du flag T0IF pour préparer
                                // la première scrutation

    while (1)                   // boucle sans fin
    {
        while(!T0IF) ;          // on attend que T0IF passe à 1

        T0IF=0 ;                // remise à 0 du flag T0IF pour
                                // préparer la scrutation suivante

        TMR0=6 ;                // recharge VI pour la demie période
                                // suivante

        RD1 = ~ RD1 ;           // changement état RD1
    }
}
```

Résumé du cours PIC : chapitre VIII.5.
(informations supplémentaires pendant manipulations TP 3)

Le TIMER0 : par interruption

Pour que le programme principal fasse autre chose (génération du carré sans intervention du main), il faut utiliser les interruptions. Le programme suivant réalise la même chose que le précédent mais le main est une simple boucle sans fin du type **while(1)**.

```
void interrupt timer0_it (void)    // la routine d'IT qui génère le signal carré
{
    if(T0IF && T0IE)                // IT sur overflow du TIMER0
    {
        T0IF = 0 ;                  // acquittement IT TIMER0
        TMR0 = 6 ;                  // recharge VI dans TMR0
        RD1 = ~ RD1 ;               // changement état RD1 à chaque IT
    }
}

void main(void)
{
    // initialisation du PORTD
    TRISD1 = 0 ;                    // RD1 en sortie
    // initialisation du registre OPTION_REG
    T0CS = 0 ;                      // quartz du PIC
    T0SE = 0 ;                      // indifférent la valeur 1 irait aussi
    PSA = 0 ;                       // pré diviseur affecté au TIMER0
    PS2 = 0 ; PS1 = 0 ; PS0 = 0 ;   // p = 2
    // initialisation du registre TMR0
    TMR0=6 ;                        // prépare première IT
    // initialisation du flag T0IF
    T0IF=0 ;                        // efface flag «overflow» du TIMER0
    // démasquage des interruptions
    T0IE=1 ;                        // démasquage local de l'IT TIMER0
    GIE=1 ;                         // démasquage de toutes les IT
    while(1) ;                      // ici le main ne fait rien donc pourrait
                                    // faire toute autre chose
}
```

Les Timers internes (suite)

Le TIMER1 : principales caractéristiques

Elles sont résumées ci-dessous :

- 4 valeurs possibles du pré diviseur **p** programmable :
1, 2, 4 et 8,
- registre TIMER appelé **TMR1** sur **16 bits** (son octet de poids fort est appelé **TMR1H** et son octet de poids faible **TMR1L**), accessible(s) en lecture ou en écriture,
- fonctionnement de **VI** (valeur initiale sur **16 bits** à charger dans le registre **TMR1**) à **0xFFFF** : timer de type ②,
- le débordement est signalé par le flag **TMR1IF** (**TiMeR 1 Interrupt Flag** situé dans le registre SFR **PIR1**) qui passe à **1** quand le registre **TMR1** passe de **0xFFFF** à **0x0000**,
- un bit **TMR1IE** (comme **TiMeR 1 Interrupt Enable** situé dans le registre SFR **PIE1**) permettra, localement, **d'autoriser** (si **TMR1IE=1**) ou **d'interdire** (si **TMR1IE=0**) la génération d'une interruption lorsque le flag **TMR1IF** passe à **1**,
- le **TIMER1** possède un bit **TMR1ON** (comme **TiMeR 1 ON**) qui permet de le mettre en route lorsqu'on le fait passer à **1**.

Les Timers internes (suite)

Le TIMER1 : synoptique

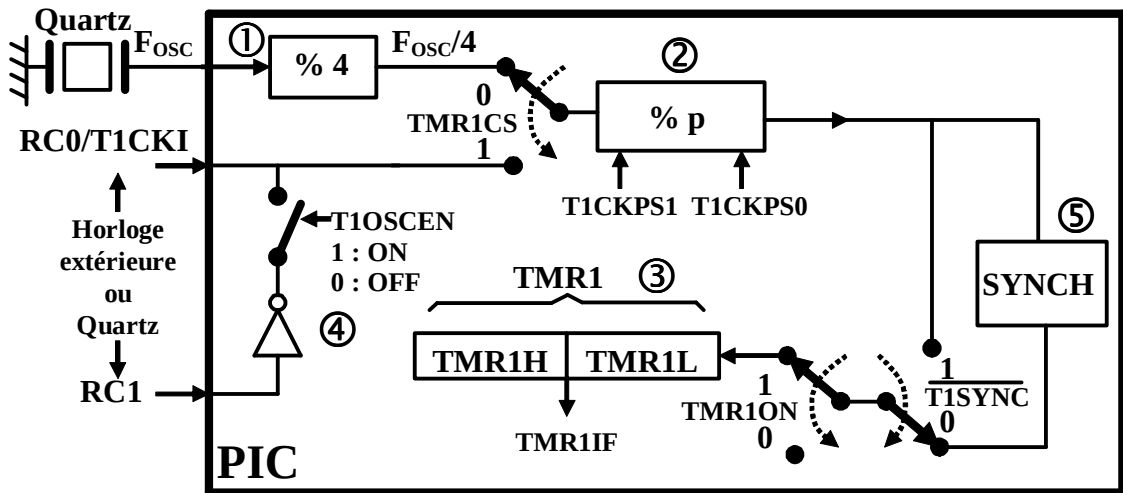


Figure VIII.78. (page 301)

- bloc ① : le pré diviseur **fixe** par $f = 4$.
- bloc ② : le pré diviseur **programmable** p , choisi par les 2 bits **T1CKPS1** et **T1CKPS0**.
- bloc ③ : le registre **TMR1** (**TMR1H** : **TMR1L**), avec son bit d'overflow **TMR1IF** qui passera à **1** quand le contenu de **TMR1** passe de **0xFFFF** à **0x0000**.

Deux multiplexeurs (pilotés par le bit **TMR1CS** et par le bit **T1SYNC**) pour choisir les différents modes de fonctionnement.

- bloc ④ : en mode quartz externe, avec fréquence mieux adaptée pour compter le temps (puissance de 2 comme 32768 ($= 2^{15}$) kHz).
- bloc ⑤ : bloc de synchronisation.
- bit **TMR1ON** pour valider, s'il est à **1**, la fonction **TIMER1**.

Lorsque **TMR1ON** = **0**, le **TIMER1** est inhibé (pas utilisable) ou arrêté (s'il fonctionnait).

Résumé du cours PIC : chapitre VIII.5.
(informations supplémentaires pendant manipulations TP 3)

Le **TIMER1** : registre de contrôle associé

Les bits de configuration sont dans le registre **T1CON** :

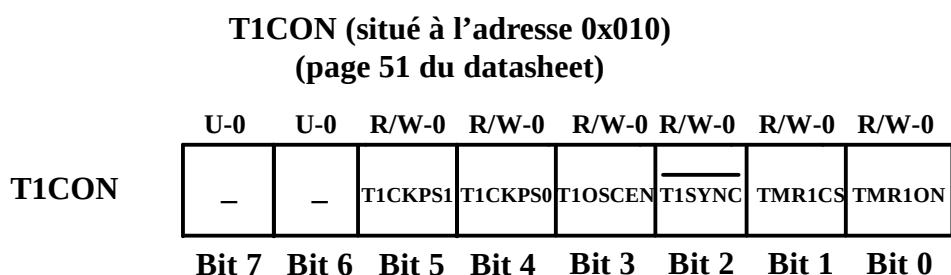


Figure VIII.79. (page 303)

Bit	Valeur	Signification et commentaires
T1CKPS1 :T1CKPS0		Choix de p du pré diviseur programmable
	00	1
	01	2
	10	4
	11	8
TMR1CS		Choix du mode de fonctionnement
	0	Comptage du temps (via le quartz du PIC)
	1	Horloge extérieure (ou évènements externes) RC0 , quartz entre RC0 et RC1
T1OSCEN		Timer 1 OSCillator ENable
	0	Pas de quartz extérieur entre RC0 et RC1
	1	Quartz extérieur entre RC0 et RC1
$\overline{\text{T1SYNC}}$		Timer 1 SYNChronised mode (valable en mode compteur externe (si TMR1CS=1)
	0	Compteur synchrone
	1	Compteur asynchrone
TMR1ON		TiMeR 1 ON
	0	Le TIMER1 est inhibé (ou arrêté)
	1	Le TIMER1 est validé : on peut l'utiliser

Figure VIII.80. (page 303)

Le **TIMER1** : exemple de mise en œuvre

Générer un « Peigne de Dirac » en sortie sur la broche **RD0** du PIC.

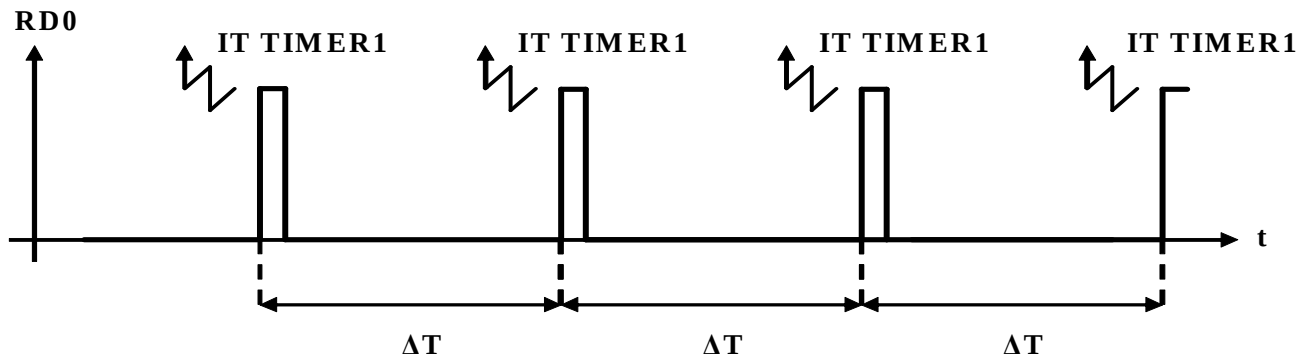


Figure VIII.81. (page 304)

On utilise l'IT du **TIMER1**, ayant pour but de sortir une impulsion de Dirac unitaire sur **RD0** du **PORTD** (programmée en sortie).

Mais dans un autre type d'application, cette IT pourrait venir incrémenter un compteur ou «rafraîchir» et «mettre à jour» des données dans un périphérique ...

On prendra $\Delta T = 100 \mu s$, mesuré par le **TIMER1**.

Avec la formule précédente (2) des timers à valeur initiale **VI** :

$$\Delta T = 100 \mu s = f.p.(2^n - VI).T_{OSC} \quad \text{avec } n = 16 \text{ et } T_{OSC} = 250 \text{ ns}$$

$$100\,000 = 4.p.(65536 - VI).250 \text{ et donc : } p.(65536 - VI) = 100.$$

On prend la plus petite valeur implantable pour p (pour avoir la plus grande précision).

Le tableau de la page précédente nous donne $p_{\min} = 1$.

$65536 - VI = 100$ ce qui nous fournit : **VI = 65436 = 0xFF9C**
(valeur initiale à charger dans le registre compteur **TMR1** du **TIMER1**, soit **0xFF** dans **TMR1H** et **0x9C** dans **TMR1L**).

Résumé du cours PIC : chapitre VIII.5.
(informations supplémentaires pendant manipulations TP 3)

Le TIMER1 : par interruption

```
void interrupt timer1_it (void)
{
    if(TMR1IF && TMR1IE)           // IT sur overflow TIMER1
    {
        TMR1IF = 0;                // acquittement IT TIMER1
        TMR1H=0xFF;                // recharge VIH dans TMR1H
        TMR1L=0x9C;                // recharge VIL dans TMR1L
        RD0 = 1 ;                  // front montant du Dirac
        RD0 = 0 ;                  // front descendant du Dirac
    }
}

void main(void)
{
    // initialisation du PORTD
    TRISD0 = 0;                    // peigne de Dirac sur RD0 en sortie
    // initialisation de T1CON
    T1CKPS1=0;T1CKPS0=0 ; // p = 1
    T1OSCEN = 0 ;                  // ou 1 car pas d'autre quartz extérieur
    T1SYNC = 0 ;                   // ou 1 car déjà synchrone
    TMR1CS = 0 ;                   // utilisation du quartz du PIC
    TMR1ON = 0 ;                   // ne démarre pas encore TIMER1
    // initialisation (sur 16 bits) du TIMER1
    TMR1H=0xFF; TMR1L=0x9C; // VIH et VIL (cf. calcul précédent)
    // initialisation du flag TMR1IF
    TMR1IF=0;                      // efface une éventuelle cause IT antérieure
    // démasquage des interruptions
    TMR1IE=1;                      // démasquage local IT TIMER1
    PEIE=1;                        // démasquage semi global
    GIE=1;                         // démasquage de toutes les IT
    TMR1ON=1;                      // démarrage du TIMER1

    while(1);                      // le main ne fait rien !!!
}
```

Les Timers internes (suite)

Le TIMER2 : principales caractéristiques

Elles sont résumées ci-dessous :

- **4** valeurs possibles du pré diviseur **p programmable** :
1, 4, 16 et 64,
- registre TIMER appelé **TMR2** sur **8 bits** accessible en lecture ou en écriture,
- fonctionnement de **0x00** à **VF** (valeur finale de **8 bits** à charger dans le registre **PR2** (comme **Period Register Timer 2**)) accessible lui aussi en lecture ou en écriture, associé au **TIMER2** : timer de type ①,
- existence d'un flag **TMR2IF** (**TiMeR 2 Interrupt Flag** situé dans le registre SFR **PIR1**) dont la gestion est un peu différente que pour le **TIMER0** ou le **TIMER1** à cause de l'existence d'un **post diviseur** (voir partie suivante),
- un bit **TMR2IE** (comme **TiMeR 2 Interrupt Enable** situé dans le registre SFR **PIE1**) permettra, localement, **d'autoriser** (si **TMR2IE=1**) ou **d'interdire** (si **TMR2IE=0**) la génération d'une interruption lorsque le flag **TMR2IF** passe à **1**,
- le **TIMER2** possède un bit **TMR2ON** (comme **TiMeR 2 ON**) qui permet de le mettre en route lorsqu'on le fait passer à **1**.

Les Timers internes (suite)

Le TIMER2 : particularités

Le **TIMER2** possède trois originalités :

- **entièrement dédié** au comptage du temps via le quartz du PIC,
- il compte non pas de VI à 0xFF (ou 0xFFFF) mais **de 0x00 à VF**,
- il possède, en plus, un **post diviseur** programmable **p'**.

Qu'est-ce c'est ?

Sans ce post-diviseur (ou si $p'=1$ ce qui revient au même), le flag **TMR2IF** passerait à **1** dès que le compteur **TMR2** atteint la valeur **VF** (contenu du registre **PR2**), ou plus exactement un tick d'horloge après.

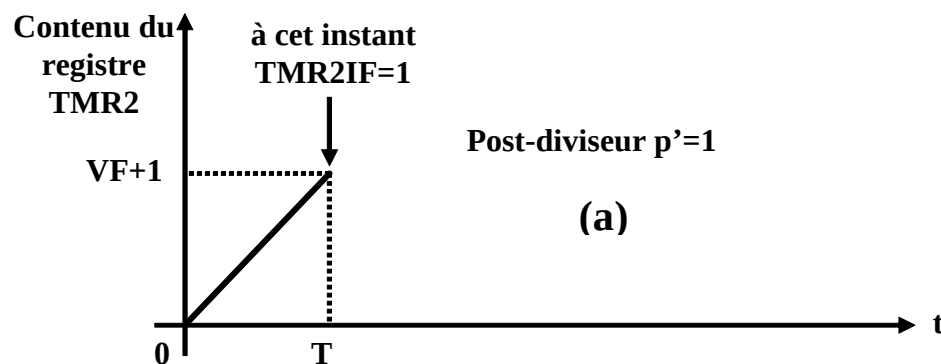


Figure VIII.85.(a) (page 308)

Les Timers internes (suite)

Le TIMER2 : particularités (suite)

Avec ce post diviseur de valeur **p'**, il faut que la coïncidence entre **TMR2** et **PR2** ait lieu **p'** fois pour que le flag **TMR2IF** passe à **1**.

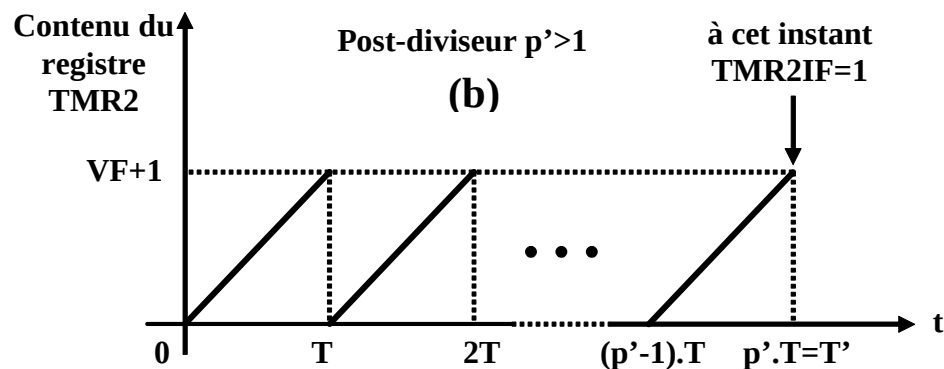


Figure VIII.85. (b) (page 308)

La durée **T'** pour faire passer le flag **TMR2IF** à **1** sera donc :

(1)' $T' = 4.p.(VF+1).p'.T_{osc}$

avec $T_{osc} = \frac{1}{F_{osc}}$ = période du quartz du PIC.

Cette solution permet d'augmenter la durée mesurée par le **TIMER2** d'un facteur **p'**.

Les Timers internes (suite)

Le TIMER2 : synoptique

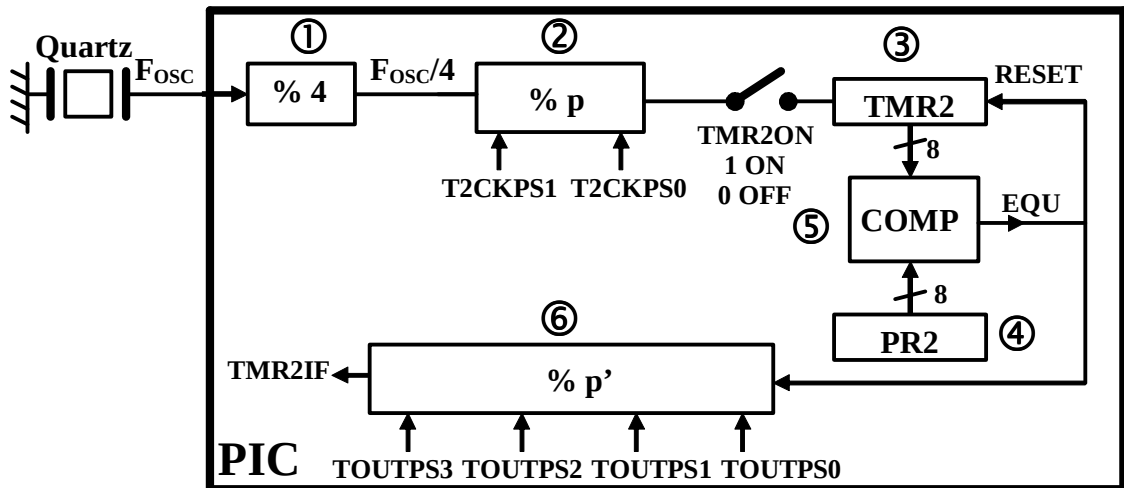


Figure VIII.86. (page 308)

- bloc ① : le pré diviseur **fixe** par **f = 4**.
- bloc ② : le pré diviseur **programmable p**, choisi par les 2 bits **T2CKPS1** et **T2CKPS0**.
- bloc ③ : le registre compteur **TMR2**.
- bit **TMR2ON** pour valider, s'il est à **1**, la fonction **TIMER2**.

Lorsque **TMR2ON = 0**, le **TIMER2** est inhibé (pas utilisable) ou arrêté (s'il fonctionnait).

Les Timers internes (suite)

Le TIMER2 : fonctionnement

Le fonctionnement du TIMER2 est assez simple.

Le contenu de **TMR2** (qui s'incrémente à chaque tick) est comparé au contenu de **PR2** (cf. ④ insensible aux ticks et qui contient la valeur **VF**) grâce à un comparateur câblé **COMP** 8 bits (cf. bloc ⑤).

La sortie **EQU** de **COMP** passera à **1** quand **TMR2** atteindra **VF** (en fait un tick d'horloge après), avec 2 conséquences importantes pour le fonctionnement du **TIMER2** :

- remise à zéro (par le signal **RESET**) de **TMR2** pour redémarrer une nouvelle période de comptage à partir de 0x00, (cela revient à dire que quand **TMR2** atteint **VF**, au lieu de passer à **VF+1** sur le tick d'horloge suivant, il passera à la valeur 0x00...)
- envoi d'une impulsion comptée par le post compteur **p'** (cf. ⑥).

Lorsque **p'** impulsions de ce type auront été comptées par le post-diviseur, le flag **TMR2IF** sera mis à **1** (ce qui peut générer une IT si le flag associé **TMR2IE** du registre SFR **PIE1** est mis à **1**).

La valeur de **p'** est choisi par les 4 bits **TOUTPS3** à **TOUTPS0**.

L'existence d'un post compter **p'** ne change pas la granularité de **TIMER2**, mais permet d'implanter plus facilement des temporisations plus importantes.

Résumé du cours PIC : chapitre VIII.5.
(informations supplémentaires pendant manipulations TP 3)

Le TIMER2 : registre de contrôle associé

Les bits de configuration sont dans le registre **T2CON** :

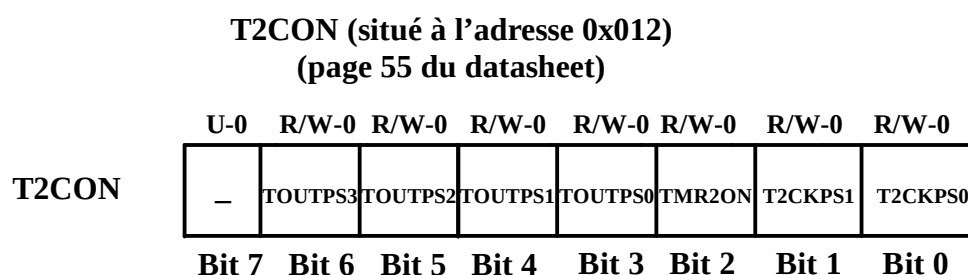


Figure VIII.87. (page 310)

Bit	Valeur	Signification et commentaires
T2CKPS1 :T2CKPS0		Choix de p du pré diviseur programmable
	00	1
	01	4
	10	16
	11	64 (testé, mais datasheet ne le signale pas)
TOUTPS3 :TOUTPS0		Choix p' du post diviseur programmable
	0000	1
	0001	2
	0010	3
	0011	4
	...	...
	1110	15
	1111	16
TMR2ON		TiMeR 2 ON
	0	Le TIMER2 est inhibé (ou arrêté)
	1	Le TIMER2 est validé : on peut l'utiliser

Figure VIII.88. (page 310)

Le TIMER2 : exemple de mise en œuvre

Soit à écrire les deux fonctions suivantes :

void **tempo_1_ms** (void) ; // temporisation fixe de 1 ms

void **tempo_N_ms** (int N) ; // temporisation paramétrable de N ms

La formule (1)' nous donne pour $T' = 1 \text{ ms} = 1\,000\,000 \text{ ns}$:

(1)' $1\,000\,000 = 4 \cdot p \cdot (VF+1) \cdot p' \cdot T_{OSC}$ avec $T_{OSC} = 250 \text{ ns}$ (4MHz)

Soit : $p \cdot p' \cdot (VF+1) = 1\,000$.

On peut prendre : $p = 1$ et $p' = 4$: $VF+1 = 250$ et donc $VF = 249$.

Utilisation du pooling sur **TMR2IF** pour la fonction **tempo_1_ms**.

```
void tempo_1_ms(void)                // temporisation fixe de 1 ms
{
    while(!TMR2IF);                  // on attend que TMR2IF passe à 1
    TMR2IF=0;                         // remise à 0 de TMR2IF
}

void tempo_N_ms(int N)                // temporisation paramétrable de N ms
{
    // initialisation de T2CON
    TOUTPS3 = 0 ; TOUTPS2 = 0 ;
    TOUTPS1 = 1 ; TOUTPS0 = 0 ; // p' = 4
    TMR2ON = 1 ;                     // TIMER2 mis en route
    T2CKPS1 = 0 ; T2CKPS0 = 0 ; // p = 1
    PR2=249;                          // pour avoir 1 ms pour tempo_1_ms
                                      // voir le calcul précédent
    TMR2IF=0;                         // efface le flag TMR2IF
    while(N--) tempo_1_ms();          // appelle N fois tempo_1_ms
}
```

Figure VIII.89. (page 311)

Les modules CCP1 et CCP2

Défaut des timers internes

Les 3 timers que l'on vient de voir présentent un défaut essentiel :

pour des **faibles valeurs de la temporisation**, les précisions obtenues sont malgré tout **médiocres**, ceci étant essentiellement dû au fait que les temps d'exécution des instructions entraînaient des **délais de latence** (et des **durées de réaction** du PIC) **non négligeables** devant ces faibles valeurs des temporisations.

En fait, ce défaut existe aussi pour des temporisations plus importantes, mais dans ce cas là, en relatif et en absolu, il peut être négligé.

Comment résoudre cette difficulté ?

Puisque le problème, comme on vient de le dire vient des instructions et donc plutôt du **software**, on va le résoudre en « **le sous-traitant** » au **hardware**.

En fait, on va rajouter quelques registres supplémentaires à la fonction timer, car les temps de réponse de registre à registre (et donc sans passer par la CPU) seront forcément plus courts.

C'est ce qui est fait dans les modules **CCP1** et **CCP2** que l'on va présenter maintenant.

Les modules CCP1 et CCP2 (suite)

Introduction

Trois types de fonctions sont gérés par les modules **CCP1** et **CCP2** (comme **C**apture/**C**ompare/**P**ulse width modulation **1** et **2**) :

- la fonction **CAPTURE** : elle fonctionne en **entrée**,
- la fonction **COMPARE** : elle fonctionne en **sortie**,
- la fonction **PWM** : elle fonctionne également en **sortie**.

La fonction (Input) Capture : généralités

Pouvoir **dater** un évènement extérieur, donc connaître la valeur du timer (le contenu de **TIMER**) quand cet évènement se produit.

Pour cela, il faut envoyer sur une des pattes d'**entrée** du PIC prévue à cet effet (appelée ici **CCP**) un signal $E(t)$ qui change d'état à l'instant t_c où cet évènement extérieur se produit.

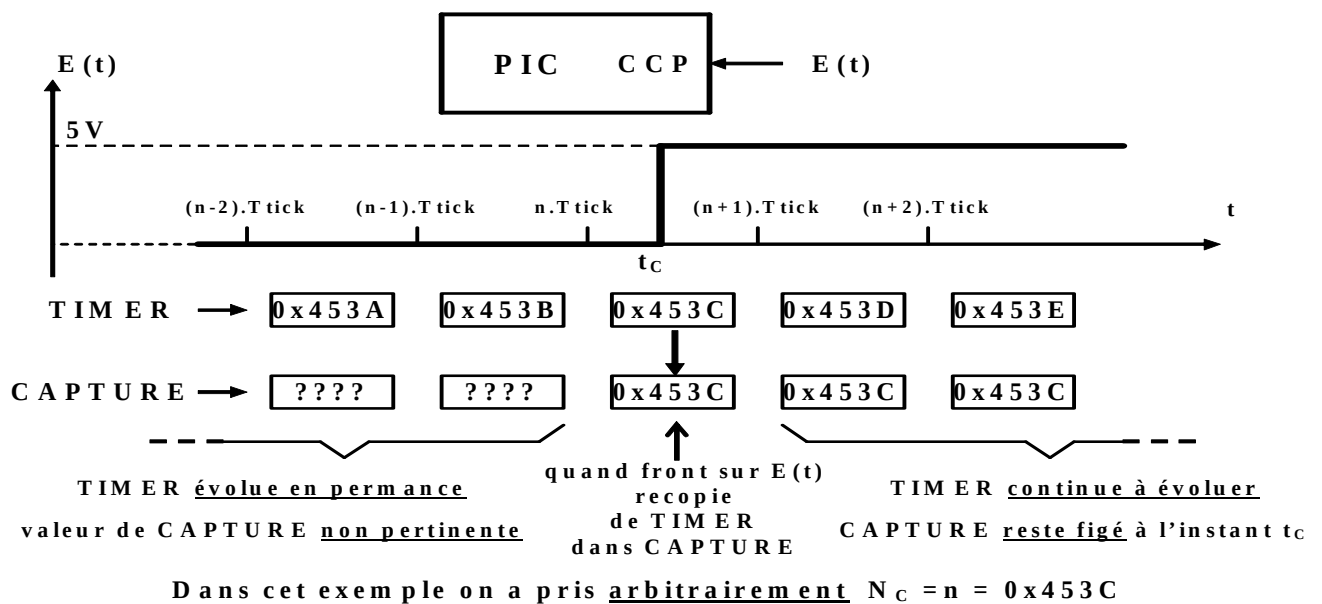


Figure VIII.93. (page 317)

La fonction (**INPUT**) **CAPTURE** est prévue pour que le PIC recopie **automatiquement** le contenu courant du registre **TIMER** dans un registre qu'on appellera ici le registre **CAPTURE** à l'instant t_c du changement d'état de $E(t)$.

(avec une précision inférieure à $\pm \frac{1}{2}$ tick d'horloge du timer).

Ensuite le registre **TIMER** continue toujours à augmenter à chaque tick, mais le registre **CAPTURE** reste figé, un peu comme si on y avait « gelé » la valeur du registre **TIMER** à l'instant t_c . de la « **capture** » (ce qui était bien le but recherché !!!).

La fonction (Input) Capture : généralités

La taille du registre **CAPTURE** est bien évidemment la même que celle du registre **TIMER**.

Une **entrée** de capture **CCP** n'est pas seulement **sensible** que sur un **front montant** de $E(t)$, mais peut aussi l'être aussi sur un **front descendant**, voire sur les **2 types de fronts**. Et ceci, en modifiant quelques bits dans un registre de contrôle adéquat.

Ce sera au programme de venir lire le contenu du registre **CAPTURE**.

Mais attention, si un autre évènement se produit (comme un second front montant sur $E(t)$), une nouvelle capture dans le registre **CAPTURE** aura lieu et écrasera la valeur précédente.

Conclusion :

Il ne faut pas trop attendre pour venir lire le contenu du registre **CAPTURE**, car sinon on risque de faire une lecture erronée.

Grâce à la technique des IT, on sait que c'est possible, en venant lire le registre **CAPTURE** sous IT dès que l'évènement se produit.

Le contenu du registre **CAPTURE** (comme $N_C = 0x453C$ dans l'exemple) ne donne pas le temps t_C directement.

Il faut pour cela prendre en compte la période T_{tick} des ticks d'horloge incrémentant le **TIMER** et faire :

$$t_C = N_C \cdot T_{\text{tick}}$$

La fonction (Input) Capture : généralités

Dans cet exemple, on vient de dater un évènement **en absolu**, c'est-à-dire par rapport à un instant initial choisi par convention.

Ce n'est pas la seule possibilité d'utiliser la fonction de capture.

Une variante (très utilisée) de cette fonction permet de **mesurer un intervalle de temps**, comme par exemple la largeur d'une impulsion

Le principe, donné ici pour une impulsion positive, est très simple :

- on initialise la fonction **CAPTURE** pour que la broche **d'entrée CCP** soit sensible aux fronts **montants**,
- quand le premier front montant de l'impulsion arrive à $t = t_{C1}$, cet instant t_{C1} (en nombre de ticks du **TIMER**) est **automatiquement** figé dans le registre **CAPTURE**. On vient alors, par programme, copier ce registre dans une première variable N_{C1} ,
- puis on modifie la nature du front actif sur **CCP** (en choisissant la prochaine capture sur un front actif **descendant**),
- quand le prochain front descendant de l'impulsion arrive à $t = t_{C2}$, cet instant t_{C2} (en nombre de ticks du **TIMER**) est **automatiquement** figé dans le registre **CAPTURE**. On vient, par programme, copier ce registre dans une seconde variable N_{C2} ,
- il ne reste plus qu'à faire la « **différence** » entre ces deux variables N_{C2} et N_{C1} (avec quelques précautions quand-même !!! pour éviter des pièges, sources d'erreurs de mesure),
- le résultat obtenu est la **largeur** N de l'impulsion, exprimée en nombre de ticks d'horloge.

La fonction (Input) Capture : synoptique

Sur le PIC, la fonction **de capture** utilise le **TIMER1** comme **référence temporelle**.

Cette fonction peut-être réalisée au choix par **l'un ou l'autre** des deux modules **CCPi**.

On peut même implanter 2 fonctions de capture **indépendantes**,
une sur **CCP1** et l'autre sur **CCP2** mais,
elles partageront la même base de temps fournie par **TIMER1**.

Le registre **CAPTURE** :

- pour **CCP1** s'appelle **CCPR1**,
 - pour **CCP2** s'appelle **CCPR2**.
- (comme Capture Compare Pwm Register 1 ou 2).

Ils sont tous les 2 de la même taille que **TMR1**, c'est-à-dire 16 bits.

L'entrée de capture pour **CCP1** est appelée **CCP1** (broche **RC2**).

L'entrée de capture pour **CCP2** est appelée **CCP2** (broche **RC1**).

Si la fonction capture est utilisée,
ces broches **doivent être programmées** en entrée via **TRISC**

La fonction (Input) Capture : synoptique

Le synoptique simplifié de l'une ou l'autre de la fonction **capture** (dans **CCP1** ou **CCP2**) est le même.

Il est donné ci-dessous pour le module **CCP1**.

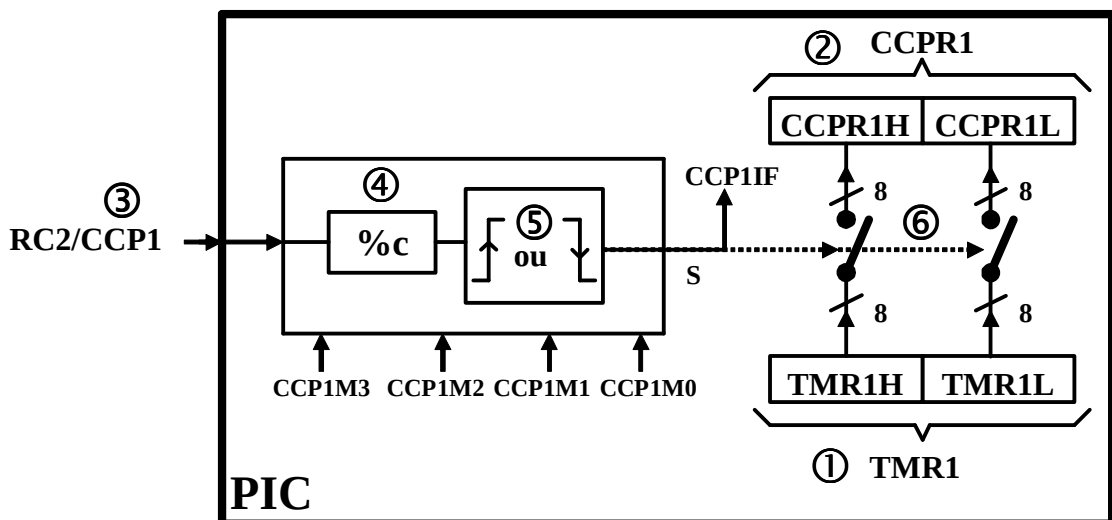


Figure VIII.94. (page 319)

- bloc ① : le registre **TIMER**, ici réalisé par le registre **TMR1**.
(les ticks incrémentant **TMR1** n'ont pas été représentés ici).
- bloc ② : le registre **CAPTURE** appelé **CCPR1** (ou **CCPR2**).
- ③ : l'**entrée de capture** de l'évènement extérieur : **CCP1/RC2** (ou **CCP2/RC1**).
- bloc ④ : ce signal de capture passe dans un pré diviseur (à ne pas confondre avec le pré diviseur **p** du **TIMER1** !!!) de valeur **paramétrable c** déterminée par les 4 bits **CCPiM3** à **CCPiM0**. Cela signifie que le front actif de capture peut être généré tous les **c** évènements.

La fonction (Input) Capture : synoptique

- bloc ⑤ : pour choisir le **front actif** de capture (soit montant, soit descendant) en fonction des 4 bits **CCPiM3** à **CCPiM0**.

Lorsque la sortie **S** du bloc ⑤ passe à **1** (c'est-à-dire lorsque le front actif a été détecté), il y aura deux conséquences importantes (et là on est au « cœur » de la fonction **CAPTURE**) :

- le contenu de **TMR1** est copié dans **CCPR1**, symbolisé par les 16 interrupteurs de la partie ⑥ qui vont se fermer, le temps du transfert, lorsque **S** passe à **1** (pour **CCP2**, **TMR1** sera copié dans **CCPR2**),
- le flag **CCP1IF** (ou **CCP2IF**) passe à **1**, pour signaler la capture.

Pour gérer ce flag, la technique du polling est toujours possible, mais il est préférable, surtout dans les cas où les contraintes temporelles sont « serrées », d'utiliser les interruptions en validant le bit correspondant **CCP1IE** (situé dans le registre SFR **PIE1**) ou **CCP2IE** (situé dans **PIE2**).

Remarque

On peut aussi **simuler** l'arrivée d'un évènement extérieur et donc provoquer une capture **par programme** en écrivant sur le bit **RC2** (ou **RC1**) du **PORTC**, à condition d'avoir préalablement mis ces lignes **en sortie** via le registre **TRISC**.

La fonction (Input) Capture

Registres de contrôle associés

Les bits de configuration sont dans le registre **CCP1CON** pour **CCP1** et **CCP2CON** pour **CCP2**, de **même** format :

CCP1CON/CCP2CON (situés aux adresses respectives 0x017 et 0x01D)
(page 58 du datasheet)

	U-0	U-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
CCP1CON CCP2CON	–	–	CCPiX	CCPiY	CCPiM3	CCPiM2	CCPiM1	CCPiM0
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0

Figure VIII.95. (page 321)

Voici les combinaisons concernant la fonction de **capture**.
(pour les autres fonctions voir plus loin)

Bit	Valeur	Signification et commentaires
CCPiX :CCPiY		Voir fonction PWM
CCPiM3 :CCPiM0		Pour choisir le mode de fonctionnement
	0000	Modules CCPi non actif (mise à zéro des sorties CCPi)
	0100	Mode de capture à chaque front descendant sur CCPi (c=1)
	0101	Mode de capture à chaque front montant sur CCPi (c=1)
	0110	Mode de capture tous les 4 fronts montant sur CCPi (c=4)
	0111	Mode de capture tous les 16 fronts montant sur CCPi (c=16)
	1xxx	Autres modes de fonctionnement

Figure VIII.96. (page 321)

La fonction (Output) Compare : généralités

C'est un peu la « fonction inverse » de la fonction **CAPTURE**.

Générer sur une des broches du PIC programmée **en sortie** un front actif à un instant t_c voulu par le programme.

Pour cela il faut préalablement écrire cette valeur de t_c (exprimée en nombre de ticks) dans un registre prévu à cet effet et qu'on appellera ici **COMPARE**.

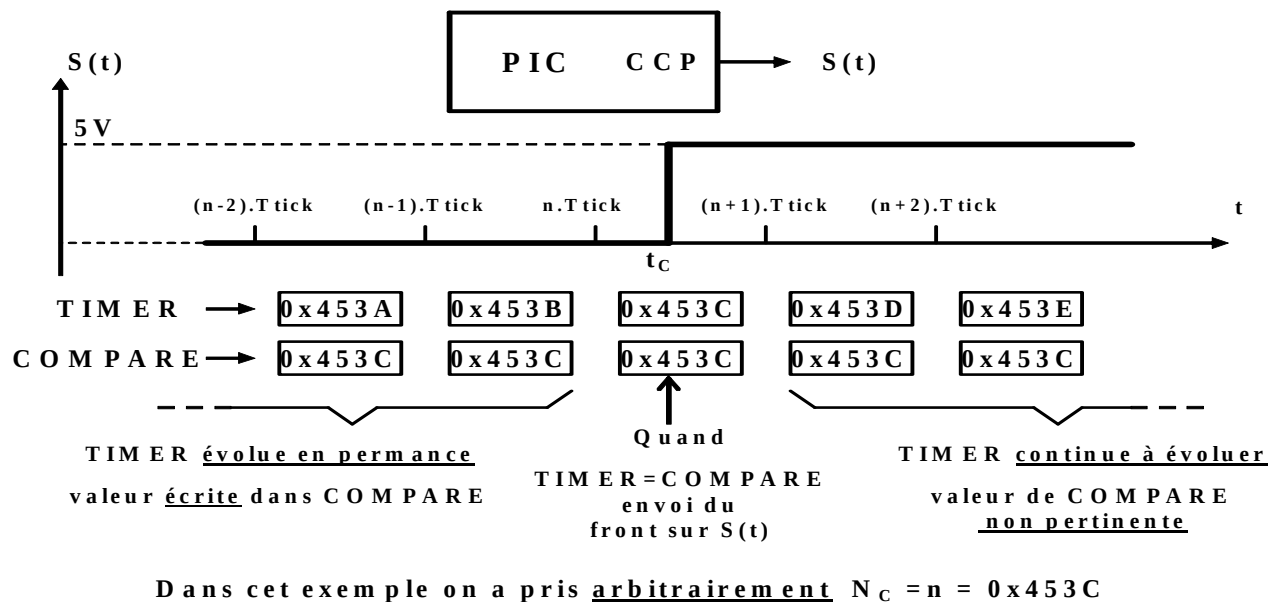


Figure VIII.100. (page 326)

La fonction (**OUTPUT**) **COMPARE** est prévue pour comparer en **permanence** (avec un « **comparateur câblé dans le hardware** ») le contenu de **TIMER** avec celui de **COMPARE**. Et quand il y a égalité, elle envoie automatiquement un front $S(t)$ sur une patte de sortie appelée ici **CCP**.

La taille du registre **COMPARE** est bien sûr la même que celle du registre **TIMER**.

La fonction (Output) Compare : généralités

Une **sortie** de capture **CCP** n'est pas seulement capable de générer un **front montant** sur S(t), mais peut aussi générer un **front descendant**, voire tout simplement **changer d'état (toggle)** à chaque comparaison valide. Et ceci, en modifiant quelques bits dans un registre de contrôle adéquat.

Remarque

Si on y regarde de plus près, cette fonction pouvait déjà être réalisée uniquement avec les timers internes du PIC. Mais, sous contrôle logiciel, avec tout ce que cela impliquait au niveau des temps de réponse (notamment pour les très faibles temporisations) comme on l'a déjà souligné. Avec la fonction **COMPARE**, tout est **automatisé** dans le **hardware** en intégrant un **comparateur câblé** et une logique de commande de la broche **CCP** également câblée. Et donc des temps de réponse réduits à leur plus petite valeur...

Ce n'est pas le temps t_C qu'il faut écrire dans **COMPARE**, mais une valeur N_C (par exemple $N_C = 0x453C$ dans l'exemple), donnée par :

$$N_C = \frac{t_C}{T_{\text{tick}}}$$

où T_{tick} est la période des ticks incrémentant le registre **TIMER**.

La fonction (Output) Compare : généralités

Dans cet exemple, on vient de générer un signal à un instant **absolu**, c'est-à-dire par rapport à un instant initial choisi par convention.

Ce n'est pas la seule possibilité d'utiliser la fonction comparaison.

Une variante (très utilisée) de la fonction **COMPARE** permet de fabriquer une impulsion de largeur T souhaitée.

Le principe, pour générer une impulsion positive, est très simple :

- on initialise la fonction **COMPARE** pour que la broche de **sortie CCP** génère un front **montant** à chaque comparaison valide,
- on initialise **COMPARE** avec la valeur N_{C1} (exprimée en nombre de ticks du **TIMER**) correspondant à l'instant t_{C1} souhaité pour le début de l'impulsion,
- à $t = t_{C1}$, au moment de la première comparaison valide, la sortie **CCP** génère **automatiquement** le front montant de l'impulsion,
- on vient alors, par programme, rajouter au registre **COMPARE**, la valeur N (en nombre de ticks du **TIMER**) correspondant à T,
- puis on modifie la nature du front actif qui sera généré sur la **sortie CCP** à la prochaine comparaison valide (en choisissant un front actif **descendant**),
- à $t = t_{C2}$, au moment de la prochaine comparaison valide, la sortie **CCP** génère **automatiquement** le front descendant de l'impulsion. Et c'est fini !!!

Attention, si N est supérieure à la durée de débordement du **TIMER** (longues impulsions), il faut prendre en compte les débordements du registre **TIMER** pour avoir la bonne largeur d'impulsions...

La fonction (Output) Compare : synoptique

Sur le PIC, la fonction **de comparaison** utilise aussi le **TIMER1** comme **référence temporelle**.

Cette fonction peut-être réalisée au choix par **l'un ou l'autre** des deux modules **CCPi**.

On peut aussi implanter 2 fonctions de comparaison **indépendantes**, une sur **CCP1** et l'autre sur **CCP2** mais, elles partageront la même base de temps fournie par **TIMER1**.

Remarque

Par contre, si les fonctions capture sont identiques sur **CCP1** et **CCP2**, quelques différences (minimes) existent entre les fonctions comparaison de **CCP1** et de **CCP2**, pour le mode **SPECIAL EVENT TRIGGER** (terme utilisé par **MICROCHIP**) qu'on peut traduire par mode de **SYNCHRONISATION (HARDWARE) SUR EVENEMENT SPECIAL**.

(pour plus de détail sur ce mode voir le polycopié de cours)

Le registre **COMPARE** pour **CCP1** est **CCPR1** et celui de **CCP2** est **CCPR2**. (les mêmes que pour la fonction **CAPTURE** !!!).

La sortie de comparaison valide est appelée **CCP1** (broche **RC2**) pour **CCP1** (la même que pour la fonction de capture !!!).

La sortie de comparaison valide est appelée **CCP2** (broche **RC1**) pour **CCP2** (la même que pour la fonction de capture !!!).

Si la fonction de comparaison est utilisée, ces broches **doivent être programmées** en **sortie** via le registre **TRISC**

La fonction (Output) Compare : synoptique

Mais comme à l'intérieur d'un même module **CCPi** les fonctions de capture et de comparaison utilisent des ressources hardware communes, elles sont donc **exclusives** l'une de l'autre.

Par exemple, si le module **CCP1** fonctionne dans le mode de capture, il ne peut pas **au même moment** fonctionner dans le mode de comparaison. Et réciproquement. Idem pour **CCP2**.

Par contre rien n'interdit :

- de faire aux mêmes moments une capture avec **CCP1** et une comparaison avec **CCP2** (et réciproquement),
- de faire à un certain moment une capture avec **CCP1** et à un autre moment différent une comparaison avec ce même module **CCP1**. Idem pour **CCP2**.

La fonction (Output) Compare : synoptique

Le synoptique simplifié de l'une ou l'autre de la fonction de **comparaison** (dans **CCP1** ou **CCP2**) est **quasiment** le même.
(sauf pour le mode **SPECIAL EVENT TRIGGER**).

Il est donné ci-dessous pour le module **CCP1**.

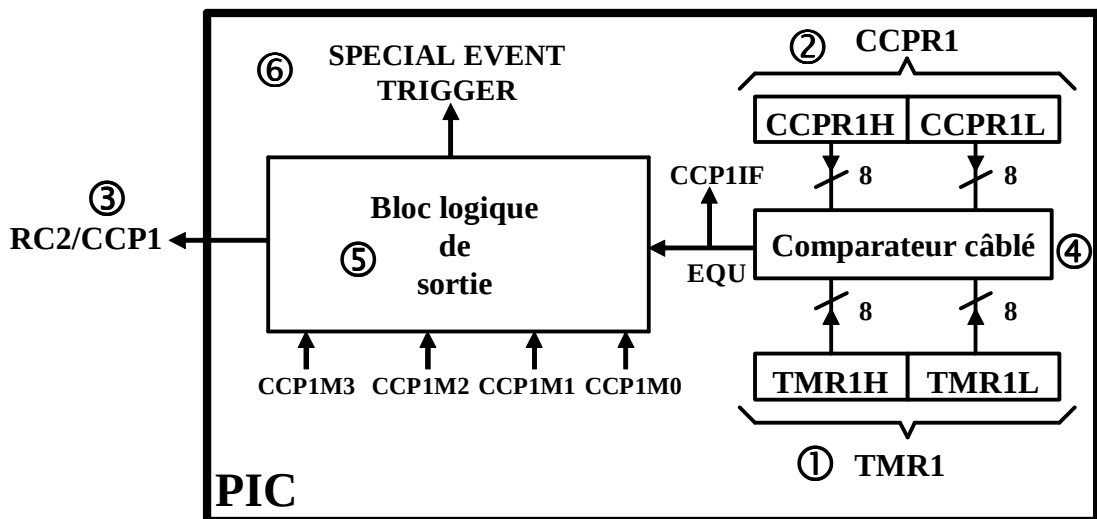


Figure VIII.101. (page 329)

- bloc ① : le registre **TIMER**, ici réalisé par le registre **TMR1**.
(les ticks incrémentant **TMR1** n'ont pas été représentés ici)
- bloc ② : le registre **COMPARE** appelé **CCPR1** (ou **CCPR2**),
- ③ : la **sortie de comparaison valide** : **CCP1/RC2** (ou **CCP2/RC1**).

La fonction (Output) Compare : synoptique

Le **comparateur câblé** 16 bits (cf. bloc ④) compare en permanence et en **temps réel** le contenu des registres **CCPR1** (ou **CCPR2**) et **TMR1**.

Sa sortie **EQU** passera à **1**, dès qu'il y aura égalité.

Ce qui provoquera aussi la mise à **1** du flag **CCP1IF** (ou **CCP2IF**).

Pour gérer ce flag, la technique du polling est toujours possible, mais il est préférable, surtout dans les cas où les contraintes temporelles sont « serrées », d'utiliser les interruptions en validant le bit correspondant **CCP1IE** (situé dans le registre SFR **PIE1**) ou **CCP2IE** (situé dans **PIE2**).

Enfin, le bloc logique de sortie (cf. bloc ⑤) permettra de choisir le front actif qui sera généré sur la patte de sortie **CCP1** (ou **CCP2**), en fonction des 4 bits **CCPiM3** à **CCPiM0**.

La fonction (Output) Compare

Registres de contrôle associés

Les bits de configuration sont dans le registre **CCP1CON** pour **CCP1** et **CCP2CON** pour **CCP2** de **même** format et déjà **vus** pour la fonction de capture :

CCP1CON/CCP2CON (situés aux adresses respectives 0x017 et 0x01D)
(page 58 du datasheet)

	U-0	U-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
CCP1CON	–	–	CCPiX	CCPiY	CCPiM3	CCPiM2	CCPiM1	CCPiM0
CCP2CON								
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0

Figure VIII.102. (page 331)

Voici les combinaisons concernant la fonction **compare**.

Bit	Valeur	Signification et commentaires
CCPiX :CCPiY		Voir fonction PWM
CCPiM3 :CCPiM0		Pour choisir le mode de fonctionnement
	0000	Modules CCPi non actif (mise à zéro des sorties CCPi)
	1000	Génération front montant sur CCPi à chaque comparaison valide (si avant broche CCPi au niveau bas !!!)
	1001	Génération front descendant sur CCPi à chaque comparaison valide (si avant broche CCPi au niveau haut !!!)
	1010	Ne fait que mettre le flag CCPiIF à 1 à chaque comparaison valide (n'agit pas sur la sortie CCPi) Mode dit d'interruption logicielle
	1011	Mode SPECIAL EVENT TRIGGER (n'agit pas sur la sortie CCPi)
	x1xx	Autres modes de fonctionnement

Figure VIII.103. (page 331)

La fonction PWM : généralités

Tout d'abord, qu'entend-on par signal **PWM** ?

C'est un signal **périodique** $S(t)$ comme ci-dessous (où on a de plus supposé un niveau haut compatible TTL).

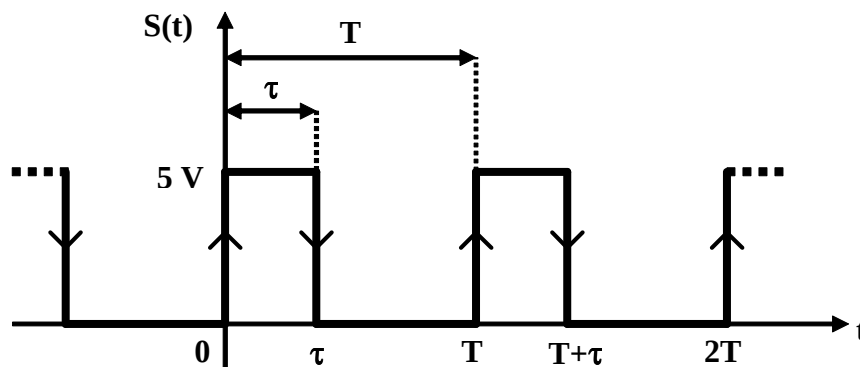


Figure VIII.106. (page 335)

Ce signal possède deux paramètres importants :

- sa **période** T (ou sa fréquence $f = \frac{1}{T}$),
- sa **durée** τ à l'**état haut** (avec bien sûr $0 \leq \tau \leq T$).

On peut aussi raisonner avec le **rapport cyclique** α défini par :

$$\alpha = \frac{\tau}{T} \quad (\text{exprimé souvent en \%})$$

Il est en effet facile de retrouver τ lorsqu'on connaît T et α .

Signal très utilisé pour la **variation de vitesse** des moteurs à courant continu, dans les **alimentations à découpage**, dans les **amplificateurs audio** dits de **classe D**, dans certains procédés de **modulation**.

Il est souvent aussi appelé signal **MLI** (comme **M**odulation de **L**argeur d'**I**mpulsions).

La fonction PWM : principe

Ce signal $S(t)$ sera sorti sur une des broches appelée ici **CCP**.

Comme pour la capture et la comparaison, la fonction **PWM** s'appuie sur un **TIMER** pour réaliser sa référence temporelle.

Le principe de génération est assez simple :

- un premier registre **PERIODE** contient la valeur de **T** (exprimée toujours en nombre de ticks d'horloge du **TIMER**),
- un second registre **TO** contient la valeur de τ (exprimée aussi en nombre de ticks d'horloge du **TIMER**),

Quand **TIMER** = **PERIODE** (au début de chaque période de $S(t)$: instants de la forme $k.T$ avec k entier), les trois actions suivantes sont **automatiquement** (et simultanément) effectuées :

- remise à zéro du registre **TIMER**,
- mise à l'état **haut** de la sortie **CCP** (front **montant**),
- chargement de **TO** avec la valeur de τ (en nombre de ticks)

Quand **TIMER** = **TO** (aux instants de la forme $k.T + \tau$ avec k entier), l'action suivante est **automatiquement** effectuée :

- mise à l'état **bas** de la sortie **CCP** (front **descendant**).

Et ainsi de suite...

Remarques

Si la valeur dans **TO** est **supérieure** à celle dans **PERIODE**, alors la sortie **CCP** restera en permanence au niveau **haut** ($\alpha = 100\%$).

Si la valeur dans **TO** est **nulle**, alors la sortie **CCP** restera en permanence au niveau **bas** (rapport cyclique $\alpha = 0\%$).

La fonction PWM : synoptique

La fonction **PWM** utilise le **TIMER2** comme référence temporelle.

Cette fonction peut-être réalisée au choix par **l'un ou l'autre** des deux modules **CCPi**.

On peut aussi implanter 2 fonctions PWM **indépendantes**,
une sur **CCP1** et l'autre sur **CCP2** mais,
elles partageront la même base de temps fournie par **TIMER2**.

Les deux signaux **PWM** ainsi générés :

- auront alors la **même période**,
car le registre **PERIODE** pour **CCP1** (et aussi pour **CCP2**) est le même : c'est le registre **PR2** du **TIMER2**,
- mais pourront avoir **des rapports cycliques différents**.

De même on peut utiliser la fonction **PWM** avec le module **CCP1** et soit la fonction **CAPTURE**, soit la fonction **COMPARE** sur **CCP2**, et ceci sans aucune limitation.

Et réciproquement.

La fonction PWM : synoptique

La durée τ du signal **PWM** est codée sur 10 bits.

Comment cela ?

Le registre **TO** pour **CCP1** (**CCP2**) est le registre **CCPR1L** (**CCPR2L**) qui contient les **8 bits de poids forts** de la durée τ (exprimée en nombre de ticks du **TIMER2**).

Ses 2 bits de poids faibles sont les deux bits **CCP1X** et **CCP1Y** (**CCP2X** et **CCP2Y**) du registre **CCP1CON** (**CCP2CON**).

(ce qui confirme bien que si la fonction **PWM** est utilisée avec **CCP1** et **CCP2**, les deux signaux **PWM** générés auront bien **deux rapports cycliques différents**).

La sortie PWM est appelée **CCP1** (broche **RC2**) pour **CCP1** (toujours la même que pour les fonctions capture et compare !!!).

La sortie PWM est appelée **CCP2** (broche **RC1**) pour **CCP2** (toujours la même que pour les fonctions capture et compare !!!).

Si la fonction **PWM** est utilisée, ces broches **doivent être programmées** en **sortie** via le registre **TRISC**

La fonction PWM : synoptique

Le synoptique simplifié de l'une ou l'autre de la fonction **PWM** (dans **CCP1** ou **CCP2**) est le même.

Il est donné ci-dessous pour le module **CCP1**.

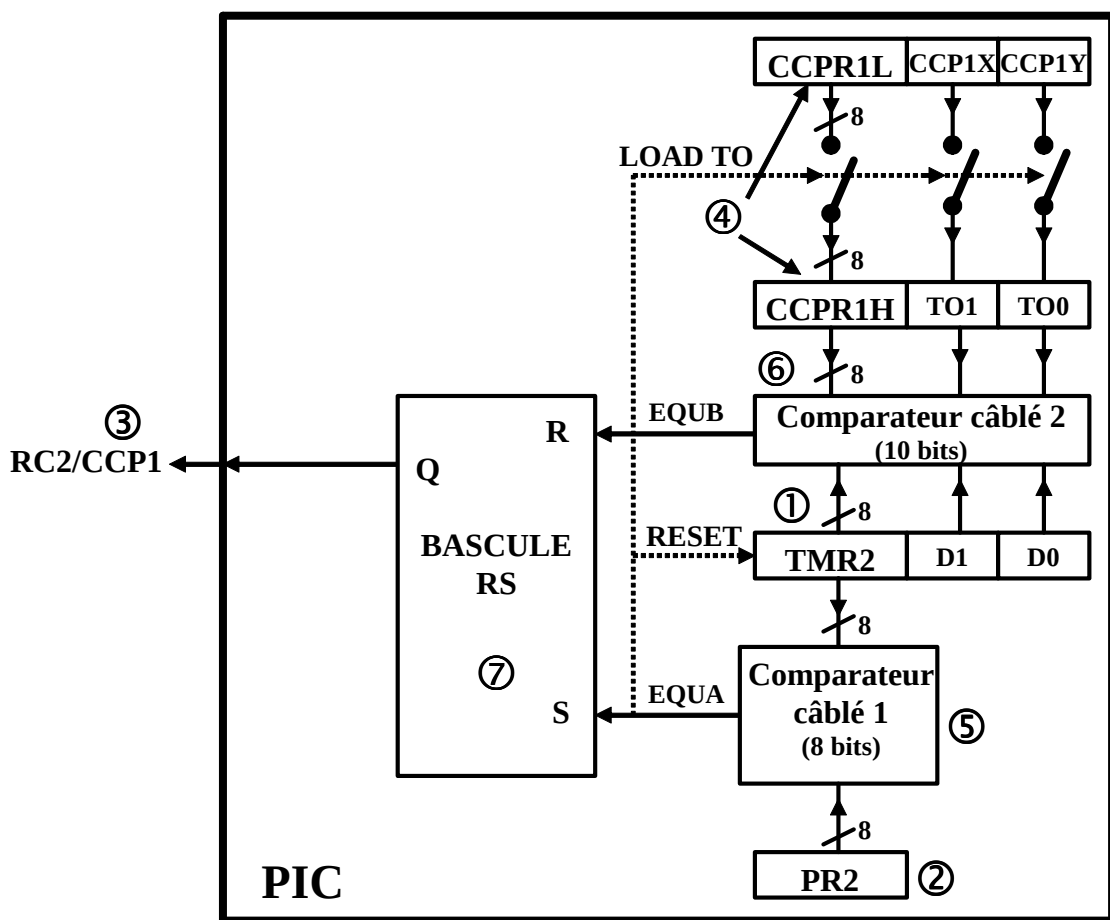


Figure VIII.107. (page 338)

- bloc ① : le registre **TIMER**, ici réalisé par le registre **TMR2**.
(les ticks incrémentant **TMR2** n'ont pas été représentés ici)
- bloc ② : le registre **PERIODE** est le registre **PR2** du **TIMER2**
(pour **CCP1** et **CCP2**)
- ③ : la sortie **CCP** véhiculant le signal **PWM** est :
CCP1/RC2 pour **CCP1** (ou **CCP2/RC1** pour **CCP2**)

La fonction PWM : synoptique

- bloc ④ : **TO** réalisé par les 2 registres **CCPR1L** et **CCPR1H** :
CCPR1L est en lecture/écriture,
CCPR1H n'est qu'en lecture dans le mode **PWM**.

En fait, **CCPR1L** sert de tampon (buffer) à **CCPR1H**.

Les 2 bits de poids faibles de τ sont les bits **CCP1X** et **CCP1Y** du registre **CCP1CON** (voir figure VIII.109. page 340).

Le **comparateur câblé** 1 8 bits (cf. bloc ⑤) compare en permanence et en **temps réel** le contenu des registres **TMR2** et **PR2**.

Sa sortie **EQUA** passera à **1**, dès qu'il y aura égalité.
(instants de la forme **k.T**).

Ce qui provoquera les 3 actions suivantes :

- **remise à 0** du registre **TMR2** : via le signal **RESET**,
- **mise à 1** de l'entrée **S (SET)** de la bascule **RS** de sortie (bloc ⑦),
ce qui provoquera la mise à **1** de sa sortie **Q**, et donc un front **montant** sur la broche de sortie **CCP1 (CCP2)**,
- **chargement** des 8 bits de **CCPR1L** dans **CCPR1H** (ainsi que des 2 bits **CCP1X** et **CCP1Y** dans les 2 cases **TO1** et **TO0**) par le signal **LOAD TO**. La durée τ est bien alors codée sur 10 bits.
Le fait de « double-bufferiser » la durée τ permet de la modifier par programme à n'importe quel moment, mais la valeur qui sera réellement prise en compte sera celle qui passera de **CCPR1L** à **CCPR1H** à cet instant.

Cela permet d'éviter ce qu'on appelle en anglais des « **glitch** » (c'est-à-dire des légers « tremblements ») sur le front descendant du signal de sortie **PWM**.

La fonction PWM : synoptique

Formules pour calculer les valeurs des paramètres caractéristiques :

T et τ du signal PWM :

(1) : $T = 4.T_{osc}.p.(PR2+1)$ T_{osc} = période du quartz du PIC
 p =valeur du prédiviseur **TIMER2**
(1, 4, 16 ou 64)

(2) : $\tau = T_{osc}.p.CCPR1$ T_{osc} = période du quartz du PIC
 p =valeur du prédiviseur **TIMER2**
(1, 4, 16 ou 64)
CCPR1 représente les 10 bits:
CCPR1L : CCP1X : CCP1Y.

Le **comparateur câblé** 2 10 bits (cf. bloc ⑥) compare en permanence et en **temps réel** le contenu de **TMR2** (auquel ont été concaténés, en guise de poids faible, les 2 bits **D1** et **D0** provenant du pré diviseur p (si $p > 1$) ou de l'horloge interne **Q** si $p = 1$) et de **CCPR1H** (auquel ont été concaténés, en guise de poids faible, les deux bits **TO1** et **TO0** provenant des bits **CCP1X** et **CCP1Y** du registre **CCP1CON**).

Sa sortie **EQUB** passera à **1**, dès qu'il y aura égalité.
(à tous les instants de la forme **$k.T + \tau$**).

Ce qui provoquera l'action suivante :

- **mise à 1** de l'entrée **R (RESET)** de la bascule **RS** de sortie (cf. bloc ⑦), provoquant ainsi la mise à **0** de sa sortie **Q**, et donc un front **descendant** sur la broche de sortie **CCP1 (CCP2)**.

La fonction PWM

Registres de contrôle associés

Les bits de configuration sont dans le registre **CCP1CON** pour **CCP1** et **CCP2CON** pour **CCP2** de **même** format et déjà **vus** pour la fonction de capture et de comparaison :

CCP1CON/CCP2CON (situés aux adresses respectives 0x017 et 0x01D)
(page 58 du datasheet)

	U-0	U-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
CCP1CON			CCPiX	CCPiY	CCPiM3	CCPiM2	CCPiM1	CCPiM0
CCP2CON	–	–						
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0

Figure VIII.108. (page 340)

Voici les combinaisons concernant la fonction **PWM**.

Bit	Valeur	Signification et commentaires
CCPiX :CCPiY	xx	Les deux bits de poids faibles de la durée τ à l'état haut du signal PWM . (les huit bits de poids forts de τ étant contenus dans CCPRiL)
CCPiM3 :CCPiM0		Pour choisir le mode de fonctionnement
	0000	Modules CCPi non actif (mise à zéro des sorties CCPi)
	11xx	Mode PWM
	x̄xxx	Autres modes de fonctionnement

Figure VIII.109. (page 340)

ELEMENTS SUR LE LCD IMPLANTÉ SUR LA CARTE PICDEM 2 PLUS

Jusqu'ici les seuls **périphériques de sortie** vus étaient :

↳ **LED** : visuel,

↳ **BUZZER** : sonore.

Simples à mettre en œuvre (commande en TOR, phase d'apprentissage) mais **limités** quant à l'information fournie...

Ici, on va utiliser un **LCD** (Liquid Cristal Display) **alphanumérique** :

↳ avec 2 **lignes** de 16 **caractères**.



Plus **compliqués** à mettre en œuvre mais plus intéressants pour **visualiser des informations** comme les messages ou les résultats numériques.

ELEMENTS SUR LE LCD IMPLANTE SUR LA CARTE PICDEM 2 PLUS (suite)

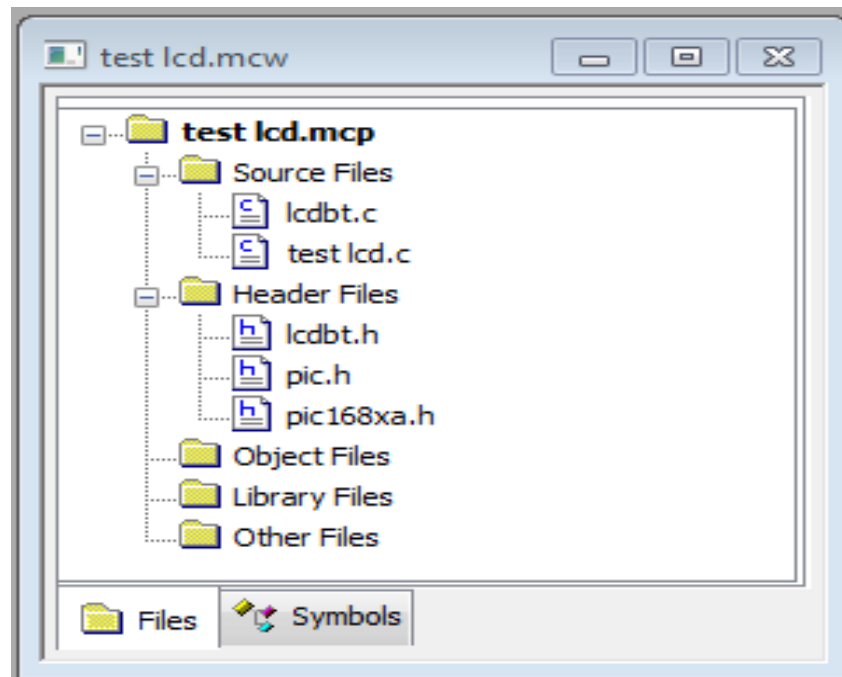
Le projet test lcd.mcp fourni

Utilisation du projet **test lcd.mcp** :

- pour donner une **trame** incomplète (et donc à terminer pour la rendre opérationnelle),
- et pour éviter de tout écrire en « partant de zéro ».

Par contre, il faut essayer de bien comprendre la structuration de ce projet au niveau :

- ↳ des **fichiers.c** associés : projet **multisources**,
- ↳ des **fichiers.h** associés : lisibilité, évolutivité,



- ↳ des **fonctions proposées** : principe du « **légo** ».

ELEMENTS SUR LE LCD IMPLANTE SUR LA CARTE PICDEM 2 PLUS (suite)

Le projet test lcd.mcp

Conseils généraux

Bien suivre la méthodologie **guidée** du texte du TP :

- **comprendre** les différentes étapes,
- **ne pas sauter** des étapes,
- **valider** son travail **au fur et à mesure** ...

Si elle est volontairement très guidée, c'est parce qu'elle pourra vous resservir de base pour implanter d'autres types d'interfaces, comme par exemple (liste non exhaustive) :

- un clavier (voir plus loin),
- un écran graphique (voir plus loin),
- un module ultrasons pour mesurer des distances (voir plus loin),
- un moteur pas à pas (voir plus loin),
- un module de reconnaissance vocale (voir plus loin),
- une caméra (projet robotique),
- un module de positionnement GPS,
- ou de façon générale, tout autre type de capteur ou d'actionneur (projet robotique)...

Les étapes d'implantation

Etape 1 : **Compréhension du hardware**

Cas idéal : ici, bien **comprendre** la documentation du **LCD**.

En voici un résumé :

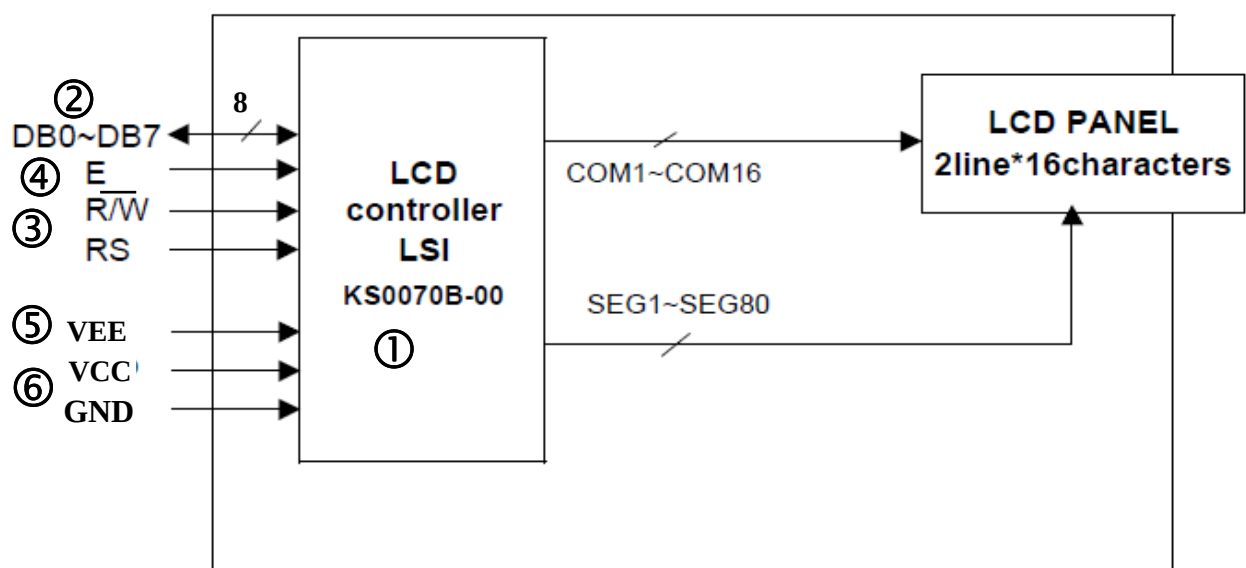


Figure III.3. (page 70 photocopié TP)

① le « cœur » du **LCD** comprenant entre autre :

deux registres accessibles en **lecture** ou **écriture** :

- ↳ un registre d'**instructions IR** ou d'**état** de taille **8** bits :
 - pour recevoir (en **écriture**) les différentes commandes :
initialisation du LCD, gestion du curseur, effacement, modes de fonctionnement ...
 - pour renvoyer (en **lecture**) une info sur l'état du **LCD**.
- ↳ un registre de **données DR** de taille **8** bits aussi :
 - pour recevoir (en **écriture**) le code **ASCII** des caractères à afficher sur le LCD.

Les étapes d'implantation (suite)

Etape 1 : (suite)

- ② les 8 broches **d'entrée/sortie DB0 à DB7** pour **échanger** les informations (contenu de **IR** et de **DR**) avec le **PIC**,
- ③ les pattes d'entrée **RS** (comme **Register Select**) et **R/ $\overline{W}$** (comme **Read/Write**) du **LCD** pour accéder à ces deux registres internes :

$\begin{array}{c} \text{RS} \\ \text{R}/\overline{\text{W}} \end{array}$	0	1
0	Ecriture dans IR	Ecriture dans DR
1	Lecture de IR	Lecture de DR

Figure III.4. (page 71 photocopié TP)

- ④ **après avoir positionné** les entrées **RS** et **R/ $\overline{W}$** suivant ce que l'on veut faire, il faut envoyer un « pulse » sur l'entrée **E**.

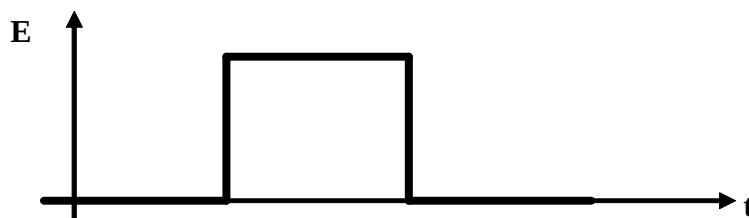


Figure III.5. (page 71 photocopié TP)

Les étapes d'implantation (suite)

Etape 1 : (suite)

Exemple de chronogrammes issus du data sheet du LCD

**Cycle d'écriture ($R/\overline{W} = 0$)
dans IR si RS=0 ou dans DR si RS=1**

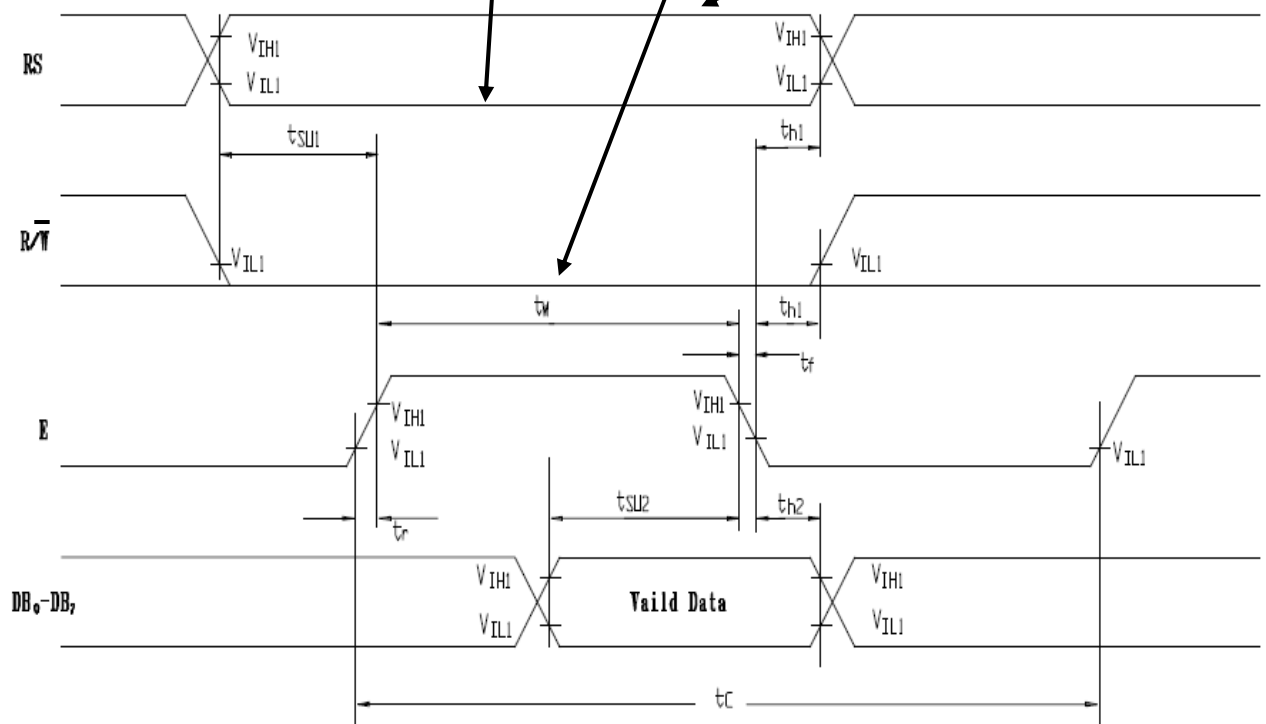


Figure III.6. (page 71 photocopié TP)

- ⑤ la patte d'entrée **VEE** du **LCD** pour régler le contraste,
- ⑥ les pattes d'entrée **VCC** et **GND** pour l'alimentation du **LCD**.

Les étapes d'implantation (suite)

Etape 2 :

Lecture du schéma d'implantation
Affectation des différentes broches
Initialisation des ports concernés du PIC

Schéma d'implantation du LCD

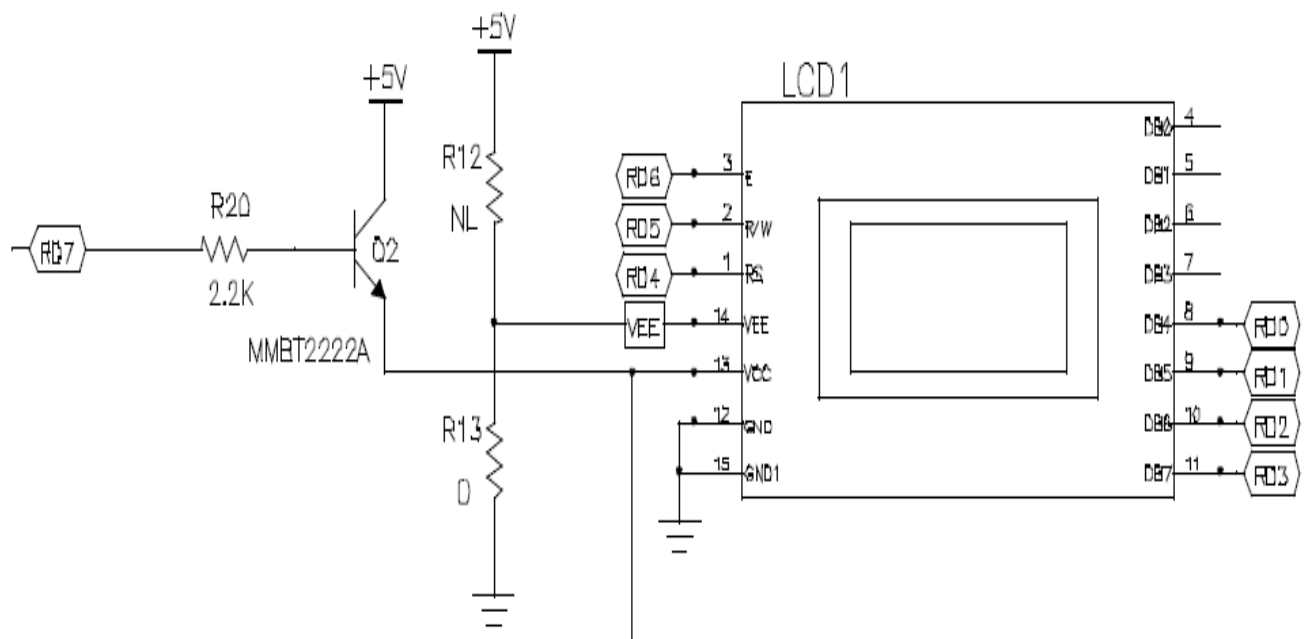


Figure III.7. (page 73 photocopié TP)

Pour faire un lien «**lisible**» entre le **hardware** et le **software** :

↳ attribuer un nom **symbolique** « **parlant** » aux différentes broches mise en œuvre par l'interface (cf.schéma ci-dessus): dans le fichier **lcdbt.h**.

↳ écrire les **fonctions d'initialisation** des ports du PIC concernés par l'interface (direction, valeurs initiales en sortie): dans le fichier **lcdbt.c**.

Résumé du cours PIC
(informations supplémentaires pendant manipulations TP 3)

Les étapes d'implantation (suite)

Etape 3 : Initialisation software du LCD

Respect de la **procédure préconisée** par le constructeur.

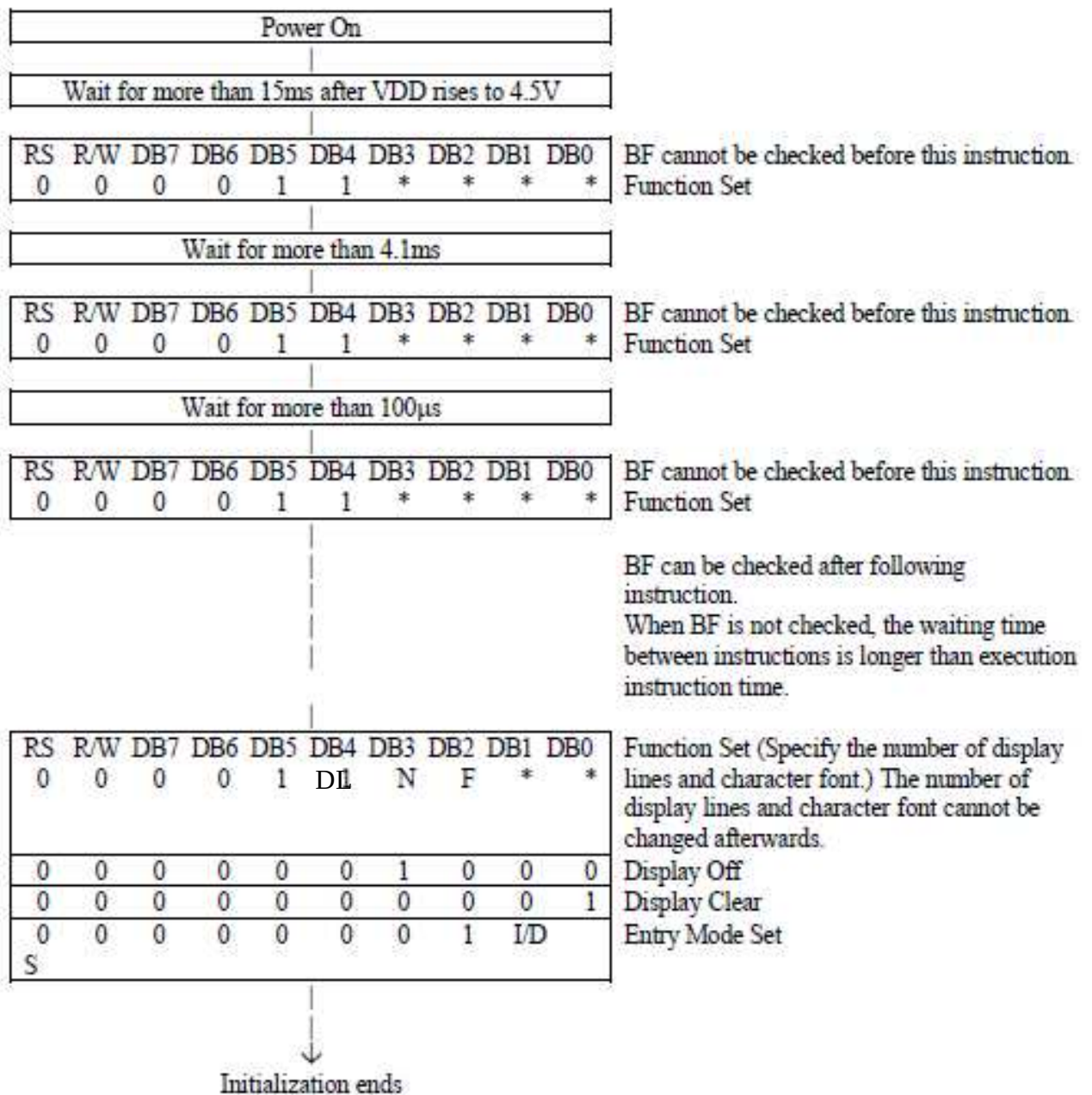


Figure III.8. (page 75 polycopié TP)

Les étapes d'implantation (suite)

Etape 3 : (suite)

C'est la fonction :

void **init_LCD** (void)

qui effectue l'initialisation du LCD.

Principales contraintes à respecter ici :

⇒ Temporisations à respecter entre chaque commande
(cf. fonction **tempo_N_ms** (int **ms**) : **TIMER0** sous pooling)

⇒ Echanges en mode **4 bits** :
(via les broches **bidirectionnelles** **RD0** à **RD3** du **PORTD**)

- d'abord le **quartet de poids fort**,
- puis le **quartet de poids faible**.

⇒ Faire le lien avec le **tableau des commandes** de la figure III.9. de la page 76 du polycopié de TP.

Objectifs de cette étape dans le TP :

Vérifier la conformité entre la fonction **init_LCD** déjà écrite et l'**organigramme** de la figure III.8. précédente.

Les étapes d'implantation (suite)

Etape 4 : **Ecriture d'un caractère sur le LCD**

Dans le fichier **lcdbt.c**, il faut écrire le corps de la fonction :

void print_char (char)

permettant d'envoyer (et donc d'afficher) un caractère dans le registre **DR** du **LCD**.

Conseils à respecter :

⇒ **Bien comprendre** et **s'inspirer** de la fonction
(aussi dans le fichier **lcdbt.c**)

void ecriture_commande (char)

déjà écrite pour vous et qui fait un peu la même chose mais dans le registre **IR** du **LCD**.

⇒ **Tester** la fonction **print_char** via le main du projet

test lcd.mcp

avant de passer à l'étape suivante.

Les étapes d'implantation (suite)

Etape 5:

Ecriture d'une chaîne de caractères sur le LCD

Toujours sans le fichier **lcdbt.c**, il faut écrire le corps de la fonction :

void **print_string** (char *)

permettant d'envoyer (et donc d'afficher) une chaîne de caractères sur le **LCD**.

Conseils à respecter :

⇒ **Utiliser bien sûr** la fonction :

void **print_char** (char)

écrite (et testée) à l'étape précédente.

⇒ **Tester** la fonction **print_string** via le main du projet

test lcd.mcp

avant de passer à l'étape suivante.

Les étapes d'implantation (suite)

Etape 6:

Bibliothèque de fonctions de gestion du curseur

Pour faciliter son utilisation ultérieure, il faut écrire un ensemble de fonctions (dans le fichier **lcdbt.c**) permettant de piloter plus convivialement le LCD.

(voir le cahier des charges de chacune de ces fonctions à la page 78 du polycopié de TP...)

- utilisation de la commande **SET DDRAM ADDRESS**

 ↪ pour choisir la position d'affichage sur l'écran

- utilisation de la commande **SHIFT**

 ↪ pour décaler le curseur ou l'écran

- utilisation de la commande **DISPLAY CLEAR**

 ↪ pour effacer l'écran

Conseils à respecter :

⇒ Tester **toutes ces fonctions** via le main du projet

test lcd.mcp

avant de passer à la suite du TP.